

Berry Documentation

version

Berry Lang

November 26, 2025

Contents

Welcome to berry's documentation!	1
Features	1
The Berry Script Language Reference Manual	3
Preface	3
Basic Information	3
1.1 Introduction	4
1.2 Start using	4
Get Interpreter	4
REPL environment	4
Hello World Program	4
More usage of REPL	5
Script file	5
1.3 Words	5
Comment	6
literal value	6
Numerical Literal Value	6
Boolean literal value	6
String literal	6
Nil literal value	7
identifier	7
Keywords	7
2. Types and Variables	8
2.1 Built-in type	8
2.1.1 Simple Type	8
nil	8
Integer type	8
Real Number Type	8
Boolean type	8
String	9
Function	9
Class	9
Instance	9
2.1.2 Class Type	10
List	10
Map	10
Range	10
Bytes	11
Comptr	11
Comobj	11
2.2 Variables	11
2.2.1 define variables	11
2.2.2 Scope and Life Cycle	12
nested scope	13
Variable Life Cycle	13

2.2.3 Type of variable	13
3. Expression	13
3.1 Basics	13
Operators and expressions	13
Operator combination expression	14
operand type	14
3.1.1 Priority and associativity	14
Use brackets to increase priority	15
3.2 Operator	15
3.2.1 Arithmetic Operators	15
3.2.2 Relational operators	15
logical operators	16
Bitwise Operator	17
Assignment operator	18
Simple assignment operator	18
Compound Assignment Operator	18
Walrus Assignment Operator	19
domain operator and subscript operator	19
Conditional Operator	19
Nested Condition Operators	20
Priority of conditional operators	20
Concatenation Operator	20
+ operator	20
. . operator	20
* operator for String Repetition	21
4. Statement	21
4.1 Simple sentence	22
4.1.1 Expression statement	22
Block	22
do Statement	23
conditional statement	23
if Statement	23
Iteration Statement	24
while Statement	24
for Statement	25
for Principle of Statement	26
Jump Statement	26
break Statement	27
continue Statement	27
import Statement	27
Exception Handling	28
Raise an exception	28
Outliers	28
Catch exceptions	29
Error Handling Patterns	29
5. Function	30

5.1 Basic Information	30
5.1.1 Function Definition	30
Named Function	30
Anonymous Function	31
Call function	31
Forward call	32
Recursive call	32
Anonymous function call	33
Formal and actual parameters	33
Function with variable number of arguments (vararg)	33
Calling a function with dynamic number of arguments	33
Functions and local variables	34
return Statement	34
closure	35
Basic Concepts	35
Use of closures	35
Lazy evaluation	35
Function private communication	36
Generate multiple functions	36
Simulate private members	36
Cache result	36
Binding free variables	36
Create closure in loop	37
Lambda expression	38
6. Object Oriented Function	38
6.1 Class and instance	39
6.1.1 Class declaration	39
Instantiate	39
Method and self Parameters	39
Synthetic methods	40
Class Variables static	40
Class Methods static	40
Constructor and Destructor	41
Constructor	41
Destructor	41
Class inheritance	41
Method Overload	42
Common method overload	42
Operator Overloading	42
Overload of compound assignment operator	43
Instance	43
Access base class object	43
7. Libraries and Modules	45
7.1 Basic library	45
7.1.1 Built-in function	45
print function	45

input function	45
classname function	46
classof function	46
str function	46
number function	46
int function	47
real function	47
bool function	47
type function	48
size function	49
super function	49
assert function	49
compile function	49
module function	50
issubclass function	50
isinstance function	50
call function	51
list Class	51
list method (Constructor)	51
tostring method	51
concat method	51
push method	51
pop method	52
insert method	52
remove method	52
item method	52
setitem method	53
size method	53
resize method	53
iter method	53
find method	53
reverse method	53
keys method	53
copy method	53
map Class	54
map method (Constructor)	54
tostring method	54
insert method	54
remove method	54
item method	54
setitem method	54
size method	54
contains method	54
find method	55
keys method	55
file Class	55

open	function	55
write	method	55
read	method	56
readbytes	method	56
readlines	method	56
seek	method	56
tell	method	56
size	method	56
flush	method	56
close	method	56
range	Class	56
bytes	Class	56
bytes	method (Constructor)	57
size	method	58
tostring	method	58
tohex	method	58
fromhex	method	58
clear	method	58
resize	method	58
Concatenation + and ..	methods	58
reverse	method	59
bytes access []	method	59
range access []	method	59
copy	method	60
get, geti	methods	60
set, seti	methods	60
add	method	60
asstring	method	61
fromstring	method	61
bits manipulation	setbits, getbits methods	61
base64 encode	tob64 method	61
base64 decode	fromb64 method	61
getfloat and setfloat	methods	61
_buffer	method	62
_change_buffer	method	62
Expansion Modules		62
JSON Module		62
load	function	62
dump	function	63
Math Module		63
pi	constant	64
abs	function	64
ceil	function	64
floor	function	64
round	function	64
min	function	65

max function	65
sin function	65
cos function	65
tan function	66
asin function	66
acos function	66
atan function	66
sinh function	67
cosh function	67
tanh function	67
sqrt function	67
exp function	67
log function	68
log10 function	68
deg function	68
rad function	68
pow function	68
srand function	69
rand function	69
Time Module	69
time function	69
dump function	69
clock function	70
String Module	70
count function	70
split function	70
find function	70
startswith function	70
endswith function	70
hex function	70
byte function	71
char function	71
tolower function	71
toupper function	71
tr function	71
replace function	71
escape function	71
format function	71
F-strings (Formatted String Literals)	72
Module os	73
os.getcwd function	73
os.chdir function	73
os.mkdir function	73
os.remove function	73
os.listdir function	73
os.system function	73

os.exit function	74
Module os.path	74
os.path.isdir function	74
os.path.isfile function	74
os.path.exists function	74
os.path.split function	74
os.path.splitext function	74
os.path.join function	74
Module global	75
Module introspect	75
Module solidify	76
8. Advanced features	76
8.1 strict mode	76
Mandatory var for local variables	76
No overriding of builtins	76
Multiple var with same name not allowed in same scope	77
No hiding of local variable from outer scope	77
8.2 Virtual members	77
module undefined	77
implicit call of member()	78
implicit call of setmember()	78
Exception handling	79
Specifics for classes	79
Specifics for modules	79
Example	79
8.3 How-to package a module	80
Behavior of import	80
Packaging a module	81
Package a singleton (monad)	81
8.4 Solidification	82
solidify module	82
Solidification of functions	83
Solidification of classes	84
Solidification of modules	86
Limitations of solidification	88
9. FFI	88
9.1 Basics	88
9.1.1 Virtual Machine	88
9.1.2 Virtual Stack	89
Operate Virtual Stack	89
Index and stack size	89
Get value from stack	90
Native function	90
Define native function	90
Use native function	90
Complete example	91

Types and Functions	92
Type	92
Functions and Macros	93
Compile-time construction technology	102
Use command	102
output path	102
Compile-time string table	103
Use constant string	103
Construct object at compile time	103
Construct Map at Compile Time	105
Compile-time construction class	105
Building Module at Compile Time	105
Construct Built-in Domain at Compile Time	106
Grammar Definition	106
Compiler Interpreter	107
1. Overview	107
2. Porting	108
3. Platform Support	108
Porting Guide	108
Configuration File	108
<i>berry_conf.h</i> File	108
Configuration Macro Switch	108
<i>berry_port.c</i> File	110
Making a Native Function	111
Instantiate a list object in a native function	113
Berry Introduction (in 20 minutes or less)	113
Hello, Berry	114
Your free calculator is here	114
Defining a function	115
The knights who say nil	116
Formatting strings	116
Evolving into a Greeter	116
Creating a greeter object	117
Subclasses	117
Greetings everyone!	118
Comments	119
Maps	119
If statements and basic expressions	120
Cycling and Looping	121
Functions and arguments	121
Closures	122
Consider Yourself Introduced	123
Extra	123
Language fast reference PDF	124
Memory Requirements	124
RAM required	124

ROM/Flash required	124
Explanation	124
API documentation	124
Index	161

Welcome to berry's documentation!

About

- [genindex](#)
- [search](#)

Berry is a ultra-lightweight dynamically typed embedded scripting language. It is designed for lower-performance embedded devices. The Berry interpreter-core's code size is less than 40KiB and can run on less than 4KiB heap (on ARM Cortex M4 CPU, Thumb ISA and ARMCC compiler).

The interpreter of Berry include a one-pass compiler and register-based VM, all the code is written in ANSI C99. In Berry not every type is a class object. Some simple value types, such as int, real, boolean and string are not class object, but list, map and range are class object. This is a consideration about performance. Register-based VM is the same meaning as above.

Berry has the following advantages:

- **Lightweight:** A well-optimized interpreter with very little resources. Ideal for use in microprocessors.
- **Fast:** optimized one-pass bytecode compiler and register-based virtual machine.
- **Powerful:** supports imperative programming, object-oriented programming, functional programming.
- **Flexible:** Berry is a dynamic type script, and it's intended for embedding in applications. It can provide good dynamic scalability for the host system.
- **Simple:** simple and natural syntax, support garbage collection, and easy to use FFI (foreign function interface).
- **RAM saving:** With compile-time object construction, most of the constant objects are stored in read-only code data segments, so the RAM usage of the interpreter is very low when it starts.

Features

Base Type

- Nil: `nil`
- Boolean: `true` and `false`
- Numerical: Integer (`int`) and Real (`real`)
- String: Single quotation-mark string and double quotation-mark string
- Class: Instance template, read only
- Instance: Object constructed by class
- Module: Read-write key-value pair table
- List: Ordered container, like `[1, 2, 3]`
- Map: Hash Map container, like `{ 'a': 1, 2: 3, 'map': {} }`
- Range: include a lower and a upper integer value, like `0..5`

Operator and Expression

- Assign operator: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`
- Relational operator: `<`, `<=`, `=`, `!=`, `>`, `>=`
- Logic operator: `&&`, `||`, `!`
- Arithmetic operator: `+`, `-`, `*`, `/`, `%`

Welcome to berry's documentation!

- Bitwise operator: &, |, ~, ^, <<, >>
- Field operator: .
- Subscript operator: []
- Connect string operator: +
- Conditional operator: condition ? val_true : val_false
- Brackets: ()
- Bytes buffer support

Control Structure

- Conditional statement: if elif else end
- Iteration statement: while, for
- Jump statement: break, continue

Function

- Local variable and block scope
- Return statement
- Nested functions definition
- Closure based on Upvalue
- Anonymous function
- Lambda expression

Class

- Inheritance (only public single inheritance)
- Method and Operator Overload
- Constructor method
- Destructive method

Module Management

- Built-in module that takes almost no RAM
- Extension module support: script module, bytecode file module and shared library (like *.so, *.dll) module
- Ability to solidify code, classes and modules in flash to reduce RAM usage
- Optional Regex support
- Optional [LVGL](#) mapping

GC (Garbage collection)

- Mark-Sweep GC

Exceptional Handling

- Throw any exception value using the raise statement
- Multiple catch mode

Bytecode file support

- Export function to bytecode file
- Load the bytecode file and execute

Native C interface

- Can be easily embedded as a library in existing code like [Tasmota](#)
- Optional [easy mapping](#) to call C code from Berry

The Berry Script Language Reference Manual

Preface

A few years ago, I tried to port the Lua scripting language to STM32F4 microcontroller, and then I experienced a Lua-based firmware on ESP8266: NodeMCU. These experiences made me experience the convenience of script development. Later, I came into contact with some scripting languages, such as Python, JavaScript, Basic, and MATLAB. At present, only a few languages are suitable for transplanting to the microcontroller platform. I used to pay more attention to Lua because it is a very compact embedded scripting language, and its design goal is to be embedded in the host program. However, for the microcontroller, the Lua interpreter may not be small enough, and I cannot run it on a 32-bit microcontroller with a relatively small memory. To this end, I started to read Lua code and developed my own scripting language-Berry on this basis.

This is an ultra-lightweight embedded scripting language, it is also a multi-paradigm dynamic language. Supports object-oriented, process-oriented and functional (less support) styles. Many aspects of this language refer to Lua, but its syntax also borrows from the design of other languages. If the reader already knows a high-level language, Berry's grammar should be very easy to grasp: the language has only some simple rules and a very natural scope design.

The main application scenario I consider is embedded systems with low performance. The memory of these systems may be very small, so it is very difficult to run a full-featured scripting language. This means that we may have to make a choice. Berry may only provide the most commonly used and basic core functions, while other unnecessary features are only used as optional modules. This will inevitably lead to the language's standard library being too small, even the language There will also be uncertain designs (such as the implementation of floating-point numbers and integers, etc.). The benefits of these trade-offs are more room for optimization, while the disadvantage is obviously the inconsistency of language standards. Berry's interpreter refers to the implementation of Lua interpreter, which is mainly divided into two parts: compiler and virtual machine. Berry's compiler is a one-pass compiler to generate bytecode. This solution does not generate an abstract syntax tree, so it saves memory. The virtual machine is a register type. Generally speaking, the register type virtual machine is more efficient than the stack type virtual machine. In addition to implementing language features, we also hope to optimize the memory usage and operating efficiency of the interpreter. At present, the performance indicators of Berry interpreter are not comparable to those of mainstream languages, but the memory footprint is very small.

It wasn't until later that I learned about the MicroPython project: a simplified Python interpreter designed for microcontrollers. Nowadays, Python is very hot, and this Python interpreter designed for microcontrollers is also very popular. Compared with the current mature technology, Berry is a new language without a sufficient user base. Its advantage is that it is easy to master the grammar and may have advantages in terms of resource consumption and performance.

If you need to port the Berry interpreter, you need to ensure that the compiler you use provides support for the C99 standard (I previously fully complied with C89, and some optimization work later made me give up this decision). At present, most compilers provide support for C99, and the common compilers such as ARMCC (KEIL MDK), ICC (IAR) and GCC in the development of ARM processors also support C99. This document introduces Berry's grammar rules, standard library and other facilities, and finally guides readers to transplant and extend Berry. This document does not explain the implementation mechanism of the interpreter, and may be explained in other documents if we have time.

The author's level is limited, and the writing is in a hurry. If there are any omissions or errors in the article, I hope the readers will not hesitate.

Guan Wenliang

April 2019

Basic Information

Table of Content

Placeholder for table of contents

0

1.1 Introduction

Berry is an ultra-lightweight, dynamically typed embedded scripting language designed for resource-constrained environments. The language primarily supports procedural programming, with additional support for object-oriented and functional programming paradigms. Berry's key design goal is to run efficiently on embedded devices with very limited memory, making the language highly streamlined while maintaining rich scripting capabilities.

1.2 Start using

Get Interpreter

Readers can go to the project's GitHub page <https://github.com/berry-lang/berry> to get the source code of the Berry interpreter. Readers need to compile the Berry interpreter by themselves. The specific compilation method can be found in the README.md document in the root directory of the source code, which can also be viewed on the GitHub page of the project.

First, you must install software such as GCC, git, and make. If you do not use version control, you can download the source code directly on GitHub without installing git. Readers can use search engines to retrieve information about installing these tools. Users on Linux and macOS systems should also install the GNU Readline library [1]. Use the git command [2] to clone the interpreter source code from the remote repository to your local machine:

```
git clone https://github.com/berry-lang/berry
```

Enter the berry directory and use the make command to compile the interpreter:

```
cd berry
make
```

Now you should be able to find the executable file of the interpreter in the *berry* directory (in Windows systems, the file name of the interpreter program is "*berry.exe*", while in Linux and macOS systems the file name is "*berry*"), you can run the executable file directly [3] To start the interpreter. In Linux or macOS, you can use the command `sudo make install` to install the interpreter, and then you can start the interpreter with the *berry* command in the terminal.

REPL environment

REPL (Read-Eval-Print Loop) is an interactive interpreter environment that allows you to execute Berry code interactively. This mode consists of four elements: **Read** user input from the terminal; **Eval** (evaluate) by compiling and executing the input code; **Print** the result of the evaluation; **Loop** back to read the next input.

Start the interpreter without arguments (enter *berry* in the terminal or double-click *berry.exe* on Windows) to enter REPL mode. You will see an interface similar to this:

```
Berry 1.0.0 (build in Feb 1 2022, 13:14:04)
[GCC 8.1.0] on Windows (default)
>
```

The first two lines of the interface display the version, compilation time, compiler and operating system of the Berry interpreter. The symbol ">" in the third line is called the prompt, and the cursor is displayed behind the prompt. When using the REPL mode, after inputting the source code, pressing the "Enter" key will immediately execute the code and output the result. Press the Ctrl + C key combination to exit the REPL. In the case of using the Readline library, use the Ctrl + D key combination to exit the REPL when the input is empty. Since there is no need to edit script files, REPL mode can be used for quick development or idea verification.

Hello World Program

Take the classic "Hello World" program as an example. Enter `print('Hello World')` in the REPL and execute it. The results are as follows:

```
Berry 1.0.0 (build in Feb 1 2022, 13:14:04)
[GCC 8.1.0] on Windows (default)
> print('Hello World')
Hello World
>
```


Welcome to berry's documentation!

The interpreter output the text "Hello World". This line of code realizes the output of the string 'Hello World' by calling the print function. In REPL, if the return value of the expression is not nil, the return value will be displayed. For example, entering the expression `1 + 2` will display the calculation result 3:

```
> 1 + 2
3
```

Therefore, the simplest "Hello World" program under REPL is to directly enter the string 'Hello World' and run:

```
> 'Hello World'
Hello World
```

More usage of REPL

You can also use the interactive mode of the Berry interpreter as a scientific calculator. However, some mathematical functions cannot be used directly. Instead, use the `import math` statement to import the mathematical library, and then use the functions in the mathematical library. "math." as a prefix, for example `sin` function should be written as `math.sin`:

```
> import math
> math.pi
3.14159
> math.sin(math.pi / 2)
1
> math.sqrt(2)
1.41421
```

Script file

Berry's script file is a file that stores Berry code, and the script file can be executed by an interpreter. Usually, the script file is a text file with the extension ".be". The command to execute the script using the interpreter is:

```
berry script_file
```

`script_file` is the file name of the script file. Using this command will run the interpreter to execute the Berry code in the `script_file` script file, and the interpreter will exit after execution.

If you want the interpreter to enter the REPL mode after executing the script file, you can add the `-i` parameter to the command to call the interpreter:

```
berry -i script_file
```

This command will first execute the code in the `script_file` file and then enter the REPL mode.

1.3 Words

Before introducing Berry's syntax, let's take a look at a simple code (you can run this code in REPL mode):

```
def func(x) # a function example
    return x + 1.5
end
print('func(10) =', func(10))
```

This code defines a function `func` and calls it later. Before understanding how this code works, we first introduce the syntax elements of the Berry language.

In the above code, the specific classification of grammatical elements is: `def`, `return` and `end` are keywords of Berry language; and "`# a function example`" in the first line is called a comment; `print`, `func` and `x` are some identifiers, they are usually used to represent a variable; `1.5` and `10` these numbers are called numerical literals, they are equivalent to the numbers used in daily life; `'func(10) ='` It is a string literal, they are used in places where you need to represent text; `+` is an addition operator, here the addition operator can be used to add the variable `x` and the value `1.5`.

The above classification is actually done from the perspective of a lexical analyzer. Lexical analysis is the first step in Berry source code analysis. In order to write the correct source code, we start with the most basic lexical introduction.

Comment

Comments are some text that does not generate any code. They are used to make comments in the source code and be read by people, while the compiler will not interpret their content. Berry supports single-line comments and cross-line block comments. Single-line comments start with the character “#” until the end of the newline character. The quick note starts with the text “#-” and ends with the text “-#”. The following is an example of using annotations:

```
# This is a line comment
#- This is a
  block comment
-#
```

Similar to C language, quick comments do not support nesting. The following code will terminate the analysis of comments at the first “-#” text:

```
#- Some comments -# ... -#
```

literal value

The literal value is a fixed value written directly in the source code during programming. Berry's literals are integers, real numbers, booleans, strings, and nil. For example, the value 34 is an integer literal.

Numerical Literal Value

Numerical literals include **Integer** literals and **Real number** (real) literals.

```
40 # Integer literal
0x80 # Hexadecimal literal (integer)
3.14 # Real literal
1.1e-6 # Real literal
```

Numeric literals are written similarly to everyday notation. Berry supports hexadecimal integer literals. Hexadecimal literals start with the prefix 0x or 0X, followed by a hexadecimal number.

Boolean literal value

Boolean values are used to represent true and false logical states. You can use the keywords true and false to represent Boolean literals.

String literal

A string is a piece of text, and its literal writing is to use a pair of ' or " to surround the string text:

```
'this is a string'
"this is a string"
```

String literals provide some escape sequences to represent characters that cannot be input directly. The escape sequence starts with the character ' ', and then follows a specific sequence of characters to achieve escape. The escape sequences specified by Berry are

Escape character	significance	Escape character	significance
\a	Ring the bell	\b	Backspace
\f	Form feed.	\n	Newline
\r	Carriage return	\t	Horizontal tab
\v	Vertical tab	\	Backslash
'	apostrophe	"	Double quotes
\?	question mark	\0	Null character

Escape character sequence

Escape sequences can be used in strings, for example

```
print('escape character LF\n\tnew line')
```

Welcome to berry's documentation!

The result of the operation is

```
escape character LF
new line
```

You can also use generalized escape sequences, in the form of `\x` followed by 2 hexadecimal digits, or `\` 3 octal digits, using this escape sequence can represent any character. Here are some examples of using the ASCII character set:

```
'\115' #- 'M' -#
'\x34' #- '4' -#
'\064' #- '4' -#
```

Nil literal value

Nil represents a null value, and its literal value is represented by the keyword `nil`.

identifier

An identifier is a user-defined name, which starts with an underscore or letter, followed by any combination of underscores, letters, or numbers. Like most languages, Berry is case-sensitive, so identifier A and identifier a are resolved as two different identifiers.

```
a
TestVariable
Test_Var
_init
baseClass
—
```

Keywords

Berry reserves the following tokens as language keywords:

```
if elif else while for def
end class break continue return true
false nil var do import as static
try except raise
```

The specific usage of keywords will be introduced in the relevant chapters. Note that keywords cannot be used as identifiers. Because Berry is case sensitive, `If` can be used for identifiers.

[1] For GNU Readline, the installation command for the Debian series of Linux distributions is `sudo apt install libreadline-dev`, and the installation command for the RedHat series of Linux distributions is `yum install readline-devel`, under macOS The installation command is `brew install readline`. In addition, it is easy to find GNU Readline documentation and related materials in search engines.

[2] commands need to be used in the “command line interface” after the preparation work is completed. The command line environment in Windows systems is usually a command prompt (CMD) window, while the command line environment in Unix-like systems is usually Called “Terminal” (Terminal). This is not very accurate, but it will not be expanded here.

[3] In Windows, you can directly double-click to run the executable file. In Linux or macOS, use the terminal to run it. You can also run the interpreter in the Windows command prompt window. Please refer to the README.md file for specific usage.

2. Types and Variables

Table of Content

Placeholder for table of contents

0

Type is an attribute of data, which defines the meaning of the data and the operations that can be performed on the data. Types can be divided into built-in types and user-defined types. Built-in types refer to some basic types built into the Berry language, among which types that are not based on class definitions are called **Simple type**. Types based on class definitions are called **Class type**, some of the built-in types are class types, and user-defined types are also class types.

2.1 Built-in type

2.1.1 Simple Type

`nil`

The Nil type is the null type, which means that the object has an invalid value, or it can be said that the object has no meaningful value. This is a very special type. Although we might say that a variable is `nil`, in fact the nil type has no value, so what we are talking about here is that the type of the variable is nil (not a value).

The default value of a variable before assignment is `nil`. This type can be used in logic operations. In logic operations, `nil` is equivalent to `false`.

Integer type

The integer type (integer) represents a signed integer, referred to as an integer. The number of bits of the integer represented by this type depends on the specific implementation, and usually consists of a 32-bit signed integer on a 32-bit platform. Integer is an arithmetic type and supports all arithmetic operations. Pay attention to the value range of integers when using this type. The typical value range of 32-bit signed integers is between `-2147483648` and `2147483647`.

Any value can be converted to `int` using the `int()` function; however `int(nil) == nil`. If the argument is an instance and if it contains a member `toint()` it will be called and the return value converted to `int`.

Real Number Type

The real type (real), to be precise, is a floating-point type. Real number types are usually implemented as single-precision floating-point numbers or double-precision floating-point numbers. The real number type is also an arithmetic type. Compared with the integer type, the real number type has higher precision and a larger value range, so this type is more suitable for mathematical calculations. It should be noted that the real number type is actually a floating point number, so there are still precision problems. For example, it is not recommended to compare two values of type `real` for equality.

When integers and real numbers participate in operations at the same time, the integers are usually converted to real numbers.

Boolean type

The Boolean type (boolean) is used for logical operations. It has two values `true` and `false`, which represent the two truth values (true and false) in logic and Boolean algebra. The Boolean type is mainly used for conditional judgment. The operands and return values of logical expressions and relational expressions are all boolean types, and statements such as `if` and `while` all use boolean types as conditional checks.

In many cases, non-boolean values can also be used as boolean types. This is because the interpreter will implicitly convert the parameters. This is also the reason that conditional check expressions such as `if` statements can use any type of parameters. The rules for converting various types to Boolean types are:

- `nil`: converts to `false`

- **Integer:** `0` converts to `false`, all other values convert to `true`
- **Real number:** `0.0` converts to `false`, all other values convert to `true`
- **String:** empty string `""` converts to `false`, all other strings convert to `true`
- **Bytes:** empty buffer converts to `false`, non-empty buffer converts to `true`
- **Comobj** and **Comptr:** `NULL` pointer converts to `false`, all other values convert to `true`
- **Instance:** if the instance has a `tobool()` method, its return value is used; otherwise converts to `true`
- **All other types:** convert to `true`

Any value can be converted to `bool` using `bool()` function.

String

A string is a sequence of characters. In terms of storage, Berry divides strings into long strings and short strings. There is only one instance of the same short character string in memory, and all short character strings are linked in a hash table. This design helps to improve the performance of string equality comparison and can reduce memory usage. Since the use frequency of long strings is low, and the overhead of hash operation is quite high, they are not linked to the hash table, so there may be multiple identical instances in the memory. The string is read-only after it is created. Therefore, “modifying” the string will generate a new string, and the original string will not be modified.

Berry does not care about the format or encoding of characters. For example, the string `'abc'` is actually the ASCII code of the characters `'a'`, `'b'` and `'c'`. Therefore, if there are wide characters in the string (the character length is greater than 1 byte), the number of characters in the string cannot be directly counted. In fact, using the `size()` function can only get the number of bytes in the string. In addition, in order to facilitate interaction with the C language, Berry's string always ends with `'\0'` characters. This feature is transparent to the Berry program.

The string type can be compared in size, so it can be used in relational operations.

Function

A function is a piece of code that is encapsulated and available for call, generally used to implement a specific function. Function is actually a big category, which includes several subtypes such as closures, native functions, and native closures. For Berry code, all function subtypes have the same behavior. Functions belong to the first type of value in Berry, so they can be passed as values. In addition, it can be directly used in expressions through the “literal” form of “anonymous functions”.

A function is a read-only object and cannot be modified once defined. You can compare whether two functions are equal (whether they are the same function), but the function type cannot be compared. **Native function** (native function) and **Native closure** (native closure) refer to functions and closures implemented in C language. One of the main purposes of native functions and native closures is to provide functions that the Berry language does not provide, such as IO operations and low-level operations. If a piece of code is used frequently and has performance requirements, it is also recommended to rewrite it as a native function or native closure.

Class

In object-oriented programming, a class is an extensible program code template. Classes are used to create instance objects, so the class can be said to be the “type” of the instance. All instance objects are of type `instance` and they all have a corresponding class, which is called instance **Class type**. To put it simply, a class is a value representing the type of an instance object, and a class is an abstraction of the characteristics of an instance. A class is also a read-only object, once defined, it cannot be modified.

Classes can only compare equals and unequals, but cannot compare sizes.

Instance

An instance is a materialized object generated by a class, and the process of generating an instance from a class is called `Instantiate`. In object-oriented programming, “instance” is usually synonymous with “object”. However, in order to distinguish from non-instance objects, we do not use the term “object” alone, but use “instance” or “instance object”. Berry instances are always allocated dynamically and need to be used with a garbage collector. In addition to memory allocation, the process of instantiation also needs to initialize the instance, this process is completed by `Constructor`. In addition, you can complete the destruction of the object through `Destructor` before reclaiming the object's memory.

In the internal implementation, the instance will contain a reference to the class, and the instance itself only stores member variables and not methods.

2.1.2 Class Type

Some of the built-in types are class types, they are `list`, `map` and `range`. Unlike custom types, built-in class types can be constructed using literals, for example `[1, 2, 3]` is a literal of type `list`.

List

The `List` class is a container that provides support for list data types. Berry's list is an ordered collection of elements, and each element in the list has a unique integer index, and each element can be accessed directly according to the index. List supports inserting or deleting elements at any position, and the element can be of any type. In addition to using indexes, you can also use iterators to access elements in the list.

The implementation of `List` is a dynamic array, and this data structure has good random access performance. The efficiency of adding and deleting elements at the end of the list is very high, but the efficiency of adding and deleting elements in the middle of the list is low.

The literal initialization method of the `List` container is to use a list of objects surrounded by square brackets, and multiple objects are separated by commas, for example:

```
[  
  'string'  
  0, 1, 2, 'list'
```

Operations: see chapter 7.

Map

`Map` is also a kind of container, `map` is a collection of key-value pairs, and each possible key appears at most once in the collection. The `Map` container provides the following basic operations:

- Add key-value pairs to the collection
- Remove key-value pairs from the collection
- Modify the value corresponding to an existing key
- Find the corresponding value by key

`Map` is implemented using a hash table and has high search efficiency. The operation of adding and deleting key-value pairs will consume more time if "re-hashing" occurs.

The `Map` container can also be initialized using literal values, written in curly braces to enclose a list of key-value pairs, separated by colons between keys and values, and separated by commas between key-value pairs. E.g:

```
{  
  'str': 'hello'  
  'str': 'hello', 'int': 45, 78: nil
```

Operations: see chapter 7.

Range

The `Range` container represents an integer range, which is usually used to iterate in an integer range. This type has a `__lower__` member and `__upper__` member, which represent the lower and upper bounds of the range, respectively. The literal value of `Range` is a pair of integers connected using the `..` operator:

```
0 .. 10  
-5 .. 5
```

When the `Range` class is used for iteration, the elements of the iteration are all integer values from the lower bound to the upper bound, including boundary values. For example, the iteration result of `0..5` is:

```
0 1 2 3 4 5
```

Therefore, it should be noted that for a range of $x .. (x+n)$, the number of iterations is $n + 1$. A common construct to iterate through elements of a list by item is:

```
for i: 0..size(l)-1
```

Open range: if you omit the last range, it is implicitly replaced with MAXINT.

```
> r = 10..  
> r  
(10..9223372036854775807)
```

Bytes

Bytes object denote a bytes buffer which can be used to manipulate bytes buffers or to read/write some C memory areas or structures.

See Chapter 7.

Comptr

Comptr (C pointer) is a type that represents a pointer to memory, typically used for interfacing with C code. It can hold memory addresses and is used for low-level memory operations. When used in boolean context, a NULL pointer (0) converts to `false`, otherwise it converts to `true`.

Comobj

Comobj (C object) is a type that represents a C object or structure. Like `comptr`, it's primarily used for C language interoperability. When used in boolean context, a NULL object converts to `false`, otherwise it converts to `true`.

2.2 Variables

A variable is a storage space with a name, and the data or information stored in the storage space is called the value of the variable. Variable names are used to refer to variables in source code. In different scopes, a variable name can bind multiple independent variables, but variables have no aliases. The value of the variable can be accessed or changed at any time during the running of the program. Berry is a dynamically typed language, so the type of variable value is determined at runtime, and the variable can store any type of value.

2.2.1 define variables

The first way to define a variable is to use an assignment statement to assign a value to a new variable name:

```
'var' = expression
```

variable is the name of the variable, and the variable name is an identifier (see section identifier). **expression** is the expression to initialize the variable.

```
a = 1  
b = 'str'
```

However, this method of defining variables has some limitations. Take the following code as an example:

```
i = 0  
do  
    i = 1  
    print(i) # 1  
end  
print(i) # 1
```

The `do` statement in the routine constitutes the inner scope. We modified the value of the variable `i` at line 3, and the value of `i` is still 1 after leaving the inner scope at line 6. If we want the variable `i` of the inner scope to be an independent variable, the method of defining the variable by directly assigning to the new variable name cannot meet the requirement, because the identifier `i` already exists in the outer scope. In this case, the variable can be defined by the `var` keyword:

```
'var' variable  
'var' variable = expression
```

There are two ways of using `var` to define a variable: The first is to follow the variable name **variable** after the keyword `var`, in this case the variable will be initialized to `nil`, and the other is written in The variable is initialized at the same time as the variable is defined. In this case, an initial value expression **expression** is required. Using `var` to define a variable has two possible results: if the current scope does not define the variable of **variable**, define and initialize the variable, otherwise it is equivalent to reinitialize the variable. Therefore, the variable defined with `var` will shield the variable with the same name in the outer scope.

Now we change the previous example to use the `var` keyword to define variables:

```
i = 0
do
  var i = 1
  print(i) # 1
end
print(i) # 0
```

From the modified routine, it can be found that the value of the variable `i` in the inner scope is 1, and its value in the outer scope is 0. This proves that after using the `var` keyword, a new variable `i` is defined in the inner scope and the variable with the same name in the outer scope is blocked. After the inner scope ends, the identifier `i` is once again bound to the variable `i` in the outer scope.

When using the `var` keyword to define a variable, you can also use a list of multiple variable names, separated by commas. You can also initialize one or more variables when defining variables:

```
var a = 0, b, c = 'test'
```

2.2.2 Scope and Life Cycle

As mentioned earlier, variable names can be bound to multiple variable entities (storage spaces), and variable names are bound to only one entity at each position. The entity bound by the variable name needs to be determined according to the position where the variable name appears.

Scope refers to the code area where the name and the entity are uniquely bound. Outside the scope, the name may be bound to other entities, or not bound to any entity. The entity is only visible in the scope bound to the name, that is, the variable is only valid in its scope. A code block (see block) is a scope. A variable is only available inside the block, and names in different blocks may bind different variable entities. The following example demonstrates the scope of variables:

```
var i = 0
do
  var j = 'str'
  print(i, j) # 0 str
end
# The variable j is not available here
print(i) # 0
```

The names `i` and `j` are defined in this routine. The name `i` is defined outside the `do` sentence, and the name defined in the outermost block has **Global scope** (global scope). The name with global scope is available in the entire program after customization. The name `j` is defined in the block in the `do` sentence, and the name of this type of definition in the non-outermost block has **Local scope** (local scope). A name with a local scope cannot be accessed outside the scope.

Berry has some built-in objects, which are all in the global scope. However, built-in objects and global variables defined in scripts are not in the same global scope. Built-in objects actually belong to **Built-in scope** (built-in scope). The scope is globally visible as the ordinary global scope, but can be covered by the ordinary global scope. Built-in objects include functions and classes in the standard library. These objects include `print` functions, `type` functions, and `map` classes. Different from other scopes, the variables in the built-in scope are read-only, so "assignment" to the variables in the built-in scope actually defines a variable with the same name in the global scope, which overrides The symbols in the built-in scope.

nested scope

Nested scope means that the scope contains another scope. We call the contained scope **Inner scope**, and the scope that contains the inner scope **Outer scope**. The name defined in the outer scope can be accessed in all inner scopes. The inner scope can also rebind the name already defined in the outer scope. The previous example using `var` to define variables describes this scenario.

Variable Life Cycle

There is no concept of variable names when the program is running, and variables exist in the form of entities at this time. The “validity period” of a variable during program execution is the variable’s **Life cycle**. Variables at runtime are only valid within the scope. After leaving the scope, the variables will be destroyed to reclaim resources.

Variables defined in the global scope are called **Global variable** and have **Static life cycle**. Such variables can be accessed during the entire program running and will not be destroyed. Variables defined in the local scope are called **Local variable** and have **Dynamic life cycle**. Such variables cannot be accessed after leaving the scope and will be destroyed.

Due to the different life cycles, local variables and global variables use different ways to allocate storage space. Local variables are allocated on a structure called **Stack** (stack), and objects allocated based on the stack can be quickly reclaimed at the end of the scope. Global variables are allocated in **Global table** (global table). Objects in the global table will not be recycled once they are created, and the table can be accessed anywhere in the program.

2.2.3 Type of variable

Berry determines the type of the variable at runtime. In other words, the variable can store any type of value. Therefore Berry is a **Dynamic typing** language. The interpreter does not deduce the type of the variable at compile time, which may cause some errors to be exposed at runtime. For example, the error generated by executing the expression `'1' + 1` is a runtime error rather than a compiler error. The advantage of using dynamic types is that many designs can be simplified, and the program will be more flexible, not to mention the need to design a complex type inference system.

Due to the lack of type checking by the interpreter, user code may need to determine the type of value by itself, and this feature can also be used to implement some special operations. This feature also makes overloaded functions unnecessary. For example, the native function `type` accepts any type of parameter and returns a string describing the parameter type.

3. Expression

Table of Content

Placeholder for table of contents

0

3.1 Basics

An expression is composed of one or more operands and operators, and produces a result when evaluated. This result is called the value of the expression. Operands can be literal values, variables, function calls, or sub-expressions. Simple expressions and operators can be combined into more complex expressions. Like arithmetic operations, operator precedence affects the evaluation order of expressions. Operators with higher precedence are evaluated first.

Operators and expressions

Berry provides some unary operators (Unary Operator) and binary operators (Binary Operator). For example, the logical AND operator `&&` is a binary operator, and the logical negation operator `!` is a unary operator. Some operators can be either unary operators or binary operators. The specific meaning of such operators depends on the context. For example, operator `-` is a unary symbol in expression `-1`, but it is a binary minus sign in expression `1 - 2`.

Operator combination expression

Both the left and right sides of a binary operator can be subexpressions, so you can use binary operators to combine expressions. A more complex expression often has multiple operators and operands. At this time, the order of evaluation of each sub-expression in the expression may affect the value of the expression. The precedence and associativity of operators guarantee the uniqueness of the expression evaluation order. For example, a combined expression:

`1 + 10/2 * 3`

The four arithmetic operations in daily use will first calculate the division expression `10/2`, then the multiplication expression, and finally the addition expression. This is because multiplication and division have higher priority than addition.

operand type

In the operation of expressions, the operands may have types that do not match the operators. In addition, binary operators usually require the left and right operands to be of the same type. The expression `'10'+10` is wrong. You cannot add a string to an integer. The problem with the expression `- 'b'` is that you cannot take a negative value on a string. Sometimes a binary operator has different operand types but can perform operations. For example, when adding an integer to a real number, the integer object will be converted to a real number and added to another real number object. The logical AND and logical OR operators allow the operands on both sides of the operator to be of any type. In logical expressions, they will always be converted to the boolean type according to certain rules.

Another situation is that operators can be overloaded when using custom classes. In essence, you can interpret this operator arbitrarily, and it is up to you to decide what type of its operand should be.

3.1.1 Priority and associativity

In a compound expression composed of multiple operators, the precedence and associativity of the operators determine the order of evaluation of the expressions. The precedence and associativity of each operator are given in Table 1.1.

The precedence specifies the order of evaluation between different operators, and expressions with higher precedence operators will be evaluated first. For example, the process of evaluating the expression `1+2*3` will first calculate the result of `2*3`, and then the result of the addition expression. Using parentheses can improve the evaluation order of low-priority expressions. For example, in the evaluation of expression `(1+2)*3`, the result of expression `1+2` in the parentheses is calculated first, and then the multiplication expression outside the parentheses is calculated.

Associativity refers to the evaluation order of the operands on both sides of the operator, where the operands may be subexpressions. For example, in the addition expression `expr1 + expr2`, the value of `expr1` is calculated first and then the value of `expr2`, because the addition operator is left-associative.

Priority	Operator	Description	Associativity
1	<code>()</code>	Grouping	.
2	<code>() [] .</code>	Function call, subscript, member access	left
3	<code>- ! ~</code>	Unary minus, logical NOT, bitwise NOT	right
4	<code>* / %</code>	Multiplication, division, modulo	left
5	<code>+ -</code>	Addition, subtraction	left
6	<code><< >></code>	Left shift, right shift	left
7	<code>&</code>	Bitwise AND	left
8	<code>^</code>	Bitwise XOR	left
9	<code> </code>	Bitwise OR	left
10	<code>..</code>	Range/ concatenation	left

11	< <= > >=	Relational comparison	left
12	== !=	Equality comparison	left
13	&&	Logical AND	left
14		Logical OR	left
15	? :	Conditional (ternary)	right
16	= += -= *= /= %= &= = ^= <<= >>=	Assignment	right
17	:=	Walrus assignment	right

Operator precedence and associativity

Use brackets to increase priority

Parentheses can be used when we need operators with lower precedence to be evaluated first. During expression evaluation, the value of the expression in parentheses is calculated first. In other words, for the entire expression, the expression in parentheses is equivalent to an operand, regardless of the composition of the expression in parentheses.

3.2 Operator

3.2.1 Arithmetic Operators

Arithmetic operators are used to implement arithmetic operations. These operators are similar to the mathematical symbols we usually use. The arithmetic operators provided by Berry are shown in Table 1.2.

Operator	Description	Example
-	Unary minus	- expr
+	Plus/string concatenation	expr + expr
-	Minus sign	expr - expr
*	Multiplication/string repetition	expr * expr
/	Division sign	expr / expr
%	Take the remainder	expr % expr

Arithmetic Operator

Binary operator + In addition to being a plus sign, it is also a string concatenation. When the operand of this operator is a string, string concatenation will be performed to concatenate two strings into a longer string. To be precise, + as a string concatenation is no longer in the category of arithmetic operators.

Binary operator * In addition to being a multiplication sign, it can also be used for string repetition. When the left operand is a string and the right operand is an integer or boolean, the string will be repeated the specified number of times. To be precise, * as a string repetition operator is no longer in the category of arithmetic operators.

The binary operator % is the remainder symbol. Its operands must be integers. The result of the remainder operation is the remainder after dividing the left operand by the right operand. For example, the result of 11%4 is 3. The real number type cannot do divisible, so the remainder is not supported.

In general, arithmetic operators do not satisfy the commutative law. For example, the values of the expressions 2/4 and 4/2 are not the same.

All arithmetic operators can be overloaded in the class. The overloaded operators are not necessarily limited to their original functional design, but are determined by the programmer.

3.2.2 Relational operators

Relational operators are used to compare the magnitude of the operands. The six relational operators supported by Berry are given in Table 1.3.

Operator	Description	Example
<	Less than	expr < expr
<=	Less than or equal to	expr <= expr
==	equal	expr == expr
!=	not equal to	expr != expr
>=	greater or equal to	expr >= expr
>	more than the	-expr

Relational operator

By comparing the magnitude relationship of the operands or judging whether the operands are equal, evaluating the relational expression will produce a Boolean result. When the relationship is satisfied, the value of the relationship expression is `true`, otherwise it is `false`. Relational operators `==` and `!=` can use any type of operand, and allow the left and right operands to have different types. Other relational operators allow the use of the following combinations of operands:

integer relop **integer**

real relop **real**

integer relop **real**

real relop **integer**

string relop **string**

In relational operations, the equal sign `==` and inequality sign `!=` satisfy the commutative law. If the left and right operands are of the same type or are both numeric types (integer and real number), the operands are judged to be equal according to the value of the operands, otherwise the operands are considered unequal. Equality and inequality are reciprocal operations: if `a==b` is true, then `a!=b` is false, and vice versa. Other relational operators do not satisfy the commutative law, but have the following properties: `<` and `>=` are reciprocal operations, and `>` and `<=` are reciprocal operations. Relational operations require that the operands must be of the same type, otherwise it is an incorrect expression.

Instances can overload operators as methods. If the relational operator is overloaded, the program needs to ensure the above properties.

Among the relational operators, `==` and `!=` operators have more relaxed requirements than `<`, `<=`, `>` and `>=`, which only allow comparisons between the same types. In actual program development, the judgment of equality or inequality is usually simpler than the judgment of size. Some operation objects may not be able to judge the size but can only judge the equality or inequality. This is the case with the Boolean type.

logical operators

Logical operators are divided into three types: logical AND, logical OR and logical NOT. As shown in Table 1.4.

Operator	Description	Example
&&	Logical AND	expr && expr
\ \	Logical OR	expr \ \ expr
!	Logical negation	!expr

Logical Operators

For the logical AND operator, when the values of both operands are `true`, the value of the logical expression is `true`, otherwise it is `false`.

For the logical OR operator, when the values of both operands are `false`, the value of the logical expression is `false`, otherwise it is `true`.

The role of the logical negation operator is to flip the logical state of the operand. When the operand value is `true`, the logical expression value is `false`, otherwise the value is `true`.

Logical operators require that the operand is of Boolean type, and if the operand is not of Boolean type, it will be converted. See section [section::type_bool] for conversion rules.

Logic operations use an evaluation strategy called **Short-circuit evaluation** (short-circuit evaluation). This evaluation strategy is: for the logical AND operator, the second operand will be evaluated if and only if the left operand is true; for the logical OR operator, if and only if the left operand is false Will evaluate the right operand. The nature of short-circuit evaluation causes the code in the logical expression to not all run.

Bitwise Operator

Bit operators can implement some binary bit operations, and bit operations can only be used on integer types. The detailed information of bit operators is shown in Table 1.5. Bit operation refers to the operation of binary bits directly on integers. Logical operations can be extended to bit operations. Taking logical AND as an example, we can perform this operation on each binary bit to achieve bitwise AND, such as $110_b \text{ AND } 011_b = 010_b$. Bit operations also support shift operations, which move numbers on a binary basis.

Operator		Example
~	Bit flip	~expr
&	Bitwise and	expr & expr
\	Bitwise or	expr \ expr
^	Bitwise exclusive or	expr ^ expr
<<	Shift left	expr << expr
>>	Shift right	expr >> expr

Bitwise operator

Although it can only be used for integers, bit operations are still versatile. Bit operations can implement many optimization techniques. In many algorithms, using bit operations can save a lot of code. For example, to determine whether a number n is a power of 2, we can judge whether the result of $n \& (n - 1)$ is 0. In some languages with high execution efficiency, shift operations can also be used to optimize multiplication and division (usually there is no obvious effect in scripting languages).

The bitwise AND operator "&" is a binary operator, which performs the binary AND operation of two integer operands: only when the binary bits corresponding to the operands are all 1, the result It was 1. For example, $1110_b \& 0111_b = 0110_b$.

The bitwise OR operator "|" is a binary operator, which performs a binary-bit OR operation on two integer operands: only when the binary bits corresponding to the operands are both 0, the bit of the result It was 0. For example, $1000_b \| 0001_b = 1001_b$.

The bitwise exclusive OR operator "^" is a binary operator, which performs binary exclusive OR operation on two integer operands: when the binary bits corresponding to the operands are different, the bit value of the result is 1. For example, $1100_b \wedge 0101_b = 1001_b$.

The left shift operator "<<" is a binary operator, which moves the left operand to the left by the number of bits specified by the right operand on a binary basis. For example $00001010_b \ll 3 = 01010000_b$. The right shift operator ">>" is a binary operator, which shifts the left operand to the right by the number of bits specified by the right operand on a binary basis. For example, $10100000_b \gg 3 = 00010100_b$.

The bitwise flip operator "~" is a unary operator, and the result of the expression is to flip the value of each binary bit of the operand. For example, $\sim 10100011_b = 01011100_b$.

The following are some examples of using bit operations. Usually we don't use binary directly. The results in the examples have been converted into common bases.

```
1 << 1 # 2
168 >> 4 # 10
456 & 127 # 72
456 | 127 # 511
0xA5 ^ 0x5A # 255
~2 # -3
```

Assignment operator

The assignment operator only appears in the assignment expression, and the operand of the operator must be a writable object. The assignment expression has no result, so continuous assignment operations cannot be used.

Simple assignment operator

The simple assignment operator = can be used for variable assignment. If the left operand variable is not defined, the variable will be defined. The assignment operator is used to bind the value of the right operand with the left operand. This process is also called "assignment". Therefore, the left operand cannot be a constant, nor can it be any object that cannot be written. These are some legal assignment expressions:

```
a = 45 b = 'string' c = 0
```

And the following assignment expression is wrong:

```
1 = 5 # Trying to assign a constant 1
a = b = 0 # Continuous assignment
```

When assigning nil, integer, real and Boolean types to variables, the value of the object will be passed to the left operand, but for other types, the assignment operation just passes the reference of the object to the left operand. Since strings, functions, and class types are read-only, all passing references will not have side effects, but you must be extra careful with instance types.

Compound Assignment Operator

Compound assignment operators are operators that combine binary operators and assignment operators. They are practical extensions to simple assignment operators. Compound assignment operators can simplify the writing of some expressions. Table 1.6 lists all the compound assignment operators

Operator	Description
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Preliminary assignment
%=	Remainder assignment
&=	Bitwise AND assignment
\ =	Bitwise OR assignment
^=	Bitwise XOR assignment
<<=	Left shift assignment
>>=	Right shift assignment

Bit operator

The compound assignment expression performs the binary operation corresponding to the compound assignment operator on the left operand and the right operand, and then assigns the result to the left operand. Taking += as an example, the expression a += b is equivalent to a = a + b. The compound assignment operator is also an assignment operator, so it has a lower priority. The binary operator corresponding to the compound assignment operator is always evaluated after the right operand, so an expression like a *= 1 + 2 should be equivalent to a = a * (1 + 2).

Unlike the simple assignment operator, the left operand of the compound assignment operator must participate in the evaluation, so the compound assignment expression does not have the function of defining variables. The assignment operator itself cannot be overloaded in the class. Users can only overload the binary operator corresponding to the compound assignment operator. This also ensures that the compound assignment operator will always conform to the basic characteristics of assignment operations.

Walrus Assignment Operator

The walrus assignment operator `:=` combines assignment with expression evaluation. Unlike regular assignment, the walrus operator can be used within expressions and returns the assigned value. This allows for more compact code where you need to both assign a value and use it in the same expression.

```
# Regular assignment (statement)
a = 12
print(a)

# Walrus assignment (expression)
print(a := 12) # Assigns 12 to 'a' and prints 12
```

The walrus operator is particularly useful in conditional statements and loops:

```
# Using walrus in if statement
if (n := input("Enter a number: ")) != "":
    print("You entered:", n)
end

# Using walrus in while loop
while (line := file.readline()) != nil:
    print(line)
end
```

The walrus operator has lower precedence than regular assignment operators, so parentheses are often needed to control evaluation order.

domain operator and subscript operator

Domain operator `.` is used to access an attribute or member of an object. You can use domain operators for both types of modules and instances:

```
l = list[]
l.push('item 0')
s = l.item(0) #'item 0'
```

The subscript operator `[]` is used to access the elements of an object, for example

```
l[2] = 10 # Read by index
n = l[2] # Write by index
```

Classes that support subscript reading must implement the `item` method, and classes that support subscript writing must implement the `setItem` method. The `map` and `list` in the standard container implement these two methods, so they support reading and writing using the subscript operator. The `string` supports subscript reading, but does not support subscript writing (strings are read-only values):

```
'string'[2] #'r'
'string'[2] = 'a' # error: value'string' does not support index assignment
```

Currently, strings support integer subscripts, and the range of subscripts cannot exceed the length of the string.

Strings also support range-based access (slicing) similar to lists:

```
s = "hello"
s[1..3]      # "ell" (characters from index 1 to 3)
s[1..]      # "ello" (characters from index 1 to end)
s[1..-1]    # "ello" (characters from index 1 to last)
s[0..-2]    # "hell" (all characters except the last one)
s[-2..-1]   # "lo" (last two characters)
```

Conditional Operator

The conditional operator (`? :`) is similar to the `if else` statement, but the former can be used in expressions. The usage form of the conditional operator is:

```
cond ? expr1 : expr2
```


cond is the expression used to judge the condition. The evaluation process of the conditional operator is: first find the value of **cond**, if the condition is true, evaluate **expr1** and return the value, otherwise, the value of **expr2**] Evaluate and return the value. **expr1** and **expr2** can have different types, so the following is correct:

```
result = scope <6?'bad': scope
```

This expression first determines whether scope is less than 6, if it is, it returns bad, otherwise it returns the value of scope. Regardless of the condition of the conditional expression, only one of **expr1** or **expr2** will be executed, similar to the short-circuit characteristic of logical AND and logical OR operations.

Nested Condition Operators

One conditional operator can be nested in another conditional operator, that is, the conditional expression can be used as **cond** or **expr** of another conditional expression. For example, use conditional expressions to divide scores into three levels: excellent, good, and bad:

```
result = scope >= 9?'excellent': scope >= 6?'good':'bad'
```

The first condition checks whether the score is not lower than 9 points. If it is, execute the branch after ? and return 'excellent'; otherwise, execute the branch after :, which is also a conditional expression. The condition checks whether the score is not lower than 6, if it is, it returns 'good', otherwise it returns 'bad'.

The conditional operator satisfies the right associativity, so the value of the branch expression must be evaluated first to get the value of the conditional expression. Therefore, in a nested conditional expression, the nested conditional expression is evaluated first, and then the outer conditional expression is evaluated.

Priority of conditional operators

Since the precedence of conditional expressions is very low (second only to assignment operators), it is often necessary to add parentheses outside the conditional expressions. For example, when a conditional expression is used as an operand of an arithmetic expression, parentheses will have different effects on the result:

```
result = 10 * (sign <0? -1: 1) # the result is -10 or 10
result = 10 * sign <0? -1: 1 # the result is -1 or 1
```

The result of the first expression is correct, and the second expression takes `10 * sign < 0` as a condition to judge, which does not meet the expectation of the conditional expression as the right operand of the multiplication.

Concatenation Operator

+ operator

When the left and right operands are both strings, the + operator is used to connect the two strings, and the new string obtained is the value of the expression. Therefore, this operator is often used for string concatenation:

```
result = 'abc' + '123' # the result is 'abc123'
```

+ Operators can also be used to connect two list instances:

```
result = [1, 2] + [3, 4] # the result is [1, 2, 3, 4]
```

Unlike the `list.push` method, the + operator merges two lists into a larger list object, with the elements of the left operand at the head of the result list, and the elements of the right operand at the end of the result list.

.. operator

.. is a special operator. If the left operand is a string, the behavior of the expression is to concatenate the left and right operands into a new string (automatic conversion if the right operand is not a string):

```
result = 'abc' .. 123 # the result is 'abc123'
```

The .. operator is often used when concatenating a string and a non-string value. If the left operand is a list instance, the .. operator will append the right operand to the end of the list, and then use this list as the value of the expression:

```
result = [1, 2] .. 3 # the result is [1, 2, 3]
```


This process will directly modify the left operand, which is very similar to the push method of List (except for strings which are immutable objects). The join operation of list can be executed in chain:

```
result = [1, 2] .. 3 .. 4 # the result is [1, 2, 3, 4]
```

All values in this process will be appended to the leftmost list object.

If the left and right operands are both integers, use the .. operator to get an integer range object:

```
result = 1 .. 10 # the result is (1..10)
```

This object is used to represent a closed interval of integers, where the left operand is the lower limit and the right operand is the upper limit. Such integer range objects are often used for iteration.

* operator for String Repetition

When the left operand is a string and the right operand is an integer or boolean, the * operator performs string repetition. The string is repeated the specified number of times to create a new string:

```
result = "aze" * 3      # "azeazeaze" - repeat string 3 times
result = "aze" * 0      # "" - empty string
result = "aze" * true   # "aze" - string if true
result = "aze" * false  # "" - empty string if false
```

When the right operand is an integer, the string is repeated that many times. A repetition count of 0 results in an empty string. When the right operand is a boolean, true results in the original string (repeated once) and false results in an empty string.

This operator is particularly useful for creating formatted output:

```
# Create indentation spaces
indent = 4
spaces = " " * indent      # "      " (8 spaces)

# Conditional pluralization
n = 3
message = n .. " item" .. ("s" * (n != 1)) # "3 items"
```

4. Statement

Table of Content

Placeholder for table of contents

0

Berry is an imperative programming language. This paradigm assumes that programs are executed step by step. Normally, Berry statements are executed sequentially, and this program structure is called sequential structure. Although the sequence structure is very basic, branch structures and loop structures are usually used in actual programs. Berry provides several control statements to realize this complex flow structure, such as conditional statements and iteration statements.

Except for line comments, carriage returns or line feeds (“\r” and “\n”) are only used as blank characters, so statements can be written across lines. In addition, you can write multiple statements on the same line.

You can add a semicolon at the end of the statement to indicate the end of the statement, but the interpreter can usually split the statement automatically without using a semicolon. You can use semicolons to tell the interpreter how to parse the code for the code that will be ambiguous. However, it is better not to write ambiguous code.

4.1 Simple sentence

4.1.1 Expression statement

Expression statements are mainly statements composed of assignment expressions or function call expressions. Other expressions can also form sentences, but they have no meaning. For example, expression `1+2` is a sentence written alone, but it has no effect. The following routines give examples of expression statements and function statements:

```
a = 1 # Assignment statement
print(a) # Function call statement
```

The first line is a simple assignment statement that assigns the literal value 1 to the variable a. The second line is a function call statement, which prints the value of variable a by calling the `print` function.

Cross-line expressions are written in the same way as single-line expressions, and no special line continuation symbols are required. E.g:

```
a = 1 +
    func() # Wrap line
```

You can also write multiple expression statements on one line, and various types of statements can be written on one line. This example puts two expression statements on the same line:

```
b = 1 c = 2 # Multiple statements
```

Sometimes the programmer wants to write two statements, but the interpreter may mistakenly think it is one statement. This problem is caused by the ambiguity in the process of grammatical analysis. Take this code as an example:

```
a = c
(b) = 1 # Be regarded as a function call
```

Suppose the 4th and 5th lines are intended to be two expression sentences: `a = c` and `(b) = 1`, but the interpreter will interpret them as a sentence: `a = c(b) = 1`. The cause of this problem is that the interpreter incorrectly parses `c` and `(b)` into function calls. To avoid ambiguity, we can add a semicolon at the end of the statement to clearly separate the statement:

```
a = c; (b) = 1;
```

A better way is not to use parentheses on the left side of the assignment number. Obviously, there is no reason to use parentheses here. Under normal circumstances, complex expressions should not appear on the left side of the assignment operator, but only simple expressions composed of variable names, domain operation expressions, and subscript operation expressions:

```
a = c b = 1
```

Using simple expressions only on the left side of the assignment sign will not cause ambiguity in sentence segmentation. Therefore, in most cases, there is no need to use semicolons to separate expressions, and we do not recommend this way of writing.

Block

A **Block** is a collection of several sentences. A block is a scope, so the variables defined in the block can only be accessed inside the block and its sub-blocks. There are many places where blocks are used, such as `if` statements, `while` statements, function declarations, etc. These statements will contain a block through a pair of keywords. For example, the block used in the `if` statement:

```
if isOpen
    close()
    print('the device was closed')
end
```

The statements in lines 2 to 3 constitute a block, which is sandwiched between the pair of keywords `if` and `end` (the conditional expression of the statement in `if` is not in the block). The block does not need to contain any statements, which constitutes an empty block, or it can be said to be a block containing an empty statement. Broadly speaking, any number of consecutive sentences can be called a block, but we prefer to expand the scope of the block as much

as possible, which can ensure that the area of the block is consistent with the scope of the scope. In the above example, we tend to think that rows 2 to 3 are a whole block, which is the largest range between `if` keywords and end keywords.

do Statement

Sometimes we just want to open up a new scope, but don't want to use any control statements. In this case, we can use the `do` statement to encapsulate the block. `do` The statement has no control function. `do` The sentence has the form

`do block end`

Among them **block** is the block we need. This statement uses a pair of `do` and `end` keywords to contain blocks. `do` The statement has no control function, nor does it generate any runtime instructions.

conditional statement

Berry provides `if` statements to realize the function of conditional control execution. This kind of program structure is generally called branch structure. `if` The statement will determine the branch of execution based on the true (true) or false (false) conditional expression. In some languages, there are other options for implementing conditional control. For example, languages such as C and C++ provide `switch` statements, but in order to simplify the design, Berry does not support `switch` statements.

if Statement

`if` statement is used to implement the branch structure, which selects the branch of the program according to the true or false of a certain judgment condition. The statement can also include `else` branch or `elif` branch. The simple `if` statement form without branches is

`if condition block end`

condition is a conditional expression. When the value of **condition** is `true`, **block** in the second line will be executed, otherwise the **block** will be skipped and the statement following `end` will be executed. In the case of **block** being executed, after the last statement in the block is executed, it will leave the `if` statement and start executing the statement following `end`.

Here is an example to illustrate the usage of the `if` statement:

```
if 8 % 2 == 0
    print('this number is even')
end
```

This code is used to judge whether the number 8 is even, and if it is, it will output `this number is even`. Although this example is very simple, it is enough to illustrate the basic usage of `if` sentences.

If you want to have a corresponding branch for execution when the condition is met and not met, use the `if` statement with the `else` branch. `if else` The form of the sentence is

```
if condition block
else
block
end
```

Different from the simple `if` statement, the `if else` statement will execute **block** under the `else` branch when the value of **condition** is false. No matter which branch is executed under **block**, after the last statement in the block is executed, the `if else` statement will pop out, that is, the statement after `end` will be executed. In other words, no matter whether the value of **condition** is true or false, one **block** will be executed.

Continue to use the judgment of parity as an example, this time change the demand to output corresponding information according to the parity of the input number. The code to achieve this requirement is:

```
if x % 2 == 0
    print('this number is even')
else
    print('this number is odd')
end
```

Before running this code, we must first assign an integer value to the variable `x`, which is the number we want to check for parity. If `x` is an even number, the program will output `this number is even`, otherwise it will output `this number is odd`. Sometimes we need to nest `if` statements. One way is to nest a `if` statement under the `else` branch. This is a very common requirement because many conditions need to be judged consecutively. For this kind of demand, use the `if else` statement to write:

```
if expr
  block
else
  if expr
    block
  end
end
```

Obviously, this way of writing will increase the indentation level of the code, and it is more cumbersome to use multiple `end` at the end. As an improvement, Berry provides the `elif` branch to optimize the above writing. Using the `elif` branch is equivalent to the above code, in the form

```
if condition
block
elif condition
block
else
block
end
```

`elif` The branch must be used after the `if` branch and before the branch, and the `elif` branch can be used multiple times in succession. If the **condition** corresponding to the `elif` branch is satisfied, the **block** under the branch will be executed. `elif` Branching is suitable for situations that require multiple conditions to be judged in sequence.

We use a piece of code that judges positive, negative, and 0 to demonstrate the `elif` branch:

```
if x > 0
  print('positive')
elif x == 0
  print('zero')
else
  print('negative')
end
```

Here too, the variable `x` must be assigned first. This code is very simple and will not be explained.

Some languages have a problem called dangling “else”, which refers to when a `if` sentence is nested inside another `if` sentence, where does the `else` branch belong? Problem with the sentence `if`. When using C/C++, we must consider the problem of dangling `else`. In order to avoid ambiguity on the problem of `if else`, C/C++ programmers often use curly braces to make a branch into a block. In Berry, the branch of the `if` statement must be a block, which also determines that Berry does not have the problem of overhanging `else`.

Iteration Statement

Iterative statements are also called loop statements, which are used to repeat certain operations until the termination condition is met. Berry provides `while` statement and `for` two iteration statements. Many languages also provide these two statements for iteration. Berry's `while` statement is similar to the `while` statement in C/C++, but Berry's `for` statement is only used to traverse the elements in the container, similar to the `foreach` statement provided by some languages and the one introduced by C++11 New `for` sentence style. The C-style `for` statement is not supported.

while Statement

`while` statement is a basic iterative statement. `while` statement uses a judgment condition. When the condition is true, the loop body is executed repeatedly, otherwise the loop is ended. The pattern of the statement is

```
while condition block end
```

When the program runs to the `while` statement, it will check whether the expression **condition** is true or false. If it is true, execute the loop body **block**, otherwise end the loop. After executing the last statement in **block**, the program will jump to the beginning of the statement `while` and start the next round of detection. If the **condition** expression is false when it is first evaluated, the loop body **block** will not be executed at all (same as the **condition** expression of the `if` statement is false). Generally speaking, the value of **condition** expression should be able to change during the loop, rather than a constant or a variable modified outside the loop, which will cause the loop to not execute or fail to terminate. A loop that never ends is called an endless loop. Usually we usually expect the loop to execute a specified number of times and then terminate. For example, when using the `while` loop to access all elements in the array, we hope that the number of loop executions is the length of the array, for example:

```
i = 0
l = ['a', 'b', 'c']
while i < l.size()
  print(l[i])
  i = i + 1
end
```

This loop gets the elements from the array `l` and prints them. We use a variable `i` as the loop counter and array index. We let the value of `i` reach the length of the array `l` to end the loop. In the last line of the loop body, we add 1 to the value of `i` to ensure that the next element of the array is accessed in the next loop, and the `while` loop ends when the number of loops reaches the length of the array.

for Statement

Berry's `for` statement is used to traverse the elements in the container, and its form is

```
for variable : expression
block end
```

expression The value of the expression must be an iterable container or function, such as the `range` class. `for` The statement obtains an iterator from the container, and obtains an element in the container every time through the call to the iterator.

variable is called an iteration variable, which is always defined in the statement `for`. Therefore **variable** must be a variable name and not an expression. The container element obtained from the iterator in each loop will be assigned to the iteration variable. This process occurs before the first statement in **block**.

The `for` statement will check whether there are any unvisited elements in the iterator for iteration. If there are, the next iteration will start, otherwise it will end the `for` statement and execute the statement following `end`. Currently, Berry only provides read-only iterators, which means that the elements in the container cannot be modified through the iteration variables in the `for` statement.

The scope of the iteration variable **variable** is limited to the loop body **block**, and the variable will not have any relationship with the variable with the same name outside the scope. To illustrate this point, let's use an example to illustrate. In this example, we use the `for` statement to access all the elements in the `rang` instance and print them out. Of course, we also use this example to demonstrate the scope of loop variables.

```
i = "Hi, I'm fine." # Outer variable
for i: 0 .. 2
  print(i) # Iteration variable
end
print(i)
```

In this example, relative to the iteration variable `i` defined in line 2, the variable `i` defined in line 1 is an external variable. Running this example will get the following result

```
0
1
2
Hi, I'm fine
```

It can be seen that the iteration variable `i` and the external variable `i` are two different variables. They just have the same name but different scopes.

for Principle of Statement

Unlike the traditional iterative statement `while`, the `for` statement uses iterators to traverse the container. If you need to use the `for` statement to traverse a custom class, you need to understand its implementation mechanism. When using the `for` statement, the interpreter hides a lot of implementation details. In fact, for such code:

```
for i: 0 .. 2
  print(i)
end
```

Will be translated into the following equivalent code by the interpreter:

```
var it = __iterator__(0 .. 2)
try
  while true
    var i = it()
    print(i)
  end
except 'stop_iteration'
  # do nothing
end
```

To some extent, the `for` statement is just a syntactic sugar, it is essentially just a simple way of writing a piece of complex code. In this equivalent code, an intermediate variable `it` is used. The value of the variable is an iterator. In this example, it is an iterator of the range container `0 .. 2`. When processing the `for` statement, the interpreter hides the intermediate variable of the iterator, so it cannot be accessed in the code.

The parameter of function `__iterator__` is a container, and the function returns an iterator of parameters. This function gets the iterator by calling the parameter method. Therefore, if the return value of the `iter` method is an instance (instance) type, this instance must have a `next` method and a `hasnext` method.

The parameter of function `__hasnext__` is an iterator, which checks whether the iterator has the next element by calling the `hasnext` method of the iterator. `hasnext` The return value of the method is of type `boolean`. The parameter of function `__next__` is also an iterator, which gets the next element in the iterator by calling the `next` method of the iterator.

So far, the `__iterator__`, `__hasnext__` and `__next__` functions simply call some methods of the container or iterator and then return the return value of these methods. Therefore, the equivalent writing of the `for` statement can also be simplified into this form:

```
do
  var it = (0 .. 2).iter()
  while (it.hasnext())
    var i = it.next()
    print(i)
  end
end
```

This code is easier to read. It can be seen from the effective code that the scope of the iterator variable `it` is the entire `for` statement, but it is not visible outside the `for` statement, while the scope of the iteration variable `i` is in the loop body, so every time iterations will define new iteration variables.

Jump Statement

The jump statement provided by Berry is used to realize the jump of the program flow in the loop process. Jump statements are divided into `break` statements and `continue` statements. These two statements must be used inside iterative statements and can only be used inside functions to jump. Some languages provide `goto` statements to realize arbitrary jumps within functions, which Berry does not provide, but the effects of `goto` statements can be replaced by conditional statements and iteration statements.

break Statement

break Used to terminate the iteration statement and jump out. After the execution of the **break** statement, the nearest level of the iteration statement will be terminated immediately and execution will continue from the position of the first statement after the iteration statement. In order to illustrate the execution flow of the **break** statement, we use an example to demonstrate:

```
while true
    print('before break')
    break
    print('after break')
end
print('out of the loop')
```

In this code, the **break** statement is in a **while** loop. Before and after the **break** statement and after the **while** statement, we have placed a **print** statement to test the execution flow of the program. The result of this code is:

```
before break
out of the loop
```

This shows that the **while** statement ends the loop at the **break** statement position on the 3rd line and the program continues to execute from the 6th line.

continue Statement

continue The statement is also used inside an iteration statement. Its function is to end an iteration and immediately start the next round. Therefore, after the execution of the **continue** statement, the remaining code in the iteration statement of the nearest layer will no longer be executed, but a new round of iteration will start. Here we use a **for** statement to demonstrate the function of the **continue** statement:

```
for i: 0 .. 5
    if i >= 2
        continue
    end
    print('i =', i)
end
print('out of the loop')
```

Here, the **for** statement will iterate 6 times. When the iteration variable **i** is greater than or equal to 2, the **continue** statement on line 3 will be executed, and the **print** statement on line 5 will not be executed thereafter. In other words, line 5 will only be executed in the first two iterations (at this time $i < 2$). The running result of this routine is:

```
i = 0
i = 1
out of the loop
```

It can be seen that the value of the variable **i** is only printed twice, which is in line with expectations. Readers can try to print the value of the variable **i** before the **continue** statement. You will find that the **for** statement does iterate 6 times, indicating that the **continue** statement does not terminate the iteration.

import Statement

Berry has some predefined modules, such as the **math** module for mathematical calculations. These modules cannot be used directly, but must be imported with the **import** statement. There are two ways to import a module:

import name

import name as variable

name For the name of the module to be imported, when using the first writing method to import the module, the imported module can be called directly by using the module name. The second way of writing is to import a module named **name** and modify the module name when calling it to **variable**. For example, a module named **math**, we use the first method to import and use:

Welcome to berry's documentation!

```
import math
math.sin(0)
```

Here directly use math to call the module. If the name of a module is relatively long and it is not convenient to write, you can use the `import as` statement. Here, assume a module named hardware. We want to call the function `setled` of the module, we can import the module hardware into the variable named hw and use:

```
import hardware as hw
hw.setled(true)
```

To find modules, all paths in `sys.path()` are explored sequentially. If you want to add a specific path before the import (like reading from SD card) you can use the following helper function:

```
def push_path(p)
  import sys
  var path = sys.path()
  if path.find(p) == nil # append only if it's not already there
    path.push(p)
  end
end
```

Exception Handling

The mechanism allows the program to capture and handle exceptions that occur during runtime. Berry supports an exception capture mechanism, which allows the exception capture and handling process to be separated. That is, part of the program is used to detect and collect exceptions, and the other part of the program is used to handle exceptions.

First of all, the problematic program needs to throw an exception first. When these programs are in an exception handling block, a specific program will catch and handle the exception.

Raise an exception

Using the `raise` statement raises an exception. `raise` The statement will pass a value to indicate the type of exception so that it can be identified by a specific exception handler. Here is how to use the `raise` statement:

```
raise exception
```

```
raise exception, message
```

The value of the expression `exception` is the thrown **Outliers**; the optional `message` expression is usually a string describing the exception information, and this expression is called **Abnormal parameter**. Berry allows any value to be used as an abnormal value, for example, a string can be used as an abnormal value:

```
raise 'my_error', 'an example of raise'
```

After the program executes to the `raise` statement, it will not continue to execute the statements following it, but will jump to the nearest exception handling block. If the most recent exception handling block is in other functions, the functions along the call chain will exit early. If there is no exception handling block, **Abnormal exit** will occur, and the interpreter will print the exception error message and the call stack of the error location. When the `raise` statement is in the `try` statement block, the exception will be caught by the latter. The caught exception will be handled by the `except` block associated with the `try` block. If the thrown exception can be handled by the `except` block, the execution of this block will continue from the statement after the last `except` block. If all `except` statements cannot handle the exception, the exception will be rethrown until it can be handled or the exception exits.

Outliers

In Berry, you can use any value as an outlier, but we usually use short strings. Berry may also throw some exceptions internally. We call these exceptions **Standard exception**. All standard exception values are of string type.

Outliers	Description	Parameter Description
<code>assert_failed</code>	Assertion failed	Specific exception information
<code>index_error</code>	(usually out of bounds)	Specific exception information

io_error	IO Malfunction	Specific exception information
key_error	Key error	Specific exception information
runtime_error	VM runtime exception	Specific exception information
stop_iteration	End of iterator	no
syntax_error	Syntax error	
by the compiler		
unrealized_error	Unrealized function	Specific exception information
type_error	Type error	Specific exception information

Standard exception list

Catch exceptions

Use the `except` statement to catch exceptions. It must be paired with the `try` statement, that is, a `try` statement block must be followed by one or more `except` statement blocks. `try-except` The basic form of the sentence is

```
try block
except ... block end
```

The `except` branch can have the following forms

```
except ..
except exceptions
except exceptions as variable
except exceptions as variable , message
except .. as variable
except .. as variable , message
```

The most basic `except` statement does not use parameters, this `except` branch will catch all exceptions; **Catch exception list exceptions** is a list of outliers that can be matched by the corresponding `except` branch, used between multiple values in the list Separate by commas; **variable** is **Abnormal variable**, if the branch catches an exception, the outlier will be bound to the variable; **message** is **Abnormal parameter variable**, if the branch catches an exception, the abnormal parameter value will be bound To the variable.

When an exception is caught in the `try` statement block, the interpreter will check the `except` branch one by one. If the exception value exists in the capture list of a branch, the code block under the branch will be called to handle the exception, and the entire `try-except` statement will exit after the code block is executed. If all the `except` branches do not match, the exception will be re-thrown and caught and handled by the outer exception handler.

Error Handling Patterns

Berry provides two main approaches to error handling:

1. **Exception Handling**: Using `try/except` blocks to catch and handle exceptions.
2. **Return Value Checking**: Many functions return special values to indicate errors rather than raising exceptions.

The second approach is common in functions that parse data or perform operations that might fail. Here are some examples:

```
# JSON parsing - returns nil on parsing error
data = json.load('{"invalid": }')
if data == nil
  print("Invalid JSON")
end

# Map access - find() returns nil if key doesn't exist
value = map.find("key")
if value == nil
```

```
    print("Key not found")
end

# String operations - find() returns -1 if substring not found
index = string.find("hello", "z")
if index == -1
    print("Substring not found")
end
```

This pattern of checking return values is particularly useful in resource-constrained environments where exception handling might be more expensive.

5. Function

Table of Content

Placeholder for table of contents

0

function is a “subroutine” that can be called by external code. As a part of the program, the function itself is also a piece of code. A function can have 0 or more parameters and will return a result, which is called the function's **return value**.

In Berry, the function is **first class value**. Therefore, in addition to calling functions, you can also pass functions as values, for example, bind functions to variables, use functions as return values, and so on.

5.1 Basic Information

The use of functions includes two parts: function definition and call. The function definition statement uses the `def` keyword as the beginning. The function definition is the process of packaging and naming the code of the function body. This process only generates the function structure and does not execute the function. The execution function must use **call operator**, which is a pair of parentheses. The call operator acts on an expression whose result is a function type. The parameters passed to the function are written in parentheses, and multiple parameters are separated by commas. The result of the calling expression is the return value of the function.

5.1.1 Function Definition

Named Function

named function is a function given a name when it is defined. Its definition statement consists of the following parts: `def` keywords, function names, lists consisting of 0 to multiple parameters, and function bodies (function body), multiple parameters in the parameter list are separated by commas, and all parameters are written in a pair of parentheses. We call the parameter when the function is defined as **Formal parameters**, and the parameter when calling the function as **Arguments**. The general form of the function definition is:

```
'def' name '(' arguments ')'  
    block  
'end'
```

The function name **name** is an identifier; **arguments** is the formal parameter list; **block** is the function body. If the function body is an empty statement, the function is called an “empty function”. The function return value statement is contained in the function body. If there is no return statement in **block**, the function will return `nil` by default. The function name is actually the variable name of the bound function object. If the name already exists in the current scope, defining the function is equivalent to binding the function object to this variable.

The following example defines a function named `add`. The function of this function is to sum two numbers and return.

```
def add(a, b)  
    return a + b  
end
```

`add` The function has two parameters `a` and `b`, and the two addends are passed into the function through these parameters for calculation. `return` The statement returns the result of the calculation.

A function as a class attribute is called a method. This part of the content will be explained in the object-oriented chapter.

Anonymous Function

Unlike named functions, **anonymous function** has no name, and its definition expression has the form:

```
def (arguments)
  block
end
```

It can be seen that compared with named functions, there is no function name in the definition of anonymous functions **name**. The definition of an anonymous function is essentially an expression, which is called **Function literal**. In order to use anonymous functions, we can bind the function literal value to a variable:

```
add = def (a, b)
  return a + b
end
```

The function of this code is exactly the same as that of the function `add` in the previous section. An anonymous function can be used to conveniently pass the function value as a literal value. Like other types of literals, function literals are also the smallest unit of expressions. Therefore, between `def` keywords and `end` are an indivisible whole.

Call function

Take the `add` function as an example. To call this function, you need to provide two values, and you can get the sum of the two numbers by calling the function:

```
res = add(5, 3)
print(res) # 8
```

We call the called function (the `add` function in the example) as **Called function**, and the function that calls the called function (the main function in the example) as **Key function**. The function call process is as follows: First, the interpreter will (implicitly) initialize the formal parameter list of the called function with the argument list, and at the same time suspend the calling function and save its state, then create an environment for the called function and execute the called function.

The function will end its execution when it encounters the `return` statement and pass the return value to the calling function. The interpreter will destroy the environment of the called function after the called function returns, then restore the environment of the calling function and continue to execute the calling function. The return value of the function is also the result of the function call expression. The following example defines a function `square` and binds this function to a variable `f`, and then calls the function `square` through the variable `f`. This usage is similar to function pointers in C language.

```
def square(n)
  return n * n
end
f = square
print(f(5)) # 25
```

It should be noted that the function object is only bound to these variables (refer to section [section::assign_operator]) and cannot be modified, so reassigning the variable corresponding to the function name will not make the function lose:

```
f = square
square = nil
print(f(5)) # 25
```

It can be seen that the function can still be called normally after reassigning `square`. Only after the function object is no longer bound to any variable will it be lost, and the resources occupied by this type of function object will be recycled by the system.

Forward call

The call of the function must be in the scope of the function variable, so it usually cannot be called before the function is defined. In order to solve this problem, you can use this method to compromise:

```
var func1
def func2(x)
    return func1(x)
end
def func1(x)
    return x * x
end
print(func2(4)) # 16
```

In this example, func2 calls func1, but the function func1 is defined after func2. After executing this code, the program will output the correct result 16. This routine uses the mechanism that the function will not be called when the function is defined. Define the variable func1 before defining func2 to ensure that the symbol func1 will not be found during compilation. Then we define the function func1 after func2 so that the function will be used to overwrite the value of the variable func1. When the function func2 is called in the last line `print(func2(4))`, the variable func1 is already the function we need, so the correct result will be output.

Recursive call

recursive function refers to functions that call themselves directly or indirectly. Recursion refers to a strategy that divides the problem into similar sub-problems and then solves them. Taking factorial as an example, the recursive definition of factorial is $0! = 1$, $n! = n \cdot (n-1)!$, we can write the recursive function for calculating factorial according to the definition:

```
def fact(n)
    if n == 0
        return 1
    end
    return n * fact(n-1)
end
```

Take the factorial of 5 as an example, the process of manually calculating the factorial of 5 is: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$. The result of calling the fact function is also 120:

```
print(fact(5)) # 120
```

In order to ensure that the depth of the recursive call is limited (too deep recursion level will exhaust the stack space), the recursive function must have an end condition. The `if` statement in the second line of the function definition is used to detect the end condition, and the recursive process ends when `n` is calculated as 0. The above factorial formula does not apply to non-integer parameters. Executing an expression like `fact(5.1)` will cause a stack overflow error due to the inability to end the recursion.

There is another situation Indirect recursion, that is, the function is not called by itself but by another function (directly or indirectly) called by it. Indirect recursion usually requires the use of forward function call techniques. Take the functions `is_odd` and `is_even` for calculating odd and even numbers as examples:

```
var is_odd
def is_even(n)
    if n == 0
        return true
    end
    return is_odd(n-1)
end
def is_odd(n)
    if n == 0
        return false
    end
    return is_even(n-1)
end
```

These two functions call each other. In order to ensure that this name is in the scope when calling the function `is_odd` on line 6, the variable `is_odd` is defined on line 1.

Anonymous function call

If an anonymous function will only be called once, the easiest way is to call it when it is defined, for example:

```
res = def (a, b) return a + b end (1, 2) # 3
```

In this routine, we use the call expression directly after the function literal to call the function. This usage is very suitable for functions that will only be called in one place.

You can also bind an anonymous function to a variable and call it:

```
add = def (a, b) return a + b end
res = add(1, 2) # 3
```

This usage is similar to the call of a named function, essentially calling the variable bound to the function value. It should be noted that it is more difficult to make recursive calls to anonymous functions, unless you use forward call techniques.

Formal and actual parameters

The function uses actual parameters to initialize the formal parameters when it is called. Under normal circumstances, the actual parameter and the shape parameter are equal and the positions correspond to each other, but Berry also allows the actual parameter to be unequal to the formal parameter: if the actual parameter is more than the formal parameter, the extra actual parameter will be discarded. Less than the formal parameters will initialize the remaining formal parameters to `nil`.

The process of parameter passing is similar to assignment operation. For `nil`, `boolean` and `numeric` types, parameter passing is by value, while other types are by reference. For the writable pass-by-reference type such as `instance`, modifying them in the called function will also modify the object in the calling function. The following example demonstrates this feature:

```
var l = [], i = 0
def func(a, b)
  a.push(1)
  b = 'string'
end
func(l, i)
print(l, i) # [1] 0
```

It can be seen that the value of variable `l` has changed after calling function `func`, but the value of variable `i` has not changed.

Function with variable number of arguments (vararg)

You can define a function to take any arbitrary number of arguments and iterate on them. For example `print()` takes any number of arguments and prints each of them separated by spaces. You need to define the last argument as a capture-all-arguments using `*` before its name.

All arguments following the formal arguments are grouped at runtime in a `list` instance. If no arguments are captured, the list is empty.

Example:

```
def f(a, b, *c) return size(c) end
f(1,2) # returns 0, c is []
f(1,2,3) # returns 1, c is [3]
f(1,2,3,4) # returns 2, c is [3,4]
```

Calling a function with dynamic number of arguments

Berry syntax allows only to call with a fixed number of arguments. Use the `call(f, [args])` function to pass any arbitrary number or arguments.

Welcome to berry's documentation!

You can statically add any number of arguments to `call()`. If the last argument is a list, it is automatically expanded to discrete arguments.

Example:

```
def f(a,b) return nil end

call(f,1)           # calls f(1)
call(f,1,2)         # calls f(1,2)
call(f,1,2,3)       # calls f(1,2,3), last arg is ignored by f
call(f,1,[2,3])     # calls f(1,2,3), last arg is ignored by f
call(f,[1,2])       # calls f(1,2)
call(f,[])          # calls f()
```

You can combine `call` and `vararg`. For example let's create a function that acts like `print()` but converts all arguments to uppercase.

Full example:

```
def print_upper(*a) # take arbitrary number of arguments, args is a list
  import string
  for i:0..size(a)-1
    if type(a[i]) == 'string'
      a[i] = string.toupper(a[i])
    end
  end
  call(print, a) # call print with all arguments
end

print_upper("a",1,"Foo","Bar") # prints: A 1 FOO BAR
```

Functions and local variables

The function body itself is a scope, so the variables defined in the function are all local variables. Unlike directly nested blocks, every time a function is called, space is allocated for local variables. The space for local variables is allocated on the stack, and the allocation information is determined at compile time, so this process is very fast. When multiple levels of scope are nested in a function, the interpreter allocates stack space for the scope nesting chain with the most local variables, rather than the total number of local variables in the function.

return Statement

`return` The statement is used to return the result of a function, that is, the return value of the function. All functions in Berry have a return value, but you can choose not to use any return statement in the function body. At this time, the interpreter will generate a default return statement to ensure that the function returns. There are two ways to write return statements:

```
`return'
`return' expression
```

The first way of writing is to write only the `return` keyword and not the expression to be returned. In this case, the default `nil` value is returned. The second way of writing is to follow the expression **expression** after the `return` keyword, and the value of the expression will be used as the return value of the function. When the program executes to the return statement, the currently running function will end execution and return to the code that called the function to continue running.

When using a separate keyword `return` as the return statement of a function, it is easy to cause ambiguity. At this time, it is recommended to add a semicolon after `return` to prevent errors:

```
def func()
  return;
  x = 1
end
```

In this example, the `x = 1` statement after the `return` statement will not be executed, so it is redundant. If this kind of redundant code is avoided, the `return` statement is usually followed by keywords such as `end`, `else` or `elif`. In this case, even if a separate `return` statement is used, there is no need to worry about ambiguity.

closure

Basic Concepts

As mentioned earlier, functions are the first type of value in Berry. You can define functions anywhere, and you can also pass functions as parameters or return values. When another function is defined in a function, the nested function can access the local variables of any outer function. We call the "local variables of the outer function" used in the function the function **Free variable**. The generalized free variables also include global variables, but there is no such rule in Berry. **Closure** is a technique that binds functions to **environments**. The environment is a mapping that associates each free variable of a function with a value. In terms of implementation, closures associate the function prototype with its own variables. Function prototypes are generated at compile time, and environment is a runtime concept, so closures are also dynamically generated at runtime. Each closure binds the function prototype to the environment when it is generated, for example, in the following example:

```
def func(i) # The outer function
  def foo() # The inner function (closure)
    print(i)
  end
  foo()
end
```

The inner function `foo` is a closure, which has a free variable `i`, which is a parameter of the outer function `func`. When the closure `foo` is generated, its function prototype is bound to the environment containing the free variable `i`. When the variable `foo` leaves the scope, the closure will be destroyed. Usually, the inner function will be the return value of the outer function, for example:

```
def func(i) # The outer function
  return def () # Return a closure (anonymous function)
    print(i)
    i = i + 1
  end
end
```

The closure returned here is an anonymous function. When the closure is returned by the outer function, the local variables of the outer function will be destroyed, and the closure will not be able to directly access the variables in the original outer function. The system will copy the value of the free variable to the environment when the free variable is destroyed. The life cycle of these free variables is the same as the closure, and can only be accessed by the closure. The returned function or closure will not be executed automatically, so we need to call the closure returned by the function `func`:

```
f = func(0)
f()
```

This code will output 0. If we continue to call the closure `f`, we will get the output 1, 2, 3... This may not be well understood: variable [2.198] is destroyed after the function `func` returns, and as a free variable of the closure `f`, `i` will be stored in the closure environment, so every time `f` is called, the value of `i` will be added to 1 (func function definition line 4).

Use of closures

Closures have many uses. Here are a few common uses:

Lazy evaluation

The closure does not do anything until it is called.

Function private communication

You can let some closures share free variables, which are only visible to these closures, and communicate between functions by changing the values of these free variables. This can avoid the use of external variables.

Generate multiple functions

Sometimes we may need to use multiple functions, these functions may only have different values of some variables. We can implement a function and then use these different variables as function parameters. A better way is to return the closure through a factory function, and use these possibly different variables as free variables of the closure, so that you don't always have to write those parameters when calling the function, and any number of similar functions can be generated.

Simulate private members

Some languages support the use of private members in objects, but Berry's class does not support private members. We can use the free variables of closures to simulate private members. This use is not the original intention of designing closures, but nowadays, this "misuse" of closures is very common.

Cache result

If there is a function that is very time-consuming to run, it will take a lot of time to call it every time. We can cache the result of this function, look it up in the cache before calling the function, and return the cached value if found, otherwise call the function and update the cached value. We can use closures to save the cached value so that it will not be exposed to the outer scope, and the cached result will be retained (until the closure is destroyed).

Binding free variables

If multiple closures bind the same free variable, all closures will always share this free variable. E.g:

```
def func(i) # The outer function
  return [# Return a closure list
    def () # The closure #1
      print("closure 1 log:", i)
      i = i + 1
    end,
    def () # The closure #2
      print("closure 2 log:", i)
      i = i + 1
    end
  ]
end
```

The function func in this example returns two closures through a list, and these two closures share free variables i. If we call these closures:

```
f = func(0)
f[0]() # closure 1 log: 0
f[1]() # closure 2 log: 1
```

As you can see, we updated the free variable i when we called the closure f[0], and this change affected the result of calling the closure f[1]. This is because if a free variable is used by multiple closures, there is only one copy of the free variable, and all closures have a reference to the free variable entity. Therefore, any modification to the free variable is visible to all closures that use the free variable.

Similarly, before the local variables of the outer function are destroyed, modifying the value of the free variable will also affect the closure:

```
def func()
  i = 0
  def foo()
    print(i)
  end
  i = 1
```



```
    return foo
end
```

In this example, we change the value of the variable `i` (which is the free variable of the closure `foo`) from 0 to 1 before the outer function `func` returns, then we call the closure afterwards. The value of the free variable `i` when the package `foo` is also 1:

```
func()() # 1
```

Create closure in loop

When constructing a closure in the loop body, you may not want the free variables of the closure to change with the loop variables. Let's first look at an example of creating a closure in a loop `while`:

```
def func()
  l = [] i = 0
  while i <= 2
    l.push(def () print(i) end)
    i = i + 1
  end
  return l
end
```

In this example, we construct a closure in a loop and put this closure in a list. Obviously, when the loop ends, the value of the variable `i` will be 3, and all the closures in the list `l` are also references using this variable. If we execute the closure returned by `func` we will get the same result:

```
res = func()
res[0]() # 3
res[1]() # 3
res[2]() # 3
```

If we want each closure to refer to different free variables, we can define another layer of functions, and then bind the current loop variables with the function parameters:

```
def func()
  l = [] i = 0
  while i <= 2
    l.push(def (n)
      return def () print(n) end
      end (i))
    i = i + 1
  end
  return l
end
```

To help understand this seemingly incomprehensible code, we focus on the code from lines 4 to 6:

```
def (n)
  return def ()
    print(n)
  end
end (i)
```

Here actually defines an anonymous function and calls it immediately. The function of this temporary anonymous function is to bind the value of the loop variable `i` to its parameter `n`, and the variable `n` is also what we need to close the free variables of the package, so that the free variables bound to the closure constructed during each loop are different. Now we will get the desired output:

```
res = func()
res[0]() # 0
res[1]() # 1
res[2]() # 2
```

There are some ways to solve the problem of loop variables as free variables. A slightly simpler way is to define a temporary variable in the loop body:

```
def func()  
  l = [] i = 0  
  while i <= 2  
    temp = i  
    l.push(def () print(temp) end)  
    i = i + 1  
  end  
  return l  
end
```

Here temp is a temporary variable. The scope of this variable is in the loop body, so it will be redefined every time it loops. We can also use the for statement to solve the problem:

```
def func()  
  l = []  
  for i: 0 .. 2  
    l.push(def () print(i) end)  
  end  
  return l  
end
```

This may be the simplest way. for The iteration variable of the statement will be created in each loop. The principle is similar to the previous method.

Lambda expression

Lambda expression is a special anonymous function. Lambda expression is composed of parameter list and function body, but the form is different from general function:

```
/ args -> expr end
```

args is the parameter list, the number of parameters can be zero or more, and multiple parameters are separated by commas or spaces (cannot be mixed at the same time); **expr** is the return expression, the lambda expression will return the expression value. Lambda expressions are suitable for implementing functions with very simple functions. For example, the lambda expression for judging the size of two numbers is:

```
/ a b -> a < b
```

This is easier than writing a function of the same function. In some general sorting algorithms, this type of size comparison function may need to be used extensively. Using lambda expressions can simplify the code and improve readability.

Like general functions, lambda expressions can form closures. Lambda expressions are called in the same way as ordinary functions. If you use the immediate calling method similar to anonymous functions:

```
lambda = / a b -> a < b  
result = lambda(1, 2) # normal calling  
result = (/ a b -> a < b)(1, 2) # direct calling
```

Since the function call operator has a higher priority, a pair of parentheses should be added to the lambda expression when making a direct call, so that it will be called as a whole.

6. Object Oriented Function

Table of Content

Placeholder for table of contents

0

For optimization considerations, Berry did not consider simple types as objects. These simple types include nil types, numeric types, boolean types, and string types. But Berry provides classes to implement the object mechanism. Among Berry's basic data types, list, map and range are class objects. An object is a collection containing data and methods, where data is composed of some variables, and methods are functions. The type of an object is called a class, and the entity of an object is called an instance.

6.1 Class and instance

6.1.1 Class declaration

To use a class, you must first declare it. The declaration of a class starts with the keyword `class`. The member variables and methods of the class must be specified in the declaration. This is an example of declaring a class:

```
class person
  static var majority = 18
  var name, age
  def init(name, age)
    self.name = name
    self.age = age
  end
  def tostring()
    return 'name: ' + str(self.name) + ', age: ' + str(self.age)
  end
  def isadult()
    return self.age >= self.majority
  end
end
```

Class member variables are declared with keyword `var`, while member methods are declared with keyword `def`. Currently, Berry does not support initializing member variables at the time of definition, so the initialization of member variables should be done by the constructor. The properties of the class cannot be modified after the declaration is completed, so the class is a read-only object.

This design is to ensure that the class can be statically constructed in the C language when the interpreter is implemented and the `const` property can be used Modified to save RAM

Berry's class does not support access restrictions, and all properties of the class are visible to the outside. In native classes, you can use some tricks to make properties invisible to Berry code (usually let the member name start with dot "."). You can use some conventions to restrict access to the members of the class, such as the convention that the attributes starting with an underscore are private attributes. This convention does not have any use at the grammatical level, but is conducive to the logical structure of the code.

Instantiate

The class itself is just an abstract description. Taking cars as an example, I know the concept of cars, and when we really want to use cars, we need real cars. The use of classes is similar. We will not only use this abstract description, but need to produce a concrete object based on this description. This process is called **Instantiation of the class**, abbreviated as instantiation, and the concrete object produced by instantiation is called **Instance**. The class itself does not have data, and instantiation produces an instance based on the information described by the class and gives the instance specific data.

Method and self Parameters

Class methods are essentially functions. Unlike ordinary functions, methods implicitly pass in a `self` parameter, and is always the first parameter, which stores a reference to the current instance. Due to the existence of `self` parameters, the number of parameters of the method will be one more than the number of parameters defined in the declaration. Here we use a simple example to demonstrate:

```
class Test
  def method()
    return self
  end
end
```

```
object = Test()  
print(object)  
print(object.method())
```

This example defines a `Test` class, which has a method `method`, which returns its `self` parameter. The last two lines in the routine print the value of the instance `object` of the `Test` class and the return value of the method `method` respectively. The running result of this example is:

```
<instance: Test()>  
<instance: Test()>
```

It can be seen that the `self` parameter of the method and the name of the use instance (`object` in the example) both represent the same object, and they are both instance references. Use `self` to access the members or attributes of the instance in the method.

Synthetic methods

You can declare synthetic dynamic members and methods using the **Virtual members** as described in Chapter 8.2.

Class Variables static

Variables or functions can be declared `static`. Static variables have the same value for all instances of the same class. They are declare as `static a = 1` or `static var a = 1`. Static variables are initialized right after the creation of the class.

Static variables can be referenced via `self` from instance methods. For example, if you declare `static var a`, you can access it as `self.a` within any instance method.

```
class Test  
  static var count = 0  
  def init()  
    self.count += 1  # access static var via self  
  end  
  def getCount()  
    return self.count # access static var via self  
  end  
end
```

Class Methods static

Methods can be declared `static` which means that they act like regular function and do not take `self` as first argument. Within static methods, there is no implicit `self` variable declared. Static methods can be called via the class or via an instance.

However, static methods receive an implicit `_class` variable that references the class itself. This allows static methods to access other class members like static variables or call other static methods.

```
class Test  
  static var value = 42  
  static def getValue()  
    return _class.value  # access static var via _class  
  end  
  static def doubleValue()  
    return _class.getValue() * 2  # call other static method via _class  
  end  
end  
> Test.getValue()  
42  
> Test.doubleValue()  
84  
  
> class static_demo  
  static def increment_static(i)
```

Welcome to berry's documentation!

```
        return i + 1
    end
    def increment_instance(i)
        return i + 1
    end
end
> a = static_demo()
> static_demo.increment_static(1)    # call via class
2

> a.increment_static(1)               # call via instance
>
> static_demo.increment_instance(1)
type_error: unsupported operand type(s) for +: 'nil' and 'int'
stack traceback:
  stdin:6: in function increment_instance
  stdin:1: in function main

> a.increment_instance(1)
2
```

Constructor and Destructor

Constructor

The constructor of the class is the `init` method. The constructor is called when the class is instantiated. Therefore, the constructor is generally used for member initialization, for example:

```
class Test
  var a
  def init()
    self.a = 'this is a test'
  end
end
```

The constructor in this example initializes the `a` member of the `Test` class to the string `'this is a test'`. If we instantiate the class, we can get the value of member `a`:

```
print(Test().a) # this is a test
```

Destructor

The destructor of the class is the `deinit` method. The destructor is called when the instance is destroyed. The destructor is generally used to complete some cleanup work. Because the garbage collection mechanism automatically releases the memory of useless objects, there is no need to release the memory in the destructor (and there is no way to release the memory in the destructor). In most cases, there is no need to use a destructor, unless a certain class requires certain processing when it is destroyed. A typical example is that a file object must close the file when it is destroyed.

Class inheritance

Berry only supports single inheritance, that is, a class can only have one base class, and the base class uses the operator `:` to declare:

```
class Test: Base
  ...
end
```

Here the `Test` class inherits from the `Base` class. The subclass will inherit all the methods and properties of the base class, and you can override them in the subclass. This mechanism is called **Overload**. Under normal circumstances, we will only overload methods, not properties.

The inheritance mechanism of the Berry class is relatively simple. Subclasses will contain references to the base class, and instance objects are similar. When instantiating a class with a base class, multiple objects are actually generated. These objects will be chained together according to the inheritance relationship, and finally we will get the instance object at the end of the inheritance chain.

Method Overload

Overload means that the subclass and the base class use the same name method, and the subclass method will override the mechanism of the base class method. To be precise, member variables can also be overloaded, but this overloading has no meaning. Method overloading is divided into ordinary method overloading and operator overloading.

Common method overload

Operator Overloading

You can use the operator overloading of the class to make the instance support the operation of the built-in operator. For example, for a class overloaded with the addition operator, we can use the addition operator to perform operations on the instance. An overloaded operator is a method with a special name, and the overloaded function form of a binary operator is

```
def operator `(` other `)`  
  block  
end
```

operator is an overloaded binary operator. The left operand of the binary operator is the `self` object, and the right operand is the value of the parameter **other**. The overloaded function form of the unary operator is

```
def operator `()`  
  block  
end
```

operator is an overloaded unary operator. To distinguish it from the subtraction operator, the unary minus sign is written as `-*` when overloaded. Operator overloaded functions should have a return value, because the default `nil` return value is usually not the expected result. Let's take an integer class as an example to illustrate the use of operator overloading. First define the integer class:

```
class integer  
  var value  
  def init(v)  
    self.value = v  
  end  
  def +(other)  
    return integer(self.value + other.value)  
  end  
  def *(other)  
    return integer(self.value * other.value)  
  end  
  def -*()  
    return integer(-self.value)  
  end  
  def tostring(other)  
    return str(self.value)  
  end  
end
```

The integer class overloads the plus, multiplication, and symbolic operators, and the `tostring` method is to make the instance use the `print` function to output the result. We can use a simple line of code to test the operator overloading function of the class:

```
integer(1) + integer(2) * -integer(3) # -5
```

The result of this line of code is an instance of `integer`. The value of the `value` member of this instance is `-5`, which is the same as the result of the same four arithmetic operations on integers.

Logical operators cannot be overloaded directly. If you need an instance to support logical operations, you must implement the `tobool` method. The method has no parameters and the return value must be of Boolean type. The logic operation of the instance is actually realized by converting the instance into a Boolean value, so the logic operation of the instance is completely in line with the nature of the general logic operation. The subscript operator is not directly overloaded, but is implemented by the methods `item` and `setitem`. `item` The method is used for subscript reading, its first parameter is the subscript value, and the return value is the result of the subscript operation; `setitem` is used for subscript writing, and its first parameter is the subscript Value, the second parameter is the value to be written, this method does not use the return value.

The overloaded operator can be assigned any meaning, even not satisfying the usual properties of operators. Considering the versatility of the code and the difficulty of understanding, it is not recommended that users give overloaded operators a function far from the general meaning.

Overload of compound assignment operator

The compound assignment operator cannot be directly overloaded, but we can achieve the purpose of “overloading” the compound assignment operator by overloading the binary operator corresponding to the compound assignment operator. For example, after overloading the “+” operator, you can use the “+=” operator for instances of related classes. It is worth noting that the use of compound assignment operations on the instance will cause the variables of the bound instance to lose their reference to the instance.

```
class integer
  var value
  def init(x)
    self.value = x
  end
  def +(other)
    return integer(self.value + other.value)
  end
end
a = integer(4) # a: <instance: 0x55edff400a78>
a += integer(5) # a: <instance: 0x55edff4011b8>
print(a.value) # 9
```

After the 11th line of code is executed, the instance bound in the variable `a` has actually changed. This line of code is equivalent to `a = integer(4) + integer(5)`. If the binary operator of the class overload does not modify the state of the instance, then the corresponding compound assignment operator will not modify any instance (it may generate new instances).

Instance

Instance is an object generated after class instantiation. A class can be instantiated multiple times to generate different instances. Berry instances are referenced by the class they belong to and the corresponding data fields. All instances of a class will refer to this class, but the data fields of these instances are independent of each other.

Access base class object

The built-in function `super` is used to access super class objects. It can be used on classes or instances.

Magic happens when you call a method from the superclass so that it behaves like you intuitively think it would. For example, the common pattern for `init()` is as follows:

```
def init(<args>)
  # do stuff before super init
  super(self).init(<args>)
  # do stuff after super init
end
```

Note that classes always contains an implicit `init()` methods that does nothing, so you can always call `init` from super class even if no `init()` method was declared.

Full example:

Welcome to berry's documentation!

```
class A
  var val
  def init(val)
    # super(self).init(val)    # this would be valid but useless
    self.val = val
  end
  def tostring()
    return "val=" + str(self.val)
  end
end

class B: A
  var magic    # true if value is 42
  def init(val)
    super(self).init(val)    # call super init
    self.magic = (val == 42)
  end
  def tostring()
    if self.magic
      return "magic!"
    else
      return super(self).tostring()
    end
  end
end
```

Example of usage

```
> b1 = B(1)
> b1
val=1
> b42 = B(42)
> b42
magic!
```

Advanced features When calling `super(self).<method>(<args>)` some magic happens. When the super-method is called, the `self` arguments refers to the lowest more specific class. However the `<method>` is searched not from the class of `self` (which is always the lowest), but from the super class of the class containing the method currently running.

Example:

```
> class A
  def init()
    print("In A::init, self is of type", classname(self))
  end
end
> class B:A
  def init()
    print("In B::init, self is of type", classname(self))
    super(self).init()
  end
end
> class C:B
  def init()
    print("In C::init, self is of type", classname(self))
    super(self).init()
  end
end
> c = C()
In C::init, self is of type C
```



```
In B::init, self is of type C
In A::init, self is of type C
>
```

Explanation:

- calling `C::init()` on `instance<C>`
- in `C::init()` `self` is `instance<C>`, `super(self).init()` refers to the super class of `C` (current method) i.e. `B`, so `B::init()` is called with `instance<C>` argument
- in `B::init()` `self` is `instance<C>`, `super(self).init()` refers to the super class of `B` (current method) i.e. `A`, so `A::init()` is called with `instance<C>` argument
- in `A::init()` `self` is `instance<C>`, print and return

Note: for backwards compatibility, `super` can take a second argument `super(instance, class)` to specify the class where to resolve the method. This feature should not be used anymore as it is error-prone.

7. Libraries and Modules

Table of Content

Placeholder for table of contents

0

7.1 Basic library

There are some functions and classes that can be used directly in the standard library. They provide basic services for Berry programs, so they are also called basic libraries. The functions and classes in the basic library are visible in the global scope (belonging to the built-in scope), so they can be used anywhere. Do not define variables with the same name as the functions or classes in the base library. Doing so will make it impossible to reference the functions and classes in the base library.

7.1.1 Built-in function

print function

Example

```
print(...)
```

Description

This function prints the input parameters to the standard output device. The function can accept any type and any number of parameters. All types will print their value directly, and for an instance, this function will check whether the instance has a `tostring()` method, and if there is, print the return value of the instance calling the `tostring()` method, otherwise it will print the address of the instance.

```
print('Hello World!') # Hello World!
print([1, 2, '3']) # [1, 2, '3']
print(print) # <function: 0x561092293780>
```

input function

Example

```
input()
input(prompt)
```

Description

`input` The function is used to input a line of character string from the standard input device. This function can use the `prompt` parameter as an input prompt, and the `prompt` parameter must be of string type. After calling the `input` function, characters will be read from the keyboard buffer until a newline character is encountered.

```
input('please enter a string:') # please enter a string:
```

`input` The function does not return until the “Enter” key is pressed, so the program “stuck” is not an error.

classname function

Example

```
classname(object)
```

Description

This function returns the class name (string) of the parameter. Therefore the parameter must be a class or instance, and other types of parameters will return `nil`.

```
classname(list) # 'list'
classname(list()) # 'list'
classname({}) # 'map'
classname(0) # nil
```

classof function

Example

```
classof(object)
```

Description

Returns the class of an instance object. The parameter object must be an instance. If the function is successfully called, it will return the class to which the instance belongs, otherwise it will return `nil`.

```
classof(list) # nil
classof(list()) # <class: list>
classof({}) # <class: map>
classof(0) # nil
```

str function

Example

```
str(value)
```

Description

This function converts the parameters into strings and returns. `str` Functions can accept any type of parameters and convert them. When the parameter type is an instance, it will check whether the instance has a `tostring()` method, if there is, the return value of the method will be used, otherwise the address of the instance will be converted into a string.

```
str(0) # '0'
str(nil) # 'nil'
str(list) # 'list'
str([0, 1, 2]) # '[0, 1, 2]'
```

number function

```
number(value)
```

Description

This function converts the input string or number into a numeric type and returns. If the input parameter is an integer or real number, it returns directly. If it is a character string, try to convert the character string to a numeric value in

decimal format. The integer or real number will be automatically judged during the conversion. Other types return `nil`.

Example

```
number(5) # 5
number('45.6') # 45.6
number('50') # 50
number(list) # nil
```

int function

```
int(value)
```

Description

This function converts the input string or number into an integer and returns it. If the input parameter is an integer, return directly, if it is a real number, discard the decimal part. If it is a string, try to convert the string to an integer in decimal. Other types return `nil`. When the parameter type is an instance, it will check whether the instance has a `toint()` method, if there is, the return value of the method will be used.

Example

```
int(5) # 5
int(45.6) # 45
int('50') # 50
int('0x10') # 16 - hex literal are valid
int(list) # nil
```

real function

```
real(value)
```

Description

This function converts the input string or number into a real number and returns. If the input parameter is a real number, it will return directly, if it is an integer, it will be converted to a real number. If it is a string, try to convert the string to a real number in decimal. Other types return `nil`.

Example

```
real(5) # 5, type(real(5)) → 'real'
real(45.6) # 45.6
real('50.5') # 50.5
real(list) # nil
```

bool function

```
bool(value)
```

Description

This function converts the input string or number into a boolean and returns it.

The conversion follows the following rules:

- `nil`: converted to `false`.
- **Integer**: when the value is `0`, it is converted to `false`, otherwise it is converted to `true`.
- **Real number**: when the value is `0.0`, it is converted to `false`, otherwise it is converted to `true`.
- **String**: when the value is `""` (empty string) it is converted to `false` otherwise it is converted to `true`.
- **List**: when the list is empty `[]` it is converted to `false` otherwise it is converted to `true`.
- **Map**: when the map is empty `{}` it is converted to `false` otherwise it is converted to `true`.

- **Comobj** and **Comptr**: when the internal pointer is NULL it is converted to `false`, otherwise it is converted to `true`.
- **Instance**: if the instance contains a method `tobool()`, the return value of the method will be used, otherwise it will be converted to `true`.
- All other types: convert to `true`.

Example

```
bool() # false
bool(nil) # false
bool(false) # false
bool(true) # true
bool(0) # false
bool(1) # true
bool("") # false
bool("a") # true
bool(3.5) # true
bool(list) # true
bool([]) # true
bool({}) # true
# advanced
import introspect
bool(introspect.toptr(0)) # false
bool(introspect.toptr(0x1000)) # true
```

type function

Example

```
type(value)
```

- *value*: Input parameter (expect to get its type).
- *return value*: A string describing the parameter type.

Description

This function receives a parameter of any type and returns the type of the parameter. The return value is a string describing the type of the parameter. Table below shows the return values corresponding to the main parameter types:

Parameter Type	return value
Nil	'nil'
Integer	'int'
Real	'real'
Boolean	'bool'
String	'string'
Function	'function'
Class	'class'
Instance	'instance'
native pointer	'ptr'

```
type(0) #'int'
type(0.5) #'real'
type('hello') #'string'
type(print) #'function'
```

size function

```
size(value)
```

Description

This function returns the size of the input string. If the input parameter is not a string, 0 is returned. The length of the string is calculated in bytes. This function also works for list and map instances and returns the number of elements.

Example

```
size(10) # 0
size('s') # 1
size('string') # 6
size([1,2]) # 2
size({"a":1}) # 1
```

super function

```
super(object)
```

Description

This function returns the parent object of the instance. When you instantiate a derived class, it will also instantiate its base class. The super function is required to access the instance of the base class (that is, the parent object).

Please look at chapter 6 about magic behavior of super(object) when calling a super method.

Example

```
class mylist: list end
l = mylist() # classname(l) --> 'mylist'
sl = super(l) # classname(sl) --> 'list'
```

assert function

```
assert(expression)
assert(expression, message)
```

Description

This function is used to implement the assertion function. assert The function accepts a parameter. When the value of the parameter is false or nil, the function will trigger an assertion error, otherwise the function will not have any effect. It should be noted that even if the parameter is a value equivalent to false in logical operations (for example, 0), it will not trigger an assertion error. The parameter message is optional and must be a string. If this parameter is used, the text information given in message will be output when an assertion error occurs, otherwise the default "Assert Failed" message will be output.

Example

```
assert(false) # assert failed!
assert(nil) # assert failed!
assert() # assert failed!
assert(0) # assert failed!
assert(false, 'user assert message.') # user assert message.
assert(true) # pass
```

compile function

```
compile(string)
compile(string, 'string')
compile(filename, 'file')
```

Description

This function compiles the Berry source code into a function. The source code can be a string or a text file. `compile` The first parameter of the function is a string, and the second parameter is a string 'string' or 'file'. When the second parameter is 'string' or there is no second parameter, the `compile` function will compile the first parameter as the source code. When the second parameter is 'file', the `compile` function will compile the file corresponding to the first parameter. If the compilation is successful, `compile` will return the compiled function, otherwise it will return `nil`.

Example

```
compile('print(\'Hello World!\')')() # Hello World!
compile('test.be', 'file')
```

module function

Example

```
module()
module(name)
```

Description

Create an empty module. The optional name parameter specifies the module name. If no name is provided, an anonymous module is created.

Example

```
m = module()           # Create anonymous module
m.x = 10                # Add member to module

named_m = module("mymodule") # Create named module
```

issubclass function

Example

```
issubclass(sub, sup)
```

Description

Returns true if sub (class) is sup (class or instance) or its derived class, otherwise returns false.

Example

```
class A end
class B: A end

issubclass(B, A)    # true
issubclass(A, B)    # false
issubclass(A, A)    # true
```

isinstance function

Example

```
isinstance(obj, base)
```

Description

Returns true if obj is an instance of base (class or instance) or its derived class, otherwise returns false.

Example

```
class A end
class B: A end

a = A()
b = B()

isinstance(a, A)    # true
```

```
isinstance(b, A)      # true (B inherits from A)
isinstance(a, B)      # false
```

call function

Example

```
call(function[, args][, list])
```

Description

Call a function with an arbitrary number of arguments. All args are pushed as static arguments. If the last argument is a list, all elements are pushed as elementary arguments.

Example

```
def add(a, b, c) return a + b + c end

call(add, 1, 2, 3)           # 6
call(add, 1, [2, 3])        # 6 - list elements become arguments
call(print, "Hello", "World") # Hello World
```

list Class

list is a built-in type, which is a sequential storage container that supports subscript reading and writing. list Similar to arrays in other programming languages. Obtaining an instance of the list class can be constructed using a pair of square brackets: [] will generate an empty list instance, and [expr, expr, ...] will generate a list instance with several elements. It can also be instantiated by calling the list class: executing list() will get an empty list instance, and list(expr, expr, ...) will return an instance with several elements.

list method (Constructor)

Initialize the list container. This method can accept 0 to multiple parameters. The list instance generated when multiple parameters are passed will have these parameters as elements, and the arrangement order of the elements is consistent with the arrangement order of the parameters.

tostring method

Serialize the list instance to a string and return it. For example, the result of executing [1, [], 1.5].tostring() is '[1, [], 1.5]'. If the list container refers to itself, the corresponding position will use an ellipsis instead of the specific value:

```
l = [1, 2]
l[0] = l
print(l) # [[...], 2]
```

concat method

Converts each element of the list to strings, and concatenate using the provided string.

```
l = [1, 2, 3]
l.concat() # '123'
l.concat(", ") # '1, 2, 3'
```

push method

Append an element to the end of the list container. The prototype of this method is push(value), the parameter value is the value to be appended, and the appended value is stored at the end of the list container. The append operation increases the number of elements in the list container by 1. You can append any type of value to the list instance.

pop **method**

Removes an element from the list container. The prototype of this method is `pop(index)`, the parameter `index` is the position from which a value is to be removed, or the end if no index is used. If the index is negative, the position is the size of the list plus the index, effectively counting from the end, with -1 being the last element. Remaining elements after the position are shifted to lower positions. The return value of the method is the removed element.

insert **method**

Insert an element at the specified position of the list container. The prototype of this method is `insert(index, value)`, the parameter `index` is the position to be inserted, and `value` is the value to be inserted. After inserting an element at the position `index`, all the elements that originally started from this position will move backward by one element. The insert operation increases the number of elements in the list container by 1. You can insert any type of value into the list container.

Suppose that the value of a list instance `l` is `[0, 1, 2]`, and we insert a string `'string'` at position 1, and we need to call `l.insert(1, 'string')`. Finally, the new list value is `[0, 'string', 1, 2]`.

If the number of elements in a list container is S , the value range of the insertion position is $\{i \in \mathbb{Z} : -S \leq i < S\}$. When the insertion position is positive, index backward from the head of the list container, otherwise index forward from the end of the list container.

remove **method**

Remove an element from the container. The prototype of this method is `remove(index)`, and the parameter `index` is the position of the element to be removed. After the element is removed, the element behind the removed element will move forward by one element, and the number of elements in the container will be reduced by 1. Like the insert method, the remove method can also use positive or negative indexes.

item **method**

Get an element in the list container. The prototype of this method is `item(index)`, the parameter `index` is the index of the element to be obtained, and the return value of the method is the element at the index position. list The container supports multiple indexing methods:

- Integer index: The index value can be a positive integer or a negative integer. If the index is negative, it is relative to the end of the list; i.e. -1 indicates the last element in the list. The return value of `item` is the element at the index position. If the index position exceeds the number of elements in the container or is before the 0th element, the `item` method will return `nil`.
- list Index: Using a list of integers as an index, `item` returns a list, and each element in the return value list is an element corresponding to each integer index in the parameter list. The value of the expression `[3, 2, 1].item([0, 2])` is `[3, 1]`. If an element type in the parameter list is not an integer, then the value at that position in the return value list is `nil`.
- range Index: Using an integer range as an index, `item` returns a list. The returned value stores the indexed elements from list from the lower limit to the upper limit of the parameter range. If the index exceeds the index range of the indexed list, the return value list will use `nil` to fill the position beyond the index.

Examples of range-based indexing (slicing):

```
l = [1, 2, 3, 4, 5]
l[1..3]      # Returns [2, 3, 4] (elements from index 1 to 3)
l[1..]       # Returns [2, 3, 4, 5] (elements from index 1 to end)
l[1..-1]     # Returns [2, 3, 4, 5] (elements from index 1 to last)
l[0..-2]     # Returns [1, 2, 3, 4] (all elements except the last one)
l[-2..-1]    # Returns [4, 5] (last two elements)

# Negative indices can also be used for modification
l[-1] = 10   # Sets the last element to 10
l[-2] += 5   # Adds 5 to the second-to-last element
```


setitem **method**

Set the value of the specified position in the container. The prototype of this method is `setitem(index, value)`, `index` is the position of the element to be written, and `value` is the value to be written. `index` is the integer index value of the writing position. Index positions outside the index range of the container will cause `setitem` to fail to execute.

size **method**

Returns the number of elements in the container, which is the length of the container. The prototype of this method is `size()`.

resize **method**

Reset list the length of the container. The prototype of this method is `resize(count)`, and the parameter `count` is the new length of the container. When using `resize` to increase the length of the container, the new element will be initialized to `nil`. Using `resize` to reduce the length of the container will discard some elements at the end of the container. E.g:

```
l = [1, 2, 3]
l.resize(5) # Expansion, l == [1, 2, 3, nil, nil]
l.resize(2) # Reduce, l == [1, 2]
```

iter **method**

Returns an iterator for traversing the current list container.

find **method**

Similar to `item` or `list[idx]`. The only difference is if the index is out of range, `find` return `nil` instead or raising an exception.

reverse **method**

Changes the list in-place and reverses the order of elements. Also returns the resulting list.

keys **method**

Returns a range object containing the indices of the list. This is useful for iterating over list indices.

```
l = ['a', 'b', 'c']
for i: l.keys()
  print(i, l[i])
end
# Output: 0 a, 1 b, 2 c
```

copy **method**

Creates a shallow copy of the list. The list structure is copied, but the elements themselves are not copied (references are kept).

```
l1 = [1, [2, 3]]
l2 = l1.copy()
l2[0] = 10
print(l1) # [1, [2, 3]] - l1[0] unchanged
print(l2) # [10, [2, 3]]

l2[1][0] = 20
print(l1) # [1, [20, 3]] - nested list is shared
print(l2) # [10, [20, 3]]
```

map Class

map Class is a built-in class type used to provide an unordered container of key-value pairs. Inside the Berry interpreter, map uses the Hash table to implement. You can use curly brace pairs to construct a map container. Using an empty curly brace pair `{}` will generate an empty map instance. If you need to construct a non-empty map instance, use a colon to separate the key and value, and use a semicolon to separate multiple key-value pairs. For example, `{0: 1, 2: 3}` has two key-value pairs (0,1) and (2,3). You can also get an empty map instance by calling the map class.

map method (Constructor)

Initialize the map container, this method does not accept parameters. Executing `map()` will get an empty map instance.

tostring method

Serialize map as a string and return. The serialized string is similar to literal writing. For example, the result of executing `'str': 1, 0: 2` is `"'str': 1, 0: 2"`. If the map container refers to itself, the corresponding position will use an ellipsis instead of the specific value:

```
m = {'map': nil, 'text': 'hello'}
m['map'] = m
print(m) # {'text': 'hello', 'map': {...}}
```

insert method

Insert a key-value pair in the map container. The prototype of this method is `insert(key, value)`, the parameter `key` is the key to be inserted, and `value` is the value to be inserted. Returns boolean `true` when the key-value pair was inserted, or `false` when the insertion failed (e.g. the pair already exists).

Note on insert behavior: The `insert` method has two possible behaviors depending on implementation: 1. Always sets the value (standard behavior) 2. Only inserts if key doesn't exist (conditional behavior)

remove method

Remove a key-value pair from the map container. The prototype of this method is `remove(key)`, and the parameter `key` is the key of the key-value pair to be deleted.

item method

Get a value in the map container. The prototype of this method is `item(key)`, the parameter `key` is the key of the value to be obtained, and the return value of the method is the value corresponding to the key.

setitem method

Set the value corresponding to the specified key in the container. The prototype of this method is `setitem(key, value)`, `key` is the key of the key-value pair to be written, and `value` is the value to be written. If there is no key-value pair with the key `key` in the container, the `setitem` method will fail.

size method

Return the number of key-value pairs of the map container, which is the length of the container. The prototype of this method is `size()`.

contains method

Returns boolean `true` if a matching key-value pair is found in the map container, otherwise `false`. The prototype of this method is `contains(key)`.

find method

Returns the value corresponding to the specified key in the container. The prototype of this method is `find(key)` or `find(key, defaultvalue)`, `key` is the key of the key-value pair to be accessed, and `defaultvalue` is the default value returned if the key is not found. If no default value is specified, `nil` is returned instead.

keys method

Returns an iterator function over the keys of the map container, to produce one key each call, or raising `stop_iteration` if at end. This works well with a `for` loop. Example of usage:

```
m = {'map': nil, 'text': 'hello'}
for k: m.keys()
  print(k)
end
```

file Class

`file` is a built-in type, which handles files in the file-system (typically using Flash storage on embedded systems). `file` supports reading and writing for strings as well as binary data. `file` instances are only created via `open()` function.

```
f = open('test.txt', 'w')      # create the file or erase if already existing
f.write('foo bar')
f.seek(1)                     # seek to second character
f.write('ar')                  # file contains 'far bar'
f.close()
#
f = open('test.txt', 'r')      # open in read mode
s = f.readline()
f.close()
print(s)                       # 'far bar'
```

open function

`open` is a builtin function and returns an instance of `file` or raises an exception. `open(path [, mode])` is used to open a file provided a `path` (string) and an optional mode string. If an error occurs, an exception `io_error` is raised.

When opening in write mode, a lock is generally created on the file, so that the file cannot be opened again until the lock is released. The lock is released when calling the `close()` method.

Mode	Description
'r'	read-only mode, the file must exist.
'w'	write-only mode, always create a empty file.
'a'	create a empty file or append to the end of an existing file.
'r+'	read-write mode, the file must exist.
'w+'	read-write mode, always create a empty file.
'a+'	read-write mode, create a empty file or append to the end of an existing file.
'b'	binary mode, it can be combined with other access modes (generally ignored)

write method

Write a string or a bytes buffer to the file. The file must be open in write mode, or an exception is raised. `write()` accepts either a `string` object, or a `bytes` object which allows to write raw bytes, including NULL characters (`\x00`). Has no effect if the file is not open.

Writes are commonly buffered in memory and grouped in fewer actual writes to flash, in order to reduce wear leveling. You can always force writing to flash using the `flush()` method.

read [method](#)

Read from the file, either until the end of file if no arguments is passed or at most count characters if you pass an integer argument read(count). The result is a string, which is not suited for binary data. If the end of file is reached, an empty string is returned. Returns nil if the file is not open.

readbytes [method](#)

Read from the file, either until the end of file if no arguments is passed or at most count characters if you pass an integer argument readbytes(count). The result is a bytes, which is well suited for binary data. If the end of file is reached, an empty bytes() is returned. Returns nil if the file is not open.

readlines [method](#)

Read a line from the file (the newline character is determined by the platform). Returns a string or an empty string if the end of file is reached. Returns nil if the file is not open.

seek [method](#)

Takes a single integer argument seek(offset). Set the file pointer to offset bytes after the start of the file. No error is returned if offset is out of range, and points to the beginning of the file if it is negative, or the end of the file if it is bigger than the size of the file.

tell [method](#)

Returns the offset, in bytes, from the beginning of the file. Returns nil if the file is not open.

size [method](#)

Returns the size, in bytes, of the file. Returns nil if the file is not open.

flush [method](#)

Flush the file buffer, i.e. force writing to flash of all pending writes. Has no effect if the file is not open.

close [method](#)

Closes the file and free all associated resources. If the file was open in write mode, the lock is released. Has no effect if the file is not open.

range [Class](#)

range The class is used to represent an integer closed interval. Use the binary operator .. to construct an instance of range. The left and right operands of the operator are required to be integers. For example, 0..10 means the integer interval $[0,10] \cap \mathbb{N}$.

If you don't specify the high range, it is set to MAXINT. Example: print(0..) # (0..9223372036854775807)

There are typically two ways to traverse a list:

```
l = [1,2,3,4]
for e:l print(e) end # 1/2/3/4
for i:0..size(l)-1 print(l[i]) end # 1/2/3/4
```

bytes [Class](#)

bytes object are represented as arrays of Hex bytes. bytes constructor takes a string of Hex and builds the in-memory buffer.

Example:

```
b = bytes()
print(b) # bytes('')
b = bytes("1155AA") # sequence of bytes 0x11 0x55 0xAA
size(b) # 3 = 3 bytes
```

```
b[0]      # 17 (0x11)
b[0] = 16  # assign first byte
print(b)   # bytes('1055AA')
```

bytes **method (Constructor)**

Initialize a bytes array. There are several options.

Option 1: empty value

`bytes()` creates a new empty bytes array. `size(bytes()) == 0`.

There is no limit in the size of a bytes array, except the available memory. An internal buffer is allocated and reallocated in case the previous one was too small. The initial buffer is 36 bytes, but you can pre-allocate a larger (or smaller) buffer if you know in advance the size needed.

Similarly the buffer is automatically shrunk if it is used less than its needed size.

```
b = bytes(4096)  # pre-allocated 4096 bytes
```

Option 2: initial value

If first argument is a string it is parsed as a list of Hex values. You can add an optional second argument to pre-allocate a bigger buffer.

```
b = bytes("BEEF0000")
print(b)  # bytes('beef0000')
b = bytes("112233", 128)  # pre-allocate 128 bytes internally
print(b)  # bytes('112233')
```

Option 3: fixed size

If the size provided is negative, the array size is fixed and cannot be lowered nor raised.

```
b = bytes(-8)
print(b)  # bytes('0000000000000000')

b = bytes("AA", -4)
print(b)  # bytes('AA000000')

b = bytes("1122334455", -4)
attribute_error: bytes object size if fixed and cannot be resized
```

Option 4: memory mapping

Caution, use with great care

In this mode, the bytes array is mapped to a specific region in memory. You must provide the base address as `comptr` and the size. Size is always fixed whether it is positive or negative. This feature is **dangerous** since you can access any memory location, causing a crash if the location is protected or invalid. Use with care.

In this case `b.ismapped()` returns `true` indicating a mapped memory buffer. In all other cases `b.ismapped()` returns `false`. This is typically used to know if Berry allocated the buffer or not, and if sub-elements need to be explicitly deallocated.

Example:

```
import introspect
def f() return 0 end

addr = introspect.toptr(f)
print(addr)  # <ptr: 0x3ffeaf88>

b = bytes(addr, 8)
print(b)  # bytes('F8EAFE3F24000000')
# this example shows the first 8 bytes of the function object in memory
```



```
print(d)          # bytes('11223344')
print(b)          # bytes('1122')
print(c)          # bytes('3344')

e = b..c          # now b is changed
print(e)          # bytes('11223344')
print(b)          # bytes('11223344')
print(c)          # bytes('3344')
```

reverse **method**

Changes in-place and reverses the order of bytes. Also returns the resulting bytes instance. By default reverses the entire buffer; you can apply to a limited range of bytes with *<start>* and *<len>* parameters. You can also reverse by groups of *<grouplen>* bytes.

`b.reverse([<start>, <len>, <grouplen>])` -> bytes object

```
b = bytes("112233445566")
b.reverse()          # bytes('665544332211') - reverse all bytes
```

```
b = bytes("112233445566")
b.reverse(2)         # bytes('112266554433') - skip 2 bytes and reverse all remaining bytes
```

```
b = bytes("112233445566")
b.reverse(1,3)       # bytes('114433225566') - skip 1 byte and reverse 3 bytes
```

```
b = bytes("112233445566")
b.reverse(0, -1, 2)  # bytes('556633441122') - skip by groups of 2
```

bytes access `[]` method

You can access individual bytes as integers, to read and write. Values not in the range 0..255 are silently chopped.

```
b = bytes("010203")
print(b[0])          # 1

# negative indices count from the end
print(b[-1])         # 3

# out of bounds generate an exception
print(b[5])          # index_error: bytes index out of range

b[0] = -1
print(b)             # bytes('FF0203')

b[1] = 256
print(b)             # bytes('FF0003')
```

range access `[]` method

You can use the `[]` accessor with a range to get a sub-list of bytes. If an index is negative, it is taken from the end of the array.

This construct cannot be used as an *lvalue*, i.e. you can't splice like `b[1..2] = bytes("0011")` # not allowed.

```
b = bytes("001122334455")
print(b[1..2])       # bytes('1122')

# remove first 2 bytes
print(b[2..-1])      # bytes('22334455')
```

```
# remove last 2 bytes
print(b[0..-3])      # bytes('00112233')

# overshooting is allowed
print(b[4..10])      # bytes('4455')

# inversed indices return an empty array
print(b[5..4])       # bytes('')
```

The standard `item` and `setitem` methods are implemented, and transparently mapped to `[]` operator.

copy method

Creates a fresh new copy of the bytes object. A new memory buffer is allocated and data is duplicated.

```
b = bytes("1122")
print(b)          # bytes('1122')

c = b.copy()
print(c)          # bytes('1122')

b.clear()
print(b)          # bytes('')
print(c)          # bytes('1122')bytes('1122')
```

get, geti methods

Read a 1/2/4 bytes value from any offset in the bytes array. The standard mode is little endian, if you specify a negative size it enables big endian. `get` returns unsigned values, while `geti` returns signed values.

`b.get(<offset>, <size>)` -> bytes object

If the offset is out of range, 0 is returned (no exception raised).

Example:

```
b = bytes("010203040506")
print(b.get(2,2))      # 1027 - 0x0403 read 2 bytes little endian
print(b.get(2,-2))     # 772 - 0x0304 read 2 bytes big endian

print(b.get(2,4))      # 100992003 - 0x06050403 - little endian
print(b.get(2,-4))     # 50595078 - 0x03040506 - big endian

b = bytes("FEFF")
print(b.get(0, 2))     # 65534 - 0xFFFE
print(b.geti(0, 2))    # -2 - 0xFFFE
```

set, seti methods

Similar to `get` and `geti`, allows to set a 1/2/4 bytes value at any offset. `seti` uses signed integers, `set` unsigned (actually it does not make a difference).

If the offset is out of range, no change is done (no exception raised).

`bytes.set(<offset>, <value>, <size>)`

add method

This methods adds value of 1/2/4 bytes (little endian or big endian) at the end of the buffer. If size is negative, the value is treated as big endian.

`b.add(<value>, <size>)`

Example:


```
b = bytes("0011")
b.add(0x22, 1)
print(b)           # bytes('001122')
b.add(0x2233, 2)
print(b)           # bytes('0011223322')
b.add(0x22334455, 4)
print(b)           # bytes('001122332255443322')
b.add(0x00)
print(b)           # bytes('00112233225544332200')
b.clear()
b.add(0x0102, -2)
print(b)           # bytes('0102')
b.add(0x01020304, -4)
print(b)           # bytes('010201020304')
```

asstring method

Converts a bytes buffer to a string. The buffer is converted as-is without any encoding considerations. If the buffer contains NULL characters, the string will be truncated.

```
b=bytes("3344")
print(b.asstring())  # '3D'
```

fromstring method

Updates a bytes buffer from a string. The string is converted as-is without any encoding considerations. If the string contains NULL characters, it will be truncated.

```
b=bytes().fromstring("Hello")
print(b)           # bytes('48656C6C6F')
```

bits manipulation setbits, getbits methods

You can read and write at sub-byte level, specifying from which bit to which bit. The offset is in bits, not bytes. Add the number of bytes * 8.

```
b.setbits(<offset_bits>, <len_bits>, <value>)
b.getbits(<offset_bits>, <len_bits>)
```

base64 encode tob64 method

Converts a bytes array to a base64 string.

```
b = bytes('deadbeef0011')
s = b.tob64()
print(s)           # 3q2+7wAR
```

base64 decode fromb64 method

Converts a base64 string into a bytes array.

```
s = '3q2+7wAR'
b = bytes().fromb64(s)
print(b)           # bytes('DEADBEEF0011')
```

getfloat and setfloat methods

Similar to get/set, allows to read or write a 32 bits float value.

```
b.getfloat(<offset>)
b.getfloat(<offset>, <number>)
```

```
b = bytes("00000000")
b.getfloat(0)      # 0
b.setfloat(0, -1.5)
print(b)           # bytes('0000C0BF')
b.getfloat(0)      # -1.5
```

`_buffer` method

Advanced feature: returns the address of the buffer in memory, to be used with C code.

```
b = bytes('1122')
b._buffer()        # <ptr: 0x600000c283c0>
```

`_change_buffer` method

Advanced feature: works only for mapped buffers (i.e. `b.ismapped() == true`), allows to remap the buffer to a new memory address. This allows to reuse the `bytes()` object without reallocating a new instance.

```
# this example uses pointer allocation, use with great care
b1 = bytes("11223344")
b2 = bytes("AABBCCDD")
b1._buffer()      # <ptr: 0x600000c2c390>
b2._buffer()      # <ptr: 0x600000c24270>

# now we create c as a mapped buffer of 4 bytes to the address of b1
c = bytes(b1._buffer(), 4)
print(c)          # bytes('11223344') -- mapped to b1
c._buffer()        # <ptr: 0x600000c2c390>

# let's change a byte to prove it
c[0] = 254
print(c)          # bytes('FE223344')
print(b1)         # bytes('FE223344') -- b1 was changed

# reallocate c to map b2
c._change_buffer(b2._buffer())
print(c)          # bytes('AABBCCDD')
c._buffer()        # <ptr: 0x600000c24270>
```

Expansion Modules

JSON Module

JSON is a lightweight data exchange format. It is a subset of JavaScript. It uses a text format that is completely independent of the programming language to represent data. Berry provides a JSON module to provide support for JSON data. The JSON module only contains two functions `load` and `dump`, which are used to parse JSON strings and multiply Berry objects and serialize a Berry object into JSON text.

`load` function

`load`(text)

Description

This function is used to convert the input JSON text into a Berry object and return it. The conversion rules are shown in Table 1.1. If there is a syntax error in the JSON text, the function will return `nil`.

JSON type	Berry type
null	nil
number	integer or real

string	string
array	list
object	map

JSON type to Berry type conversion rules

Example

```
import json
json.load('0') # 0
json.load(' [{"name": "liu", "age": 13}, 10.0]') # [{'name': 'liu', 'age': 13}, 10]
```

dump function

```
dump(object, ['format'])
```

Description

This function is used to serialize the Berry object into JSON text. The conversion rules for serialization are shown in Table 1.2.

Berry type	JSON type
nil	null
integer	number
real	number
list	array
map	object
mapKey of	string
other	string

Berry type to JSON type conversion rules

Example

```
import json
json.dump('string') #'"string"'
json.dump('string') #'"string"'
json.dump({0:'item 0','list':[0, 1, 2]}) # '{"0": "item 0", "list": [0, 1, 2]}'
json.dump({0:'item 0','list':[0, 1, 2], 'func': print}, 'format')
#-
{
  "0": "item 0",
  "list": [
    0,
    1,
    2
  ],
  "func": "<function: 00410310>"
}
-#
```

Math Module

This module is used to provide support for mathematical functions, such as commonly used trigonometric functions and square root functions. To use the math module, first use the `import math` statement to import. All examples in this section assume that the module has been imported correctly.

pi constant

The approximate value of Pi π , a real number type, approximately equal to 3.141592654.

Example

```
math.pi # 3.14159
```

abs function

abs(value)

Description

This function returns the absolute value of the parameter, which can be an integer or a real number. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. **abs** The return type of the function is a real number.

Example

```
math.abs(-1) # 1
math.abs(1.5) # 1.5
```

ceil function

ceil(value)

Description

This function returns the rounded up value of the parameter, that is, the smallest integer value greater than or equal to the parameter. The parameter can be an integer or a real number. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. **ceil** The return type of the function is a real number.

Example

```
math.ceil(-1.2) # -1
math.ceil(1.5) # 2
```

floor function

floor(value)

Description

This function returns the rounded down value of the parameter, which is not greater than the maximum integer value of the parameter. The parameter can be an integer or a real number. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. **floor** The return type of the function is a real number.

Example

```
math.floor(-1.2) # -2
math.floor(1.5) # 1
```

round function

round(value)

Description

This function returns the rounded value of the parameter following C convention, rounded down if value is positive and rounded up if value is negative. This rounding feels generally more intuitive. The parameter can be an integer or a real number. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. **floor** The return type of the function is a real number.

Example

```
math.round(-1.2) # -1
math.round(-1.5) # -2
math.round(1.2) # 1
math.round(1.5) # 2
```

min function

```
min( [value] * )
```

Description

This function accepts any number of arguments and returns the minimum value of all arguments. Arguments can be of type *int* or *real*. The return value is *int* if all arguments are *int*, or *real* if at least one argument is *real*. An exception is raised if any argument is neither *int* or *real* including *nil*. The function returns *nil* if no argument is passed.

Example

```
math.min(0, -1, 2) # -1
math.min(10.2, 5, 3.5) # 3.5
math.min() # nil
math.min(1, nil) # "type_error" exception
```

max function

```
max( [value] * )
```

Description

This function accepts any number of arguments and returns the maximum value of all arguments. Arguments can be of type *int* or *real*. The return value is *int* if all arguments are *int*, or *real* if at least one argument is *real*. An exception is raised if any argument is neither *int* or *real* including *nil*. The function returns *nil* if no argument is passed.

Example

```
math.max(0, -1, 2) # 2
math.max(10.2, 5, 3.5) # 10.2
math.max() # nil
math.max(1, nil) # "type_error" exception
```

sin function

```
sin(value)
```

Description

This function returns the sine function value of the parameter. The parameter can be an integer or a real number, and the unit is radians. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. *sin* The return type of the function is a real number.

Example

```
math.sin(1) # 0.841471
math.sin(math.pi * 0.5) # 1
```

cos function

```
cos(value)
```

Description

This function returns the value of the cosine function of the parameter. The parameter can be an integer or a real number in radians. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. *cos* The return type of the function is a real number.

Example

```
math.cos(1) # 0.540302  
math.cos(math.pi) # -1
```

tan function

tan(value)

Description

This function returns the value of the tangent function of the parameter. The parameter can be an integer or a real number, in radians. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. tan The return type of the function is a real number.

Example

```
math.tan(1) # 1.55741  
math.tan(math.pi / 4) # 1
```

asin function

asin(value)

Description

This function returns the arc sine function value of the parameter. The parameter can be an integer or a real number. The value range is $[-1,1]$. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. asin The return type of the function is a real number and the unit is radians.

Example

```
math.asin(1) # 1.5708  
math.asin(0.5) * 180 / math.pi # 30
```

acos function

acos(value)

Description

This function returns the arc cosine function value of the parameter. The parameter can be an integer or a real number. The value range is $[-1,1]$. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. acos The return type of the function is a real number and the unit is radians.

Example

```
math.acos(1) # 0  
math.acos(0) # 1.5708
```

atan function

atan(value)

Description

This function returns the arctangent function value of the parameter. The parameter can be an integer or a real number. The value range is $[-\infty,+\infty]$. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. atan The return type of the function is a real number and the unit is radians.

Example

```
math.atan(1) * 180 / math.pi # 45
```

`sinh` function

`sinh`(value)

Description

This function returns the hyperbolic sine function value of the parameter. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. `sinh` The return type of the function is a real number.

Example

```
math.sinh(1) # 1.1752
```

`cosh` function

`cosh`(value)

Description

This function returns the hyperbolic cosine function value of the parameter. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. `cosh` The return type of the function is a real number.

Example

```
math.cosh(1) # 1.54308
```

`tanh` function

`tanh`(value)

Description

This function returns the hyperbolic tangent function value of the parameter. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. `tanh` The return type of the function is a real number.

Example

```
math.tanh(1) # 0.761594
```

`sqrt` function

`sqrt`(value)

Description

This function returns the square root of the argument. The parameter of this function cannot be negative. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. `sqrt` The return type of the function is a real number.

Example

```
math.sqrt(2) # 1.41421
```

`exp` function

`exp`(value)

Description

This function returns the value of the parameter's exponential function based on the natural constant e. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. `exp` The return type of the function is a real number.

Example

```
math.exp(1) # 2.71828
```

log function

log(value)

Description

This function returns the natural logarithm of the argument. The parameter must be a positive number. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. log The return type of the function is a real number.

Example

```
math.log(2.718282) # 1
```

log10 function

log10(value)

Description

This function returns the logarithm of the parameter to the base 10. The parameter must be a positive number. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. log10 The return type of the function is a real number.

Example

```
math.log10(10) # 1
```

deg function

deg(value)

Description

This function is used to convert radians to angles. The unit of the parameter is radians. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. deg The return type of the function is a real number and the unit is an angle.

Example

```
math.deg(math.pi) # 180
```

rad function

rad(value)

Description

This function is used to convert angles to radians. The unit of the parameter is angle. If there are no parameters, the function returns 0, if there are multiple parameters, only the first parameter is processed. rad The return type of the function is a real number and the unit is radians.

Example

```
math.rad(180) # 3.14159
```

pow function

pow(x, y)

Description

Welcome to berry's documentation!

The return value of this function is the result of the expression xy , which is the parameter x to the y power. If the parameters are not complete, the function returns 0, if there are extra parameters, only the first two parameters are processed. `pow` The return type of the function is a real number.

Example

```
math.pow(2, 3) # 8
```

srand function

`srand(value)`

Description

This function is used to set the seed of the random number generator. The type of the parameter should be an integer.

Example

```
math.srand(2)
```

rand function

`rand()`

Description

This function is used to get a random integer.

Example

```
math.rand()
```

Time Module

This module is used to provide time-related functions.

time function

`time()`

Description

Returns the current timestamp. The timestamp is the time elapsed since Unix Epoch (1st January 1970 00:00:00 UTC), in seconds.

dump function

`dump(ts)`

Description

The input timestamp `ts` is converted into a time map, and the key-value correspondence is shown in Table below:

key	value	key	value	key	value
`'year'`	Year (from 1900)	' month '	Month (1-12)	`'day'`	Day (1-31)
`'hour'`	Hour (0-23)	`'min'`	Points (0-59)	`'sec'`	Seconds (0-59)
'we ekday'	Week (1-7)				

`time.dump` The key-value relationship of the function return value

clock function

clock()

Description

This function returns the elapsed time from the start of execution of the interpreter to when the function is called in seconds. The return value of this function is of type `real`, and its timing accuracy is determined by the specific platform.

String Module

The String module provides string processing functions.

To use the string module, first use the `import string` statement to import. All examples in this section assume that the module has been imported correctly.

count function

```
string.count(s, sub[, begin[, end]])
```

Count the number of occurrences of the sub string in the string s. Search from the position between begin and end of s (default is 0 and size(s)).

split function

```
string.split(s, pos)
```

Split the string s into two substrings at position pos, and returns the list of those strings.

```
string.split(s, sep[, num])
```

Splits the string s into substrings wherever sep occurs, and returns the list of those strings. Split at most num times (default is `string.count(s, sep)`).

find function

```
string.find(s, sub[, begin[, end]])
```

Check whether the string s contains the substring sub. If the begin and end (default is 0 and size(s)) are specified, they will be searched in this range.

startswith function

```
string.startswith(s, sub[, case_insensitive])
```

Check whether the string starts with the substring sub; case-insensitive if `case_insensitive` is true.

endswith function

```
string.endswith(s, sub[, case_insensitive])
```

Check whether the string ends with the substring sub; case-insensitive if `case_insensitive` is true.

hex function

```
string.hex(number)
```

Convert number to hexadecimal string.

byte **function**

```
string.byte(s)
```

Get the code value of the first byte of the string s.

char **function**

```
string.char(number)
```

Convert the number used as the code to a character.

tolower **function**

```
string.lower(s)
```

Transforms the string s to lowercase, A-Z only

toupper **function**

```
string.toupper(s)
```

Transforms the string s to uppercase, a-z only

tr **function**

```
string.tr(s, chars, replacement)
```

Replaces any occurrence of character(s) from chars to corresponding replacements, or remove if replacement is empty (or shorter)

replace **function**

```
string.replace(s, text1, text2)
```

Replaces each occurrence of text1 with text2 (for single characters, this is slower than `string.tr()`)

escape **function**

```
string.escape(s[, berry_mode] )
```

Escapes the string with double quotes suitable for C, if berry_mode is true escape to single quotes suitable for Berry.

format **function**

```
string.format(fmt[, args])  
format(fmt[, args])
```

Returns a formatted string. The pattern starting with '%' in the formatting template fmt will be replaced by the value of [args]: %[flags][fieldwidth][.precision]type

Type	Description
%d %i	Decimal integer
%u	Unsigned decimal integer
%o	Octal integer
%x	Hexadecimal integer lowercase
%X	Hexadecimal integer uppercase

%f	Floating-point in the form [-]nnnn.nnnn
%e %E	Floating-point in exp. form [-]n.nnnn e [+ -]nnn, uppercase if %E
%g %G	Floating-point as %f if $-4 < \text{exp.} \leq \text{precision}$, else as %e; uppercase if %G
%c	Character having the code passed as integer
%s	String
%q	Escaped string
%%	The '%' character (escaped)

Type	Description
.	Left-justifies, default is right-justify
.	Prepends sign (applies to numbers)
(space)	Prepends sign if negative, else space
#	Adds "0x" before %x, force decimal point; for %e, %f, leaves trailing zeros for %g

Field width and precision	Description
n	Puts at least n characters, pad with blanks
0n	Puts at least n characters, left-pad with zeros
.n	Use at least n digits for integers, rounds to n decimals for floating-point or no more than n chars. for strings

F-strings (Formatted String Literals)

F-strings provide a more compact and readable way to format strings. They are syntactic sugar around the `format()` function and have the same performance.

F-strings are preceded by `f` and can use single or double quotes. Strings can be split across multiple literals and lines:

```
f"This uses double quotes"
f'This uses single quotes'
f"This" 'uses' "a combination" 'of quotes'
```

Values and expressions are surrounded by `{ }`:

```
name = "Alice"
f"Hello {name}"           # "Hello Alice"
f"1 + 1 is {1 + 1}"       # "1 + 1 is 2"
```

For literal brackets in the output, use double-brackets:

```
# JSON example
f'{{"name": "{name}"}}'    # '{"name": "Alice"}'
```

The default format is `%s` (string). You can specify a format after a colon `:` (the `%` character is not required):

```
price = 12.34
f"The price is {price:.2g}" # "The price is 12.34"
```

For debugging, use the equal sign `=` to dump a value with its name:

```
name = "bob"
price = 12.34
f"{name=} {price=:.2g}"    # "name=bob price=12.34"
```

More examples:

```
# Equivalent transformations
f"a = {self.a}"           # format("a = %s", self.a)
f"{self.a:04i}"          # format("%04i", self.a)
f"{self.a=}"             # format("self.a=%s", self.a)
f"{self.a=:g}"           # format("self.a=%g", self.a)
```

Module os

The OS module provides system-related functions, such as file and path-related functions. These functions are platform-related. Currently, Windows VC and POSIX style codes are implemented in the Berry interpreter. If it runs on other platforms, the functions in the OS module are not guaranteed to be provided.

os.getcwd function

```
os.getcwd()
```

Get the path of the current working directory.

Returns: String containing the current directory path.

os.chdir function

```
os.chdir(path)
```

Switch the current working directory to the specified path.

Parameters: - path: String containing the target directory path.

os.mkdir function

```
os.mkdir(path)
```

Create a directory at the specified path. Creates only one level of directory.

Parameters: - path: String containing the directory path to create.

os.remove function

```
os.remove(path)
```

Delete a file or directory at the specified path.

Parameters: - path: String containing the path of the file or directory to remove.

os.listdir function

```
os.listdir([path])
```

Return a list of file and folder names contained in the specified directory.

Parameters: - path: Optional string containing the directory path. Defaults to current directory ('.').

Returns: List of strings containing the names of files and directories.

os.system function

```
os.system(cmd[, args])
```

Execute a system command.

Parameters: - cmd: String containing the command to execute. - args: Optional additional arguments for the command.

`os.exit` **function**

```
os.exit()
```

Exit the interpreter process immediately.

Module `os.path`

The `os.path` module provides path manipulation utilities.

`os.path.isdir` **function**

```
os.path.isdir(path)
```

Check if the specified path is a directory.

Parameters: - `path`: String containing the path to check.

Returns: `true` if the path is a directory, `false` otherwise.

`os.path.isfile` **function**

```
os.path.isfile(path)
```

Check if the specified path is a file.

Parameters: - `path`: String containing the path to check.

Returns: `true` if the path is a file, `false` otherwise.

`os.path.exists` **function**

```
os.path.exists(path)
```

Check if the specified path exists (either as a file or directory).

Parameters: - `path`: String containing the path to check.

Returns: `true` if the path exists, `false` otherwise.

`os.path.split` **function**

```
os.path.split(path)
```

Split a path into directory name and base name components.

Parameters: - `path`: String containing the path to split.

Returns: List containing two strings: `[dirname, basename]`.

`os.path.splitext` **function**

```
os.path.splitext(path)
```

Split a path into filename and extension components.

Parameters: - `path`: String containing the path to split.

Returns: List containing two strings: `[filename, extension]`.

`os.path.join` **function**

```
os.path.join(args)
```

Concatenate path components into a complete path using the appropriate path separator for the current platform.

Parameters: - args: Variable number of string arguments representing path components.

Returns: String containing the joined path.

Module global

Module `global` provides a way to access global variables via a module. The Berry compiler checks that a global exists when compiling code. However there are cases when globals are created dynamically by code and are not yet known at compile time. Using the module `global` gives complete freedom to access statically or dynamically global variables.

Accessing a global is simply made with `global.<name>` for reading and writing. You can also use the special syntax `global.(name)` if `name` is a variable containing the name of the global as string.

Example:

```
> import global
> a = 1
> global.a
1
>
> b
syntax_error: stdin:1: 'b' undeclared (first use in this function)
> global.b = 2
> b
2
> global.b
2
> var name = "b"
> global.(name)
2
```

Calling `global()` returns the list of all global names currently defined (builtins are not included).

```
> import global
> a = 1
> global.b = 2
> global()
['_argv', 'b', 'global', 'a']
```

`global.contains(<name>)` -> bool provides an easy way to know if a global name is already defined.

```
> import global
> global.contains("g")
false
> g = 1
> global.contains("g")
true
```

Module introspect

Module `introspect` provides primitives to dynamically access variables or modules. Use with `import introspect`.

`introspect.members(object: class or module or instance or nil)` -> list returns the list of names of members for the class, instance or module. Keep in mind that it does not include potential virtual members created via `member` and `setmember`.

`introspect.members()` returns the list of global variables (not including builtins) and is equivalent to `global()`

`introspect.get(object: class or instance or module, name:string)` -> any and `introspect.set(object: class or instance or module, name:string, value:any)` -> nil allows to read and write any member by name.

`introspect.get(o, "a")` is equivalent to `o.a`, `introspect.set(o, "a", 1)` is equivalent to `o.a = 1`. There is also an alternative syntax: `o.( "a" )` is equivalent to `o.a` and `o.( "a" ) = 1` is equivalent to `o.a = 1`.

`introspect.module(name:string)` -> any is equivalent to `import name` except that it does not create the global or local variable, but returns the module. This is the only way to load a module with a dynamic name, `import name` only takes a static name.

`introspect.toptr(addr:int)` -> `comptr` converts an integer to a `comptr` pointer.
`introspect.fromptr(addr:comptr)` -> `int` does the reverse and converts a pointer to an `int`. Warning: use with care. On platforms where `int` and `void*` don't have the same size, these functions will most certainly give unusable results.

`introspect.ismethod(f:function)` -> `bool` checks if the provided function is a method of an instance (taking `self` as first argument), or a plain function. This is mainly use to prevent a common mistake of passing an instance method as callback, where you should use a closure capturing the instance like `/ -> self.do()`.

`introspect.name(obj:any)` -> `string` or `nil` returns the name of an object (function, class, module) if any or `nil`.

Module solidify

This module allows to solidify Berry bytecode into flash. This allows to save RAM since the code is in Flash, makes it a good alternative to native C functions.

See 8.4 Solidification

8. Advanced features

Table of Content

Placeholder for table of contents

0

8.1 strict mode

Berry allows full freedom from the developer. But after some experience in coding with Berry, you will find that there are common mistakes that are hard to find and that the compiler could help you catch. The `strict` mode does additional checks **at compile time** about some common mistakes.

This mode is enabled with `import strict` or when running Berry with `-s` option: `berry -s`

Mandatory var for local variables

This is the most common mistake, a variable assigned without `var` is either global if a global already exists or local otherwise. Strict mode rejects the assignment if there is no global with the same name.

No more allowed:

```
def f()  
  i = 0    # this is a local variable  
  var j = 0  
end
```

syntax_error: stdin:2: strict: no global 'i', did you mean 'var i'?

But still works for globals:

```
g_i = 0  
def f()  
  g_i = 1  
end
```

No overriding of builtins

Berry allows to override a builtin. This is however generally not desirable and a source of hard to find bugs.


```
map = 1
syntax_error: stdin:1: strict: redefinition of builtin 'map'
```

Multiple var with same name not allowed in same scope

Berry tolerated multiple declaration of a local variable with the same name. This is now considered as an error (even without strict mode).

```
def f()
  var a
  var a # redefinition of a
end
syntax_error: stdin:3: redefinition of 'a'
```

No hiding of local variable from outer scope

In Berry you can declare local variables with the same name in inner scope. The variable in the inner scope hides the variable from outer scope for the duration of the scope.

The only exception is that variables starting with dot '.' can mask from outer scope. This is the case with hidden local variable .it when multiple for are embedded.

```
def f()
  var a # variable in outer scope
  if a
    var a # redefinition of a in inner scope
  end
end
syntax_error: stdin:4: strict: redefinition of 'a' from outer scope
```

8.2 Virtual members

Virtual members allows you to dynamically and programmatically add members and methods to classes and modules. You are no more limited to the members declared at creation time.

This feature is inspired from Python's `__getattr__()` / `__setattr__()`. The motivation comes from LVGL integration to Berry in Tasmota. The integration needs hundreds of constants in a module and thousands of methods mapped to C functions. Statically creation of attributes and methods does work but consumes a significant amount of code space.

This features allows to create two methods:

Berry method	Description
member	(name:string) -> anyShould return the value of the specified name
setmember	(name:string, value:any) -> nilShould store the value to the virtual member with the specified name

module undefined

The `member()` function must be able to distinguish between a member with a nil value and the member not existing. To avoid any ambiguity, the `member()` function can indicate that the member does not exist in two ways:

- either raise an exception
- or `import undefined` and return the undefined module. This is used as a marker for the VM to know that the attribute does not exist, while benefitting from consistent exceptions

Example of a dynamic object to which you can add members, but would return an error if the member was not previously added.

```
class dyn
  var _attr
```

```
def init()
  self._attr = {}
end
def setmember(name, value)
  self._attr[name] = value
end
def member(name)
  if self._attr.contains(name)
    return self._attr[name]
  else
    import undefined
    return undefined
  end
end
end
```

Example of usage:

```
> a = dyn()
> a.a
attribute_error: the 'dyn' object has no attribute 'a'
stack traceback:
  stdin:1: in function main
> a.a = 1
> a.a
1
> a.a = nil
> a.a
>
```

implicit call of member ()

When the following code `a.b` is executed, the Berry VM does the following:

- Get the object named `a` (local or global), raise an exception if it doesn't exist
- Check if the object `a` is of type `module`, `instance` or `class`. Raise an exception otherwise
- Check if object `a` has a member called `b`. If yes, return its value, if no proceed below
- If object `a` is of type `class`, raise an exception because virtual members do not work for static (class) methods
- Check if object `a` has a member called `member` and it is a function. If yes call it with parameter "`b`" as string. If no, raise an exception
- Check the return value. If it is the module `undefined` raise an exception indicating that the member does not exist

implicit call of setmember ()

When the following code `a.b = 0` (mutator) is executed, the Berry VM does the following:

- Get the object named `a` (local or global), raise an exception if it doesn't exist
- Check if the object `a` is of type `module`, `instance` or `class`. Raise an exception otherwise
 - If `a` is of type `class`, check if member `b` exists. If yes, change its value. If no, raise an exception. (virtual members don't work for classes or static methods)
 - If `a` is of type `instance`, check if member `b` exists. If yes, change its value. If no, proceed below
 - Check if `a` has a member called `setmember`. If yes call it, if no raise an exception
 - If `a` is of type `module`. If the module is not read-only, create or change the value (`setmember` is never called for a writable module). If the module is read-only, then `setmember` is called if it exists.

Exception handling

To indicate that a member does not exist, `member()` shall return `undefined` after `import undefined`.

You can also raise an exception in `member()` but be aware that Berry might try to call methods like `tostring()` that will land on your `member()` method if they don't exist as static methods.

To indicate that a member is invalid, `setmember()` should raise an exception or return `undefined`. Returning anything else like `nil` indicates that the assignment was succesful.

Be aware that you may receive member names that are not valid Berry identifiers. The syntax `a.("<->")` will call `a.member("<->")` with a virtual member name that is not lexically valid, i.e. cannot be called in regular code, except by using indirect ways like `introspect` or `member()`.

Specifics for classes

Access to members of class object do not trigger virtual members. Hence it is not possible to have virtual static methods.

Specifics for modules

Modules do support reading static members with `member()`.

When writing to a member, the behavior depends whether the module is writable (in memory) or read-only (in firmware).

If the module is writable, the new members are added directly to the module and `setmember()` is never called.

If the module is read-only, then `setmember()` is called whenever you try to change or create a member. It is then your responsibility to store the values in a separate object like a global.

Example

Example:

```
class T
  var a
  def init()
    self.a = 'a'
  end

  def member(name)
    return "member "+name
  end

  def setmember(name, value)
    print("Set '"+name+"': "+str(value))
  end
end
t=T()
```

Now let's try it:

```
> t.a
'a'
> t.b
'member b'
> t.foo
'member foo'
> t.bar = 2
Set 'bar': 2
```

This works for modules too:

```
m = module()
m.a = 1
```

Welcome to berry's documentation!

```
m.member = def (name)
    return "member "+name
end
m.setmember(name, value)
    print("Set '"+name+"': "+str(value))
end
```

Trying:

```
> m.a
1
> m.b
'member b'
> m.c = 3    # the allocation is valid so `setmember()` is not called
> m.c
3
```

More advanced example:

```
> class A
    var i

    def member(n)
        if n == 'ii' return self.i end
        return nil    # we make it explicit here, but this line is optional
    end

    def setmember(n, v)
        if n == 'ii' self.i = v end
    end
end
> a=A()

> a.i    # returns nil
> a.ii   # implicitly calls `a.member("ii")`
attribute_error: the 'A' object has no attribute 'ii'
stack traceback:
  stdin:1: in function `main`
# returns an exception since the member is nil (considered is non-existent)

> a.ii = 42    # implicitly calls `a.setmember("ii", 42)`
> a.ii        # implicitly calls `a.member("ii")` and returns `42`
42
> a.i        # the concrete variable was changed too
42
```

8.3 How-to package a module

This guide drives you through the different options of packaging code for reuse using Berry's `import` directive.

Behavior of `import`

When you use `import <module> [as <name>]`, the following steps happen:

- There is a global cache of all modules already imported. If `<module>` was already imported, `import` returns the value in cache already returned by the first call to `import`. No other actions are taken.
- `import` searches for a module of name `<module>` in the following order:
 1. in native modules embedded in the firmware at compile time
 2. in file system, starting with current directory, then iterating in all directories from `sys.path`: look for file `<name>`, then `<name>.bec` (compiled bytecode), then `<name>.be`. If `BE_USE_SHARED_LIB` is enabled, it

also looks for shared libraries like `<name>.so` or `<name>.dll` although this optional is generally not available on MCUs.

- The code loaded is executed. The code should finish with a return statement. The object returned is stored in the global cache and made available to caller (in local or global scope).
- If the returned object is a module and if the module has an `init` member, then an extra step is taken. The function `<module>.init(m)` is called passing as argument the module object itself. The value returned by `init()` replaces the value in the global cache. Note that the `init()` is called at most once during the first import.

Note: an implicit `init(m)` function is always present in all modules, even if none was declared. This implicit function has no effect.

Packaging a module

Here is a simple example of a module:

File `demo_module.be`:

```
# simple module
# use `import demo_module`

demo_module = module("demo_module")

demo_module.foo = "bar"

demo_module.say_hello = def ()
    print("Hello Berry!")
end

return demo_module      # return the module as the output of import
```

Example of use:

```
> import demo_module

> demo_module
<module: demo_module>

> demo_module.say_hello()
Hello Berry!

> demo_module.foo
'bar'
> demo_module.foo = "baz"      # the module is writable, although this is highly discouraged
> demo_module.foo
'baz'
```

Package a singleton (monad)

The problem of using modules is that they don't have instance variables to keep track of data. They are essentially designed for state-less libraries.

Below you will find an elegant way of packaging a class singleton returned as an `import` statement.

To do this, we use different tricks. First we declare the class for the singleton as an inner class of a function, this prevents from polluting the global namespace with this class. I.e. the class will not be accessible by other code.

Second we declare a module `init()` function that creates the class, creates the instance and returns it.

By this scheme, `import <module>` actually returns an instance of a hidden class.

Example of `demo_monad.be`:

```
# simple monad
# use `import demo_monad`
```

Welcome to berry's documentation!

```
demo_monad = module("demo_monad")

# the module has a single member `init()` and delegates everything to the inner class
demo_monad.init = def (m)

  # inner class
  class my_monad
    var i

    def init()
      self.i = 0
    end

    def say_hello()
      print("Hello Berry!")
    end
  end

  # return a single instance for this class
  return my_monad()
end

return demo_monad      # return the module as the output of import, which is eventually repl
```

Example:

```
> import demo_monad
> demo_monad
<instance: my_monad(>      # it's an instance not a module

> demo_monad.say_hello()
Hello Berry!

> demo_monad.i = 42          # you can use it like any instance
> demo_monad.i
42

> demo_monad.j = 0          # there is strong member checking compared to modules
attribute_error: class 'my_monad' cannot assign to attribute 'j'
stack traceback:
  stdin:1: in function main
```

8.4 Solidification

Solidification is the process of capturing compiled Berry structures and code (classes, modules, maps, lists...) and storing them into firmware. It reduces dramatically the use of memory, but has some limitations.

solidify module

Solidification is handle by solidify module. This module is not compiled by default because of its size (~10kB). You need to compile with `#define BE_USE_SOLIDIFY_MODULE 1` directive.

The module has a single member `dump(x)` that takes a single argument (the object to solidify) and output to stdout the solidified code.

By default, solidify adds all string constants to the global pool. You can generate weak strings instead (eligible to pruning by the linker) by setting the second argument to `true`.

By default `solidify.dump` outputs the solidified code to standard output. You can specify a file as third argument. The file needs to be open in writable mode, and is not closed so that you can concatenate multiple objects.

```
solidify.dump(object:any, [, strings_weak:bool, file_out:file]) -> nil
```

Solidification of functions

You can solidify a single function.

Example:

```

> def f() return "hello" end
> import solidify
> solidify.dump(f)

/*****
** Solidified function: f
*****/
be_local_closure(f, /* name */
  be_nested_proto(
    0, /* nstack */
    0, /* argc */
    0, /* varg */
    0, /* has upvals */
    NULL, /* no upvals */
    0, /* has sup protos */
    NULL, /* no sub protos */
    1, /* has constants */
    ( &(const bvalue[ 1]) { /* constants */
      /* K0 */ be_nested_str(hello),
    }),
    &be_const_str_f,
    &be_const_str_solidified,
    ( &(const binstruction[ 1]) { /* code */
      0x80060000, // 0000 RET 1 K0
    })
  )
);
/*****/

```

To compile using weak strings (i.e. strings that can be eliminated by the linker if the object is not included in the target executable), use `solidify.dump(f, true)`:

```

/*****
** Solidified function: f
*****/
be_local_closure(f, /* name */
  be_nested_proto(
    0, /* nstack */
    0, /* argc */
    0, /* varg */
    0, /* has upvals */
    NULL, /* no upvals */
    0, /* has sup protos */
    NULL, /* no sub protos */
    1, /* has constants */
    ( &(const bvalue[ 1]) { /* constants */
      /* K0 */ be_nested_str_weak(hello),
    }),
    be_str_weak(f),
    &be_const_str_solidified,
    ( &(const binstruction[ 1]) { /* code */
      0x80060000, // 0000 RET 1 K0
    })
  )
);
/*****/

```

Solidification of classes

When you solidify a class, it embeds all the sub-elements. An C stub is also added to create the class and add it to the global scope.

```
> class demo
  var i
  static foo = "bar"

  def init()
    self.i = 0
  end

  def say_hello()
    print("Hello Berry!")
  end
end
> import solidify
> solidify.dump(demo)

/*****
** Solidified function: init
*****/
be_local_closure(demo_init, /* name */
  be_nested_proto(
    1, /* nstack */
    1, /* argc */
    2, /* varg */
    0, /* has upvals */
    NULL, /* no upvals */
    0, /* has sup protos */
    NULL, /* no sub protos */
    1, /* has constants */
    ( &(const bvalue[ 2]) { /* constants */
      /* K0 */ be_nested_str(i),
      /* K1 */ be_const_int(0),
    }),
    &be_const_str_init,
    &be_const_str_solidified,
    ( &(const binstruction[ 2]) { /* code */
      0x90020101, // 0000 SETMBR R0 K0 K1
      0x80000000, // 0001 RET 0
    })
  )
);
/*****/

/*****
** Solidified function: say_hello
*****/
be_local_closure(demo_say_hello, /* name */
  be_nested_proto(
    3, /* nstack */
    1, /* argc */
    2, /* varg */
    0, /* has upvals */
    NULL, /* no upvals */
    0, /* has sup protos */
    NULL, /* no sub protos */
    1, /* has constants */
```


Welcome to berry's documentation!

```
( (&(const bvalue[ 1]) { /* constants */
/* K0 */ be_nested_str(Hello_X20Berry_X21),
}),
&be_const_str_say_hello,
&be_const_str_solidified,
( (&(const binstruction[ 4]) { /* code */
0x60040001, // 0000 GETGBL R1 G1
0x58080000, // 0001 LDCONST R2 K0
0x7C040200, // 0002 CALL R1 1
0x80000000, // 0003 RET 0
}))
)
);
/*****

**** Solidified class: demo
*****/
be_local_class(demo,
1,
NULL,
be_nested_map(4,
( (struct bmapnode*) &(const bmapnode[]) {
{ be_const_key(i, -1), be_const_var(0) },
{ be_const_key(say_hello, 2), be_const_closure(demo_say_hello_closure) },
{ be_const_key(init, -1), be_const_closure(demo_init_closure) },
{ be_const_key(foo, 1), be_nested_str(bar) },
})),
(bstring*) &be_const_str_demo
);
/*****

void be_load_demo_class(bvm *vm) {
be_pushntvclass(vm, &be_class_demo);
be_setglobal(vm, "demo");
be_pop(vm, 1);
}
```

Sub-classes are also supported.

```
> class demo_sub : demo
var j

def init()
super(self).init()
self.j = 1
end
end
> solidify.dump(demo_sub)

/*****
**** Solidified function: init
*****/
be_local_closure(demo_sub_init, /* name */
be_nested_proto(
3, /* nstack */
1, /* argc */
0, /* varg */
0, /* has upvals */
```

```

NULL,          /* no upvals */
0,             /* has sup protos */
NULL,          /* no sub protos */
1,             /* has constants */
( (&const bvalue[ 3]) { /* constants */
/* K0 */ be_nested_str(init),
/* K1 */ be_nested_str(j),
/* K2 */ be_const_int(1),
}),
&be_const_str_init,
&be_const_str_solidified,
( (&const binstruction[ 7]) { /* code */
0x60040003, // 0000 GETGBL R1 G3
0x5C080000, // 0001 MOVE R2 R0
0x7C040200, // 0002 CALL R1 1
0x8C040300, // 0003 GETMET R1 R1 K0
0x7C040200, // 0004 CALL R1 1
0x90020302, // 0005 SETMBR R0 K1 K2
0x80000000, // 0006 RET 0
})
)
);
/*****

**** Solidified class: demo_sub
*****/

extern const bclass be_class_demo;
be_local_class(demo_sub,
1,
&be_class_demo,
be_nested_map(2,
( (struct bmapnode*) &(const bmapnode[]) {
{ be_const_key(init, -1), be_const_closure(demo_sub_init_closure) },
{ be_const_key(j, 0), be_const_var(0) },
})),
be_str_literal("demo_sub")
);
/*****

void be_load_demo_sub_class(bvm *vm) {
be_pushntvclass(vm, &be_class_demo_sub);
be_setglobal(vm, "demo_sub");
be_pop(vm, 1);
}

```

Solidification of modules

When you solidify a module, it embeds all the sub-elements. It also works with embedded lists or maps.

```

> def say_hello() print("Hello Berry!") end
> m = module("demo_module")
> m.i = 0
> m.s = "foo"
> m.f = say_hello
> m.l = [0, 1, "a"]
> m.m = {"a": "b", "2": 3}
> import solidify
> solidify.dump(m)

```

```

/*****
** Solidified function: say_hello
*****/
be_local_closure(demo_module_say_hello, /* name */
    be_nested_proto(
        2, /* nstack */
        0, /* argc */
        0, /* varg */
        0, /* has upvals */
        NULL, /* no upvals */
        0, /* has sup protos */
        NULL, /* no sub protos */
        1, /* has constants */
        ( &(const bvalue[ 1]) { /* constants */
            /* K0 */ be_nested_str(Hello_X20Berry_X21),
        }),
        &be_const_str_say_hello,
        &be_const_str_solidified,
        ( &(const binstruction[ 4]) { /* code */
            0x60000001, // 0000 GETGBL R0 G1
            0x58040000, // 0001 LDCONST R1 K0
            0x7C000200, // 0002 CALL R0 1
            0x80000000, // 0003 RET 0
        })
    )
);
/*****/

/*****
** Solidified module: demo_module
*****/
be_local_module(demo_module,
    "demo_module",
    be_nested_map(5,
        ( (struct bmapnode*) &(const bmapnode[]) {
            { be_const_key(l, -1), be_const_simple_instance(be_nested_simple_instance(&be_class_m
            be_const_list( * be_nested_list(3,
        ( (struct bvalue*) &(const bvalue[]) {
            be_const_int(0),
            be_const_int(1),
            be_nested_str(a),
        ))) } } ),
        { be_const_key(m, 3), be_const_simple_instance(be_nested_simple_instance(&be_class_m
        be_const_map( * be_nested_map(2,
        ( (struct bmapnode*) &(const bmapnode[]) {
            { be_const_key(a, -1), be_nested_str(b) },
            { be_const_key(2, -1), be_const_int(3) },
        ))) } } ),
        { be_const_key(i, 4), be_const_int(0) },
        { be_const_key(f, -1), be_const_closure(demo_module_say_hello_closure) },
        { be_const_key(s, -1), be_nested_str(foo) },
        )))
    );
BE_EXPORT_VARIABLE be_define_const_native_module(demo_module);
/*****/

```

Limitations of solidification

Solidification works for many objects: `class`, `module`, `functions` and embedded constants or objects like `int`, `real`, `string`, `list` and `map`.

Limitations:

- Upvals are not supported. You cannot solidify a closure that captures upvals from outer scope
- Capturing global variables requires to compile with `-g` “named globals” option (enabled by default on Tasmota)
- String constants are limited to 255 bytes, long strings (above 255 characters are not supported - because nobody ever had a need for)
- Solidified objects are read-only, this has some consequences on classes. You can solidify a class with its static members when it is created, but you cannot solidify a function that creates a class deriving from another class or with static members. The core reason is that setting the superclass or assigning the static members is implemented using mutating code on the new class - which cannot work on a read-only non-mutating class.
- Solidified code may be dependent on the size of `int` and `real` and may not be ported across MCUs with different sizes of types. You may need to re-solidify for each target.

9. FFI

Table of Content

Placeholder for table of contents

0

Foreign Function Interface (FFI) is an interface for interaction between different languages. Berry provides a set of FFI to realize the interaction with C language, this set of interfaces is also very easy to use in C++. Most of the FFI interfaces are functions, and their declarations are placed in the `berry.h` file. In order to reduce the amount of RAM used, FFI also provides a mechanism for generating a fixed hash table during C compilation. This mechanism must use external tools to generate C code.

9.1 Basics

The most important interactive function in FFI should be the function of calling Berry code and C function mutually. In order to realize that two languages call each other's functions, we must first understand the parameter passing mechanism of the Berry function.

9.1.1 Virtual Machine

Unlike compiled languages, Berry language cannot run directly on a physical machine, but in a specific software environment, which is **Virtual Machine** (VM). Similar to a real computer, the source code in text form cannot be executed in a virtual machine, but must be converted into “bytecode” by a compiler. The Berry virtual machine is defined as a C structure `bvm`, the content of this structure is invisible to FFI. Through some FFI functions, we can create and initialize a virtual machine. We introduce the use of virtual machines through a simple example:

```
void berry_test(void)
{
    bvm *vm = be_vm_new(); // Construct a VM
    be_loadstring(vm, "print('Hello Berry')"); // Compile test code
    be_pcall(vm, 0); // Call function
    be_vm_delete(vm); // Destroy the VM
}
```

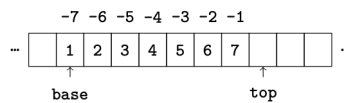
This code gives a complete example of using a virtual machine. First, call the function `be_vm_new` to construct a new virtual machine, and then all operations are completed in this virtual machine object. `be_vm_new` The function will automatically link the standard library when creating a virtual machine. The function of lines 4 to 5 is to compile the source code in a string into a Berry function and then call it. Finally, call the `be_vm_delete` function on line 6 to destroy the virtual machine. Executing this function will get a line of output in the terminal:

Hello Berry

In all scenarios, the virtual machine construction, library loading and destruction process are the same as the 3rd, 4th and 6th lines in the above example. If necessary, the way to compile or load the source code may be different. For example, for the source code in the form of a file, it can be compiled through the `be_loadfile` function. The source code will be compiled into a Berry function, and the function will be stored on the top of the stack. The Berry function can be executed by calling the FFI function `be_pcall` or `be_call`. You can also use the REPL through the `be_repl` function. The interface of the REPL will be described in the relevant chapters.

9.1.2 Virtual Stack

Berry uses a virtual stack and native functions written in C to pass values. Each element in the stack is a Berry value. When Berry code calls a native function, it always creates a new stack and pushes all the parameters onto the stack. This virtual stack can also be used in C code to store data, and the value stored in the stack will not be reclaimed by the garbage collector.



Virtual_Stack

The virtual stack used by Berry is shown in Figure above.

The virtual stack grows from left to right. When Berry code calls a native function, it will get an initial stack. The position of the first value of the stack is called **base**, and the last position is called **top**, the native function Only the value from the bottom of the stack to the position before the top of the stack can be accessed. The position of the bottom of the stack is fixed, while the position of the top of the stack can be moved, and the top of the stack is always empty. The reason for this property is that after the new value is pushed into the virtual stack, the original position of the top of the stack will be written The new value, the top pointer of the stack will move forward to the next position; conversely, if the value at the top of the virtual stack is popped, the top pointer of the stack will decrease 1. At this time, although the position of the top pointer of the stack is objectively Value, but this value is invalid and may be cleared at any time, so the pointer position on the top of the stack is still empty. When the virtual stack is empty, the bottom pointer base is equal to the top pointer top. The virtual stack does not strictly follow the operating rules of the stack: in addition to push and pop, the virtual stack can also be accessed by index, and even insert or delete values at any position. There are two ways to index elements in the stack: one is based on the bottom of the stack **Absolute index**, the absolute index value is a positive integer starting from 1; the other is based on the top of the stack **Relative index**, The relative index value is a negative integer starting from -1 . Take Figure above as an example, the index value 1, 2...8 is an absolute index, and the absolute index of an element is the distance from the element to the bottom of the stack. The index value $-1, -2 \dots -8$ is a relative index, and the relative index value of an element is the negative number of the distance from the element to the top of the stack. If an index value *index* is valid, then the element it refers to should be between the bottom of the stack and the top of the stack, which means that the expression

$1 \leq \text{abs}(*\text{index}*) \leq *top* - *base* + 1$ is satisfied.

For convenience, we stipulate that the stack bottom pointer base is used as a reference, and its absolute index 1, and the previous value of base is not considered (usually, base is not the bottom position of the entire stack). For example, when a native function returns, the location where the return value is stored is just before base, and these locations are usually not accessible by the native function.

Operate Virtual Stack

Index and stack size

As mentioned earlier, two indexing methods can be used to access the virtual stack, and the index value must be valid. At the same time, in many cases it is also necessary to push new values onto the stack. In this case, the programmer must ensure that the stack will not overflow. By default, Berry guarantees `BE_STACK_FREE_MIN` space for native functions to use. This value can be modified in the file *berry.h*. Its default value is usually 10, which should be sufficient in most cases. If you really need to expand the stack, you can call the FFI function `be_stack_require`. The prototype of this function is:

```
void be_stack_require(bvm *vm, int count);
```

The parameter count is the amount of space needed. When the remaining space in the virtual stack is insufficient, the stack capacity will be expanded, otherwise this function will do nothing.

caveat: If a stack overflow occurs, or if an invalid index is used to access the stack, the program will crash. You can turn on the debugging switch `BE_DEBUG` (section [section::BE_DEBUG]), which will turn on the assertion function, and you can get some debugging information at runtime to catch errors such as stack overflow or invalid index.

Get value from stack

There is a set of functions in FFI to get values from the virtual stack. These functions usually convert the values in the stack into simple values supported by the C language and then return. The following are commonly used FFIs to get values from the stack:

```
bint be_toint(bvm *vm, int index);
breal be_toreal(bvm *vm, int index);
int be_tobool(bvm *vm, int index);
const char* be_tostring(bvm *vm, int index);
void* be_tocomptr(bvm *vm, int index);
```

The parameter form of these functions is the same, but the return value is different. The first four functions are easy to understand. Just like their names, the function of `be_toint` is to convert the values in the virtual stack to C integer values (`bint` is usually an alias of type `int`) and return. The function of the last function `be_tocomptr` is to take out a pointer value of a general type from the virtual stack. The specific meaning of this pointer is explained by the C program itself.

These functions use the same way to interpret the parameters: the parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the element to be retrieved, which can be a relative index or an absolute index. You cannot use FFI to remove Berry's complex data types from the virtual stack, so you cannot remove a map type or class type from the stack. One of the benefits of this design is that there is no need to consider garbage collection in native functions.

Native function

Native function It is implemented by C language and can be called by Berry code. The native function can be an ordinary function. In this case, calling the native function will not generate any dynamically allocated space, just like a normal C function call. Native functions can also be closures, and space needs to be allocated for free variables when creating native closures. Under normal circumstances, simple native functions are sufficient to meet the needs. They save resources than native closures and are easier to use.

Define native function

The native function itself is a C function, but they all have a specific form. The definition of the native function is:

```
int a_native_function(bvm *vm)
{
    // do something ...
}
```

The native function must be a C function whose parameter is a pointer to `bvm` and the return value is `int`. Berry's functions must return a value, and native functions are no exception. Unlike the return value of the C language, the return value of the native function is not the value carried by the C return statement. You can use these FFIs to return the value of the native function, and they also make the C function return:

```
be_return(bvm *vm);
be_return_nil(bvm *vm);
```

These FFIs are actually two macros, and there is no need to use the C return statement when using them. `be_return` Will put the top of the virtual stack

Use native function

After the native function is defined, it must be added to the interpreter in some way before it can be called in Berry code. One of the easiest ways is to add native functions to Berry's built-in object table. The process of setting native objects as Berry built-in objects is called **registered**. The FFI of the registered native function is:

```
void be_regfunc(bvm *vm, const char *name, bntvfunc f);
```

vm is the current virtual machine instance, name is the name of the native function, and f is the pointer of the native function. The specific behavior of this function is related to the value of the BE_USE_PRECOMPILED_OBJECT macro (although the FFI is still available when using the compile-time construction technique, it cannot dynamically register the built-in variables. In this case, please refer to the method of registering the built-in objects. 1.3). The definition of native function type bntvfunc is:

```
typedef int (*bntvfunc)(bvm*);
```

In fact, the bntvfunc type is the function pointer type with the parameter bvm and the return value type int. be_regfunc The function should be called before parsing the Berry source code.

You can also push the native function into the virtual stack, and then use an FFI function be_call to call it. A more common usage is to use the native function object in the virtual stack as the return value.

Complete example

We end this section with a simple example. Here, we are going to implement a add function that adds two numbers and returns the result of the calculation. First, we define a native function to implement this function:

```
static int l_add(bvm *vm)
{
    int top = be_top(vm); // Get the number of arguments
    /* Verify the number and type of arguments */
    if (top == 2 && be_isnumber(vm, 1) && be_isnumber(vm, 2)) {
        breal x = be_toreal(vm, 1); // Get the first argument
        breal y = be_toreal(vm, 2); // Get the second argument
        be_pushreal(vm, x + y); // Push the result onto the stack
        be_return(vm); // Return the value at the top of the stack
    }
    be_return_nil(vm); // Return nil when something goes wrong
}
```

Native functions usually do not need to be used outside the C file, so they are generally declared as static types. Use the be_top function to get the absolute index of the top of the virtual stack (top value), which is the capacity of the stack. We can call be_top before the native function performs the virtual stack operation, at this time the virtual stack capacity is equal to the real parameter amount. For the add function, we need two parameters to participate in the operation, so check whether the number of parameters is 2 in the fourth line (top == 2). And we need to check whether the two parameters are both numeric types, so we need to call the be_isnumber function to check. If everything is correct, the parameters will be taken out of the virtual stack, then the calculation result will be pushed onto the stack, and finally returned using be_return. If the parameter verification fails, be_return_nil will be called to return the value of nil.

Next, register this native function to the built-in object table. For simplicity, we register it after loading the library:

```
bvm *vm = be_vm_new(); // Construct a VM
be_regfunc(vm, "myadd", l_add); // Register the native function "myadd"
```

The second line is where the native function is registered, and we named it myadd. At this point, the definition and registration of the native function is complete. As a verification, you can compile the interpreter, then enter the REPL and run some tests. You should get results like this:

```
> myadd
<function: 0x562a210f0f90>
> myadd(1.0, 2.5)
3.5
> myadd(2.5, 2)
4.5
> myadd(1, 2)
3
```

Types and Functions

Type

This section will introduce some types that need to be used in FFI. These types are generally used by FFI functions. Generally, the types and declarations in FFI can be found in the *berry.h* file. Unless otherwise specified in this section, the definition or declaration is provided in *berry.h* by default.

bvm Type is used to store the state information of the Berry virtual machine. Details of this type are not visible to external programs. Therefore, this definition can only be found in the *berry.h* file:

```
typedef struct bvm bvm;
```

Most FFI functions use the bvm type as the first parameter, because they all operate on the virtual machine internally. Hiding the internal implementation of bvm helps reduce the coupling between the FFI standard and the VM. Outside the interpreter, usually only bvm pointers are used. To create a new bvm object, use the `be_vm_new` function, and destroy the bvm object using the `be_vm_delete` function.

Native function type. The definition of this type is:

```
typedef int (*bntvfunc)(bvm*);
```

This type is a native function pointer, and some FFIs that register or add native functions to the virtual machine use parameters of this type. Variables or parameters of this type need to be initialized with a function name whose parameter is of type bvm and whose return value is of type int.

This type is used when registering native functions in batches or building native classes. It is defined as:

```
typedef struct {
    const char *name; // The name of the function or object
    bntvfunc function; // The function pointer
} bnfuncinfo;
```

The name member of bnfuncinfo represents the name of a function or object, and the function member is a native function pointer.

This type is a built-in integer type of Berry. It is defined in the *berry.h* document. By default, bint is implemented using the long long type, and the implementation of bint can be modified by modifying the configuration file.

This is Berry's built-in real number type, which is actually the floating point type in C language. breal is defined as:

```
#if BE_SINGLE_FLOAT != 0
    typedef float breal;
#else
    typedef double breal;
#endif
```

You can use the macro `BE_SINGLE_FLOAT` to control the specific implementation of breal: when the value of `BE_SINGLE_FLOAT` is 0, the double type implementation breal will be used, otherwise the float type implementation breal will be used.

[section::errorcode]

This enumeration type is used in some FFI return values. The definition of this type is:

```
enum berrorcode {
    BE_OK = 0,
    BE_IO_ERROR,
    BE_SYNTAX_ERROR,
    BE_EXEC_ERROR,
    BE_MALLOC_FAIL,
    BE_EXIT
};
```

The meaning of these enumeration values are:

- BE_OK: There is no error, the function is executed successfully.

- **BE_IO_ERROR**: A file reading error occurred when the interpreter was reading the source file. The error is generally caused by the absence of the file.
- **BE_SYNTAX_ERROR**: A syntax error occurred when the interpreter was compiling the source code. After this error occurs, the interpreter will not generate bytecode, so it cannot continue to execute bytecode.
- **BE_EXEC_ERROR**: Runtime error. When this error occurs, execution of Berry code is stopped and the environment is restored to the most recent recovery point.
- **BE_MALLOC_FAIL**: Memory allocation failed. This error is caused by insufficient heap space.
- **BE_EXIT**: Indicates that the program exits and the enumeration value is not an error. Executing Berry's `exit` function causes the interpreter to return this value.

It should be noted that when a **BE_MALLOC_FAIL** error occurs, dynamic memory allocation can no longer be performed, which means that string objects can no longer be allocated, so the function that returns this error usually does not give more detailed error information.

Functions and Macros

This function is used to create a new virtual machine instance. The function prototype is:

```
bvm* be_vm_new(void);
```

The return value of the function is a pointer to the virtual machine instance. `be_vm_new` The number is the first function called when the Berry interpreter is created. This function will do a lot of work: apply for memory for the virtual machine, initialize the state and attributes of the virtual machine, initialize the GC (garbage collector), and The standard library is loaded into the virtual machine instance, etc.

This function is used to destroy a virtual machine instance, the function prototype is:

```
void be_vm_delete(bvm *vm);
```

The parameter `vm` is the pointer of the virtual machine object to be destroyed. Destroying the virtual machine will release all the objects in the virtual machine, including the values in the stack and the objects managed by the GC. The virtual machine pointer after destruction will be an invalid value, and it can no longer be referenced.

This function is used to load a piece of source code from the buffer and compile it into bytecode. The prototype of the function is:

```
int be_loadbuffer(bvm *vm, const char *name, const char *buffer, size_t length);
```

The parameter `vm` is the virtual machine pointer. `name` is a string, which is usually used to mark the source of the source code. For example, the source code input from the standard input device can pass the string `"stdin"` to this parameter, and the source code input from the file can be The file name is passed to this parameter. `buffer` The parameter is the buffer for storing the source code. The organization of this buffer is very similar to the string of C. It is a continuous sequence of characters, but the buffer pointed to by `buffer` does not require `'\0'` characters as Terminator. `length` The parameter indicates the length of the buffer. This length refers to the number of bytes of source code text in the buffer.

To give a simple example, if we want to use the `be_loadbuffer` function to compile a string, the general usage is:

```
const char *str = "print('Hello Berry')";  
be_loadbuffer(vm, "string", str, strlen(str));
```

Here we use the string `"string"` to represent the source code, you can also modify it to any value. Note that the C standard library function `strlen` function is used here to get the length of the string buffer (actually the number of bytes in the string).

If the compilation is successful, `be_loadbuffer` will compile the source code into a Berry function and place it on the top of the virtual stack. If the compilation encounters an error, `be_loadbuffer` will return an error value of type `berrorcode` (Section `errorcode`, and if possible, will store the specific error message string at the top of the virtual stack.

`be_loadstring` is a macro defined as:

```
#define be_loadstring(vm, str) be_loadbuffer((vm), "string", (str), strlen(str))
```

This macro is just a simple wrapper for the `be_loadbuffer` function. The parameter is a pointer to the virtual machine instance, and the `str` parameter is a pointer to the source code string. It is very convenient to use `be_loadstring` to compile strings, for example:

```
be_loadstring(vm, "print('Hello Berry')");
```

This way of writing is more concise than using `be_loadbuffer`, but you must ensure that the string ends with a `'\0'` character.

This function is used to compile a source code file. The function prototype is:

```
int be_loadfile(bvm *vm, const char *name);
```

The function of this function is similar to the `be_loadbuffer` function, except that the function will be compiled by reading the source code file. The parameter `vm` is the pointer of the virtual machine instance, and the parameter `name` is the file name of the source file. This function will call the file interface, and by default it will use functions such as `fopen` in the C standard library to manipulate files.

If you use the file interface of the C standard library, you can use relative path or absolute path file names. If the file does not exist, `be_loadfile` will return a `BE_IO_ERROR` error (section errorcode) and push the error message onto the top of the stack. Other error messages are the same as the `be_loadbuffer` function. It is recommended to use the `be_loadfile` function to compile the source file, instead of reading all the source files into a buffer, and then call the `be_loadbuffer` function to compile the source code. The former will read the source file in segments and only create a small read buffer in the memory, thus saving more memory.

This function returns the absolute index value of the top element in the virtual stack. This value is also the number of elements in the virtual stack (the capacity of the virtual stack). Call this function before adding or subtracting elements in the virtual stack to get the number of parameters of the native function. The prototype of this function is:

```
int be_top(bvm *vm);
```

This function is usually used to obtain the number of parameters of a native function. When used for this purpose, it is recommended to call `be_top` at the top of the native function body. E.g:

```
static int native_function_example(bvm *vm)
{
    int argc = be_top(vm); // Get the number of arguments
    // ...
}
```

This function converts the type of the Berry object into a string and returns it. For example, it returns `"int"` for an integer object, and `"function"` for a function object. The prototype of this function is:

```
const char* be_typeof(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the object to be operated. The `type` function in the Berry standard library is implemented by calling `be_typeof`. Please refer to section `baselib_type` for the return string corresponding to the parameter type.

This function is used to get the class name of an object or class. The function prototype is:

```
const char* be_classname(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the object to be operated. If the value at `index` is an instance, the `be_classname` function will return the class name string to which the instance belongs, and if the value at `index` is a class, it will directly return the class name string. In other cases `be_classname` will return `NULL`.

This function returns the length of the specified Berry string. The function prototype is:

```
int be_strlen(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the object to be operated. This function returns the number of bytes in the string at `index` (the `'\0'` characters at the end of the Berry string are not counted). If the value of the `index` position is not a string, the `be_strlen` function will return `0`.

Although the Berry string is compatible with the C string format, it is not recommended to use the `strlen` function of the C standard library to measure the length of the Berry string. For Berry strings, `be_strlen` is faster than `strlen` and has better compatibility.

This function is used to splice two Berry strings. The function prototype is:

```
void be_strconcat(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance. This function will concatenate the string at the parameter position of `index` with the string at the top position of the stack, and then put the resulting string into the position indexed by `index`.

This function pops the value at the top of the stack. The function prototype is:

```
void be_pop(bvm *vm, int n);
```

The parameter `vm` is the pointer of the virtual machine instance, and the parameter `n` is the number of values to be popped from the stack. Note that the value of `n` cannot exceed the capacity of the stack.

This function will remove a value from the stack. The function prototype is:

```
void be_remove(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and the parameter `index` is the index of the object to be removed. After the value at `index` is moved out, the following values will be filled forward, and the stack capacity will be reduced by one. The value of `index` cannot exceed the capacity of the stack.

This function returns the absolute index value of a given index value, and its function prototype is:

```
int be_absindex(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and the parameter `index` is the input index value. If `index` is positive, the return value of `be_absindex` is the value of `index`. If `index` is negative, the return value of `be_absindex` is the absolute index value corresponding to `index`. When `index` is a negative value (relative index), its index position cannot be lower than the bottom of the stack.

This function creates a new list value, and its function prototype is:

```
void be_newlist(bvm *vm);
```

The parameter `vm` is the pointer of the virtual machine instance. After this function is successfully called, the new list value will be pushed onto the top of the stack. list value is an internal representation of a list, not to be confused with an instance of the list class.

This function creates a new map value, and its function prototype is:

```
void be_newmap(bvm *vm);
```

The parameter `vm` is the pointer of the virtual machine instance. After this function is successfully called, the new map value will be pushed onto the top of the stack. map value is an internal representation of a list, not to be confused with an instance of the map class.

This function pushes the global variable with the specified name onto the stack. Its function prototype is:

```
void be_getglobal(bvm *vm, const char *name);
```

The parameter `vm` is the pointer of the virtual machine instance, and the parameter `name` is the name of the global variable. After this function is called, the global variable named `name` will be pushed onto the top of the virtual stack.

This function is used to set the value of the member variable of the instance object class. The function prototype is:

```
void be_setmember(bvm *vm, int index, const char *k);
```

The parameter `vm` is the pointer of the virtual machine instance, the parameter `index` is the index of the instance object, and the parameter `k` is the name of the member. This function will copy the value at the top of the stack to the member `k` of the index position instance. Note that the top element of the stack will not pop up automatically.

This function is used to get the value of the member variable of the instance object class. The function prototype is:

```
void be_getmember(bvm *vm, int index, const char *k);
```

The parameter `vm` is the pointer of the virtual machine instance, the parameter `index` is the index of the instance object, and the parameter `k` is the name of the member. This function pushes the value of the member of the index position instance `k` onto the top of the virtual stack.

This function is used to get the value of list or map. The function prototype is:

```
void be_getindex(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and the parameter `index` is the index of the object to be operated. This function is used to get an element from the `map` or `list` container (internal values, not instances of `map` or `list` classes), and the index of the element is stored at the top of the stack (relative index is -1). After calling this function, the value obtained from the container will be pushed onto the top of the stack. If there is no subscript pointed to by the container, the value of `nil` will be pushed onto the top of the stack. For example, if the element with index 1 in the virtual stack is a `list`, and we want to extract the element with index 0 from it, then we can use the following code:

```
be_pushint(vm, 0); // Push the index value 0 onto the virtual-stack
be_getindex(vm, 1); // Get an element from the list container
```

We first push the integer value 0 onto the stack, and this value will be used as the index to get the element from the `list` container. The second line of code implements to get elements from the `list` container. The index value of the `list` container in the example is 1 in the virtual stack. The retrieved element is stored at the top of the stack, and we can use the relative index -1 to access it.

This function is used to set a value in `list` or `map`. The function prototype is:

```
void be_setindex(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and the parameter `index` is the subscript of the object to be operated. This function is used to write an element of the `map` or `list` container. The index of the value to be written in the virtual stack is -1 , and the index of the subscript of the write position in the virtual stack is -2 . If the element with the specified subscript does not exist in the container, the write operation will fail.

Assuming that the position with index 1 in the virtual stack has a value of `map`, and it has an element with a subscript of "test", an example of setting the element at the subscript of "test" to 100 is:

```
be_pushstring(vm, "test"); // Push the index "index"
be_pushint(vm, 100); // Push the value 100
be_setindex(vm, 1); // Set the key-value pair to map["test"] = 100
```

We must first push the subscript and the value to be written on the stack in order. For `map`, it is a key-value pair. In the example, the first two lines of code complete these tasks. The third line calls the `be_setindex` function to write the value into the `map` object.

This function is used to read an Up Value of the native closure. The function prototype is:

```
void be_getupval(bvm *vm, int index, int pos);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the native closure index value of the Up Value to be read; `pos` is the position of the Up Value in the native closure Up Value table (numbering starts from 0). The read Up Value will be pushed onto the top of the virtual stack.

This function is used to set an Up Value of the native closure. The function prototype is:

```
void be_setupval(bvm *vm, int index, int pos);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the native closure index value to be written into the Up Value; `pos` is the position of the Up Value in the native closure Up Value table (numbering starts from 0). This function obtains a value from the top of the virtual stack and writes it to the target Up Value. After the operation is completed, the top value of the stack will not be popped from the stack.

This function is used to get the parent object of the base class or instance of the class. The function prototype is:

```
void be_getsuper(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the class or object to be operated. If the value at `index` is a class with a base class, the function will push its base class onto the top of the stack; if the value at `index` is an object with a parent object, the function will take its parent. The object is pushed onto the top of the stack; otherwise, a value of `nil` is pushed onto the top of the stack.

This function is used to get the number of elements contained in the container. The function prototype is:

```
int be_data_size(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the container object to be operated. If the value at `index` is a Map value or List value, the function returns the number of elements contained in the container, otherwise it returns -1.

This function is used to append a new element to the end of the container. The function prototype is:

```
void be_data_push(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the container object to be operated. The object at `index` must be a List value. This function gets a value from the top of the stack and appends it to the end of the container. After the operation is completed, the value at the top of the stack will not be popped from the stack.

This function is used to insert a pair of elements into the container. The function prototype is:

```
void be_data_insert(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the container object to be operated. The object at `index` must be a List value or a Map value. The inserted element forms a pair of key-value pairs. The value is stored at the top of the stack, and the key is stored at the previous index on the top of the stack. It should be noted that the key inserted into the Map container cannot be a `nil` value, and the key inserted into the List container must be an integer value. If the operation is successful, the function will return `bttrue`, otherwise it will return `bfalse`.

This function is used to remove an element in the container. The function prototype is:

```
void be_data_remove(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the container object to be operated. The object at `index` must be a List value or Map value. For the Map container, the key to delete the element is stored on the top of the virtual stack (need to be pressed before the function call); for the List container, the index of the element to be deleted is stored on the top of the virtual stack (need to be before the function call push into). If the operation is successful, the function will return `bttrue`, otherwise it will return `bfalse`.

This function is used to reset the capacity of the container. The function prototype is:

```
void be_data_resize(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the container object to be operated. This function is only available for List containers, and the new capacity is stored on the top of the virtual stack (must be an integer).

This function is used to get the next element of the iterator. The function prototype is:

```
int be_iter_next(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the iterator to be operated. The iterator object may be an iterator of a List container or a Map container. For the List iterator, this function pushes the iteration result value onto the top of the stack, while for the Map iterator, it pushes the key value into the previous position and the top of the stack respectively. Calling this function will update the iterator. If the function returns 0, the call fails, returning 1 to indicate that the current iterator is a List iterator, and returning 2 to indicate that the current iterator is a Map iterator.

This function is used to test whether there is another element in the iterator. The function prototype is:

```
int map_hasnext(bvm *vm, int index)
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the iterator to be operated. The iterator object may be an iterator of a List container or a Map container. If there are more iterable elements in the iterator, return 1, otherwise return 0.

This function is used to test whether there is a reference to the specified object in the reference stack. It must be used in conjunction with `be_refpush` and `be_refpop`. This API can avoid recursion when traversing objects that have their own references. The function prototype is:

```
int be_refcontains(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the object to be operated. This function is used for the value of an instance type. If there is a reference to the object in the reference stack, it returns 1, otherwise it returns 0.

Push the reference of the specified object onto the reference stack. The function prototype is:

```
int be_refpush(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance; `index` is the index of the object to be operated. This function is used for the value of an instance type.

Pop the object at the top of the reference stack. The function prototype is:

```
int be_refpop(bvm *vm);
```

The parameter `vm` is the pointer of the virtual machine instance. This function is used in pairs with `be_refpush`. The following is the use of the reference stack API to avoid the problem of infinite recursive traversal when the object itself is referenced:

```
int list_traversal(bvm *vm)
{
    // ...
    if (be_refcontains(vm, 1)) {
        be_return(vm);
    }
    be_refpush(vm, 1);
    // Traversing the container, may call list_traversal recursively.
    be_refpop(vm);
    be_return(vm);
}
```

This is a simplified traversal process of the List container. For the complete code, please refer to the source code of the function `m_tostring` in `be_listlib.c`. We assume that the index of the List object is 1. First, we check whether the List already exists in the reference stack (line 4), and if the reference already exists, return directly, otherwise proceed with subsequent processing. To make `be_refcontains` usable, we need to use `be_refpush` and `be_refpop` to process the reference stack before and after the actual traversal operation (lines 7 and 9).

This function tests the amount of free space on the stack and expands the stack space if it is insufficient. The function prototype is:

```
void be_stack_require(bvm *vm, int count);
```

The parameter `vm` is the pointer of the virtual machine instance; `count` is the required free stack capacity. If the free capacity of the virtual stack allocated by the VM to the native function is lower than this value, an expansion operation will be performed.

This function returns whether the value indexed by the parameter `index` in the virtual stack is `nil`, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isnil(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured.

This function returns whether the value indexed by the parameter `index` in the virtual stack is of type `bool`, if it is, the function returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isbool(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured.

This function returns whether the value indexed by the parameter `index` in the virtual stack is an integer type, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isint(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured.

This function returns whether the value indexed by the parameter `index` in the virtual stack is a real number type, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:


```
int be_isreal(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is an integer or a real number type, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isnumber(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is a string type, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isstring(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is a closure type, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isclosure(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is a primitive closure type, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isntvclos(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is a function type, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isfunction(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. There are three types of functions: closure, native function and native closure.

This function returns whether the value indexed by the parameter `index` in the virtual stack is of type `proto`, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isproto(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. `proto` The type is the function prototype of the Berry closure.

This function returns whether the value indexed by the parameter `index` in the virtual stack is of type `class`, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isclass(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is of type `instance`, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isinstance(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is an instance or sub-instance of class `bytes`, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_isbytes(bvm *vm, int index);
```

This function returns whether the value indexed by the parameter `index` in the virtual stack is of type `list`, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_islist(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is of type `map`, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_ismap(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured. This function returns whether the value indexed by the parameter `index` in the virtual stack is a universal pointer type, if it is, it returns 1, otherwise it returns 0. The prototype of this function is:

```
int be_iscomptr(bvm *vm, int index);
```

The parameter `vm` is the pointer of the virtual machine instance, and `index` is the index of the value to be measured.

```
bint be_toint(bvm *vm, int index);
```

Get the value of the index position of `index` from the virtual stack and return it as an integer type. This function does not check the correctness of the type. If the value is an instance, the method `toint()` is called if it exists.

```
breal be_toreal(bvm *vm, int index);
```

Get the value of the index position of `index` from the virtual stack and return it as a floating-point number type. This function does not check the correctness of the type.

```
bint be_toindex(bvm *vm, int index);
```

Get the value of the index position of `index` from the virtual stack and return it as an integer type. This function does not check the correctness of the type. Unlike `be_toint`, the return value type of `be_toindex` is `int`, while the return value of the former is `bint`.

```
bbool be_tobool(bvm *vm, int index);
```

Get the value of the index position of `index` from the virtual stack and return it as a Boolean type. If the indexed value is not of Boolean type, it will be converted according to the rules in section `type_bool`, and the conversion process will not cause the indexed value to change. If the value is an instance, the method `tobool()` is called if it exists.

```
const char* be_tostring(bvm *vm, int index);
```

Get the value of the index position of `index` from the virtual stack and return it as a string type. If the indexed value is not a string type, the indexed value will be converted to a string, and the conversion process will replace the value at the indexed position in the virtual stack with the converted string. The string returned by this function always ends with `'\0'` characters. If the value is an instance, the method `tostring()` is called if it exists.

```
void* be_tocomptr(bvm* vm, int index);
```

Get the value of the index position of `index` from the virtual stack and return it as a general pointer type. This function does not check the correctness of the type.

```
const void* be_tobytes(bvm *vm, int index, size_t *len);
```

Get the value of the index position of `index` from the virtual stack and return it as a bytes buffer. The pointer of the buffer is returned, and the size is stored in `*len` (unless `len` is NULL). This function works only for instances of the `bytes` class, or returns NULL.

```
void be_pushnil(bvm *vm);
```

Push a `nil` value onto the virtual stack.

```
void be_pushbool(bvm *vm, int b);
```

Push a Boolean value onto the virtual stack. The parameter `b` is the boolean value to be pushed onto the stack. When the value is 0, it means false, otherwise it is true.

```
void be_pushint(bvm *vm, bint i);
```

Push an integer value `i` onto the virtual stack.

```
void be_pushreal(bvm *vm, breal r);
```


Push a floating point value `r` onto the virtual stack.

```
void be_pushstring(bvm *vm, const char *str)
```

Push the string `str` onto the virtual stack. The parameter `str` must point to a C string that ends with a null character `'\0'`, and a null pointer cannot be passed in.

```
void be_pushnstring(bvm *vm, const char *str, size_t n);
```

Push the string `str` of length `n` onto the virtual stack. The length of the string is subject to the parameter `n`, and the null character is not used as the end mark of the string.

```
const char* be_pushfstring(bvm *vm, const char *format, ...);
```

Push the formatted string into the virtual stack. The parameter `format` is a formatted string, which contains the text to be pushed onto the stack, and the format parameter contains a label, which can be replaced by the value specified by the subsequent additional parameter and formatted as required. According to the label of the format string, a series of additional parameters may be required, and each additional parameter will replace the corresponding `%` label in the format parameter.

specifier	Description	
d	Format as decimal signed integer (positive numbers do not output sign)	
f	Single or double precision floating point number formatted as a decimal	
g	Single or double precision floating point number formatted as exponential	
s	Format as string	
c	Format as a single character	
p	Format as pointer address	
%	Escaped as <code>%</code> Character (no parameter)	

format Label parameter description

`be_pushfstring` The function is similar to the standard function of C `printf`, but the function of formatting strings is relatively basic and does not support operations such as customizing the width and decimal places. A typical example is:

```
be_pushfstring(vm, "%s: %d", "hello", 12); // Good, it works!  
be_pushfstring(vm, "%s: %.5d", "hello", 12); // Error, the specified width is not supported.
```

This means that `be_pushfstring` can only perform simple formatting operations. If the requirements cannot be met, it is recommended to use `sprintf` formatted strings for operations.

```
void be_pushvalue(bvm *vm, int index);
```

Push the value with index `index` onto the top of the virtual stack.

```
void be_pushntvclosure(bvm *vm, bntvfunc f, int nupvals);
```

Push a native closure onto the top of the virtual stack. The parameter `f` is the C function pointer of the native closure, and `nupvals` is the upvalue number of the closure.

```
void be_pushntvfunction(bvm *vm, bntvfunc f);
```

Push a native function onto the top of the virtual stack, and the parameter `f` is the native function pointer.

```
void be_pushclass(bvm *vm, const char *name, const bnfncinfo *lib);
```

Push a native class onto the top of the virtual stack. The parameter `name` is the name of the native class, and the parameter `lib` is the attribute description of the native class.

```
void be_pushcomptr(bvm *vm, void *ptr);
```

Push a general pointer onto the top of the virtual stack. The general pointer `ptr` points to a certain C data area. Since the content pointed to by this pointer is not maintained by Berry's garbage collector, users have to maintain the life cycle of the data themselves.

```
void* be_pushbytes(bvm *vm, const void *buf, size_t len);
```

Push a `bytes()` buffer starting at position `buf` and of size `len`. The buffer is copied into Berry allocated memory, you don't need to keep the buffer valid after this call.

```
bbool be_pushiter(bvm *vm, int index);
```

Push an iterator onto the top of the virtual stack.

Push an error message onto the top of the stack. After executing the FFI, the interpreter will directly return to the position that can handle the error, and the code immediately following will not be executed. The function prototype is:

```
void be_pusherror(bvm *vm, const char *msg);
```

The parameter `vm` is the pointer of the virtual machine instance; `msg` is the string containing the error information.

Move the value at the `from` index to the `to` index position. This function does not delete the value of `from` index position, only modifies the value of `to` index position.

Compile-time construction technology

The compile-time construction technology is mainly implemented by `coc` which is located in the `coc/coc` path of the interpreter source code directory. `coc` Tool is used to generate constant strings, and constant objects as C code, and will be compiled into constants when the interpreter is compiled. In principle, the `coc` tool will generate code from the declaration information of the constant object (in accordance with a specific format). The process will automatically calculate the Hash value and generate the Hash table.

`Makefile` in the root directory of the interpreter project will automatically compile this tool and run the tool before compiling the interpreter source code. The content of `Makefile` ensures that when using the `make` command, the code for constructing the object at compile time will always be updated through the tool (if it needs to be updated). The code for constructing objects at compile time can be manually generated through the `make prebuild` command, which is stored in the `generate` folder.

The compile-time construction can be turned on or off by modifying the `BE_USE_PRECOMPILED_OBJECT` macro. In any case, the tool `coc` is called to generate constant object codes (the codes are not used when compile-time construction is turned off).

Use command

`coc` Tool is used to generate code for constant objects. The format of the command is

```
tools/coc/coc -o <dst_path> <src_path(s)> -c <include_path>
```

The output path `dst_path` is used to store the generated code, and the source path `src_path` is a list of paths that need to be scanned for the source code (use spaces to separate multiple paths). `include_path` contains a C header file scanned to detect compilation directives. `coc` tries to compile only the necessary constants. Since `generate` is used as the generated code path in the source code of the interpreter, `dst_path` must be `generate`. Taking the standard interpreter project as an example, the command to use the tool in `map_build` should be

```
tools/coc/coc -o generate default src -c default/berry_conf.h
```

The meaning of this command is: the output path is `generate`, and the source path is `src` and `default`.

output path

Strictly speaking, the `generate` folder used as the output path cannot be placed anywhere, it must be stored in a parent directory containing the path. The include path refers to the path where the header file will be searched in the project. Taking the standard interpreter source code as an example, the include path is `src` and `default`. Therefore, in the standard interpreter project, the `generate` folder is stored in the root directory of the interpreter source code (the parent directory of `src` and `default`).

The reason for the above rules is that the following codes are used in the interpreter source code to refer to constant objects:

```
#include "../generate/be_fixed_xxx.h"
```

If readers want to define constant objects themselves, they also need to use such code to include the corresponding header files. This section will introduce how to use these tools to define and use constant objects.

Compile-time string table

The compile-time string table is used to store constant strings. Constant strings are objects that are transparent to the script. They are not created or destroyed when the interpreter is running, but are always stored as constants in the data segment of the interpreter program. If you need to use a string as a constant string, you can add the prefix `be_const_str_` in front of the string in the interpreter source code, and the declaration can be placed anywhere in the source file (including comments). For example, to create a constant string with the content "string", you need to declare the symbol `be_const_str_string` in the source file, and this symbol is also the variable name that refers to the constant string in the C code.

All keywords will create constant strings. If you modify the keyword-related code in the Berry interpreter, the corresponding code in `coc` must also be modified.

If the string contains special symbols, they are automatically escaped as `_XHH` where `HH` is the hex representation (uppercase) of the character. For example " is represented by `_X3A`. This representation is bijective so it's easy to convert to and from the original string.

Use constant string

Normally, there is no need to manually declare constant strings, nor to use them manually. If you really need to call the constant string manually, include the header file `be_constobj.h` to use all constant string variables (this header file has declarations for all constant strings). The typical use of constant strings is to construct objects at compile time. The declaration and definition of constant strings in this process are automatically handled by the tool.

In any case, the FFI function `be_pushstring` should be used directly to create a string. When a string has a constant string, it will not repeatedly create a new string object, but directly use the corresponding constant string.

By default, all strings used are referenced in a global `m_const_string_table` hashtable. However, some projects may have many compilation variants for which some sets of string are not needed. If all string constants are stored in all variants, this creates a waste of flash size. For this reason, some strings can be declared as weak strings in the sense of having a weak reference. In such case the string constant is declared in C code, but not included in the global map object. This means that the linker can choose to not include the string constants if it is not referenced by any code. The con is that if you dynamically create a string object with the same value, a new object is created in memory (while it would not for a regular string constant). To indicate weak strings, use the `strings: weak` modifier (see below).

Construct object at compile time

Objects constructed at compile time are also called constant objects. The data structure of these objects is constructed when the interpreter is compiled and cannot be modified at runtime. `map_build` A set of declaration rules is defined in the tool to generate C code for constant objects. The declaration information of the constant object is directly stored in the source file (*.c). In order to distinguish it from other content, a complete declaration information should be included in the following boot code:

```
@const_object_info_begin
@const_object_info_end
```

The constant object declaration information does not conform to the C language syntax, so they should be placed in a multi-line comment (included with `/* */`). All constant objects have the same declaration form. The declaration structure of a constant object is called "object declaration block", which is composed of

```
type object_name (attributes) {
    member_fields
}
```

`type` is the type of constant object, it can be `map`, `class`, `module` or `vartab`. `object_name` is the variable name of the constant object in C language. `attributes` is the attribute list of constant objects. An attribute is composed of attribute name and attribute value. The attribute name and attribute value are separated by semicolons, and multiple attributes are separated by commas. For example, the attribute list `scope: global, name: map` means

that the `scope` attribute of a constant object is `global`, and the `name` attribute is `map`. Also `strings: weak` indicate to generate weak string constants for the names of member fields or any string constant. `member_fields` is the list of member domains of constant objects. A member is composed of name and value, separated by commas. Each line can declare one member, and multiple members must be declared on multiple lines.

The **coc** tool uses regular expressions to parse the object declaration block. In the parsing process, the entire object declaration block will be matched first, and the information type and `object_name` will be matched. For the information of `attributes` and `member_fields`, further Analysis. In order to facilitate the implementation, the tool does not have strict requirements on the syntax of the object declaration block, and lacks a complete error handling mechanism, so you should ensure that the syntax is correct when writing the object declaration block.

In order to facilitate understanding, we illustrate with a simple constant class:

```
/* @const_object_info_begin
class be_class_map (scope: global, name: map) {
    .data, var
    init, func(m_init)
    tostring, func(m_tostring)
}
@const_object_info_end */
#include "../generate/be_fixed_be_class_map.h"
```

In this example, the declaration information of the entire constant class is in the C language comment, so it will not affect the compilation of the C code. The object declaration block is placed between `@const_object_info_begin` and `@const_object_info_end` to ensure that the **coc** tool detects the object declaration block.

Since it is a constant class declaration, the value of `type` in the object declaration block is `class`, and `be_class_map` is the variable name of the constant object in the C code. Two attributes are declared in the attribute list of the object (the part enclosed in parentheses), and the meaning of these attributes will be introduced in the "Compile-Time Construction Class" section of this section. Three members are defined in the member list surrounded by curly braces, multiple members are separated by newlines, and the name of the member and the value of the member are separated by a comma. There are several legal formats for member names:

- Berry variable name format: start with a letter or underscore, followed by several letters, underscores or numbers.
- Use "." as the first character, followed by letters, underscores or numbers.
- Overloadable operators, such as "+", "-" and "<<" etc.

The value of a member can be of the following types:

- `var`: This symbol will be compiled into an integer object (`be_const_var`), and the value of the integer object is automatically incremented from 0. `var` It is designed for the declaration of member variables in the class, and its automatic numbering feature is used to realize the serial number of member variables.
- `func(symbol)`: Declare native member functions or methods of constant objects. The symbol will be compiled into a native function value (`be_const_func`), `symbol` is the native function pointer corresponding to the member. `m_init` and `m_tostring` in the example are two native functions.
- `closure(symbol)`: Declare pre-compiled bytecode member functions or methods of constant objects. The symbol will be compiled into a native function value (`be_const_closure`), `symbol` is the name of the solidified function. See module `solidify` to know how to solidify objects.
- `nil()`: This symbol will be compiled into an nil value (`be_const_nil`).
- `int(value)`: This symbol will be compiled into an integer object (`be_const_int`), the value of the integer object is `value`.
- `real(value)`: This symbol will be compiled into a real number object (`be_const_real`), the value of the real number object is `value`.
- `comptr(value)`: This symbol will be compiled into a pointer object (`be_const_comptr`), the value of the pointer is `value` and can be used to pass the address of a C global structure.
- `class(symbol)`: This symbol will be compiled into a class object (`be_const_class`). `symbol` is a pointer to this type of object, and the pointer needs to point to a constant type object.

- `module(symbol)`: This symbol will be compiled into a module object (`be_const_module`). `symbol` is a pointer to the module object, and the pointer needs to point to a constant module object.
- `ctype_func(symbol)`: This symbol will be compiled into a native function (`be_const_ctype_func`). `symbol` is a pointer to the C mapping definition. This feature is used by [berry_mapping](#)

In order to use the `be_class_map` object, we need to include the corresponding header file in the C code to ensure that the object will be compiled. The usual practice is to include the corresponding header file near the object declaration block. In the example, line 8 contains it. The corresponding header file can be used to construct `be_class_map` objects at compile time.

After processing by the `coc` tool, each object declaration block will be compiled into a header file named `be_fixed_be_xxx.h`, and `xxx` is the C variable name of the object. In order to compile constant objects in C code, we need to include the corresponding header files. It is usually recommended to include the corresponding header files near the object declaration block. The 8th line in the example contains `be_fixed_be_class_map.h` to construct the `be_class_map` object at compile time.

Construct Map at Compile Time

Maps constructed at compile-time are also constant map objects. They are generally not declared directly using object declaration blocks, but are declared in other compile-time construction structures. When constructing the constant map, the constant object type information should be `map`, which supports a scope attribute. When the scope attribute value is `local`, the constant object is `static`, the attribute When it is `global`, it is `extern`, and the value of this attribute is `local` by default. The constant map object's `member_fields` supports common member name/member value specifications, and member values are only stored as data without special interpretation. The following is an example of using the object declaration block to directly declare a constant map object:

```
map map_name (scope: local/global) {
    init, func(m_init)
}
```

Compile-time construction class

To construct a class at compile time, use the object declaration block to declare, and the type information of the object is `class`. The declared properties of this object are scope and name. scope The scope of the C variable of the attribute declaration object, when the value is `local` (default), the scope is `static`, when it is `global`, the scope is `extern`; name The value of the attribute is that class Name, anonymous class can omit this parameter. Since the attribute list of a class only stores methods and member variable indexes, the `member_fields` of the class constructed at compile time can only use the values `var` and `func()`. A simple compile-time construction class declaration block is:

```
class be_class_map (scope: global, name: map) {
    .data, var
    init, func(m_init)
    tostring, func(m_tostring)
}
```

Building Module at Compile Time

The type information of the building block declaration block at compile time is `module`.

```
module math (scope: global) {
    sin, func(m_sin)
    cos, func(m_cos)
    pi, real(M_PI)
}
```

Construct Built-in Domain at Compile Time

```

vartab m_builtin (scope: local) {
  assert, func(l_assert)
  print, func(l_print)
  list, class(be_class_list)
}

```

Grammar Definition

This chapter will give some grammar definitions related to Berry. We use **Extended Backus Normal Form** (EBNF) to define or express grammar. We did not use strict EBNF grammar to define, but made a lot of simplifications, but these simplifications will not affect readers' understanding of the grammar.

The EBNF definition of Berry language grammar is as follows:

```

(* program define *)
program = block;
(* block define *)
block = {statement};
(* statement define *)
statement = class_stmt | func_stmt | var_stmt | if_stmt | while_stmt |
             for_stmt | break_stmt | return_stmt | expr_stmt | import_stmt |
             try_stmt | throw_stmt | ;;
if_stmt = 'if' expr block {'elif' expr block} ['else' block] 'end';
while_stmt = 'while' expr block 'end';
for_stmt = 'for' ID ':' expr block 'end';
break_stmt = 'break' | 'continue';
return_stmt = 'return' [expr];
(* function define statement *)
func_stmt = 'def' ID func_body;
func_body = '(' [arg_field {',' arg_field}] ') block 'end';
arg_field = ['*'] ID;
(* class define statement *)
class_stmt = 'class' ID [':' ID] class_block 'end';
class_block = {'var' ID {',' ID} | 'static' ['var'] ID ['=' expr] {',' ID ['=' expr] } | 'st
import_stmt = 'import' (ID ('as' ID) | {',' ID}) | STRING 'as' ID);
(* exceptional handling statement *)
try_stmt = 'try' block except_block {except_block} 'end';
except_block = except_stmt block;
except_stmt = 'except' (expr {',' expr} | '..') ['as' ID [',' ID]];
throw_stmt = 'raise' expr [',' expr];
(* variable define statement *)
var_stmt = 'var' ID ['=' expr] {',' ID ['=' expr]};
(* expression define *)
expr_stmt = expr [assign_op expr];
expr = suffix_expr | unop expr | expr binop expr | range_expr | cond_expr;
cond_expr = expr '?' expr ':' expr; (* conditional expression *)
assign_op = '=' | '+=' | '-=' | '*=' | '/=' |
             '%=' | '&=' | '|=' | '^=' | '<=' | '>=';
binop = '<' | '<=' | '==' | '!=' | '>' | '>=' | '|' | '&&' |
          '<<' | '>>' | '&' | '|' | '^' | '+' | '-' | '*' | '/' | '%';
range_expr = expr '..' [expr]
unop = '-' | '!' | '~';
suffix_expr = primary_expr {call_expr | ('.' ID) | '[' expr ']'};
primary_expr = '(' expr ') | simple_expr | list_expr | map_expr | anon_func | lambda_expr;
simple_expr = INTEGER | REAL | STRING | ID | 'true' | 'false' | 'nil';
call_expr = '(' [expr {',' expr}] ');
list_expr = '[' {expr ','} [expr] ']';
map_expr = '{' {expr ':' expr ','} [expr ':' expr] '}';

```


- C++ compiler supporting C++11 standard (only for native compilation)
- 32 or 64 bit target platform

The C++ compiler is only used to compile *map_build* tools, so there is no need to provide a C++ cross compiler for the Berry interpreter when cross-compiling, but the user should prepare a native C++ compiler (unless the user can obtain the *map_build* tool executable file).

2. Porting

The following is how to port the Berry interpreter to the user's project:

1. Add all source files in the *src* directory to the user project, and the directory should be added to the include path
2. Users need to implement by themselves *default* files other than *berry.c* in the directory. If conditions permit, they don't need to modify them
3. Use *map_build* tool to generate constant object code and then compile

3. Platform Support

Currently Berry interpreter has been tested on some platforms. Windows, Linux and MacOS operating systems running on X86 CPUs can run normally. Embedded platforms that have been tested include Cortex M3/M0/M4/M7. The Berry interpreter should be able to run well on the basis of the necessary C runtime library. At present, when only Berry language core is compiled, the interpreter code generated by the ARMCC compiler is only about 40KiB, and the interpreter can run on a device with only 8KiB of RAM.

Porting Guide

Configuration File

The configuration header file of the Berry interpreter is *berry_conf.h*. This file includes a batch of macros for configuration and defines some platform-related content.

berry_conf.h File

Configuration Macro Switch

The configuration macros introduced in this section are usually used for compiling switches of some source codes. For the convenience of description, we call this macro "macro switch". For the macro switch, "on" refers to setting the macro switch to a non-zero value, and "off" refers to setting the value of the macro switch to 0.

Some macro switches have multiple states, not just "on" or "off". These macro switches are generally used for configurations with multiple options. There are also some configuration macros that are not macro switches. No matter what the value of these macros is, there will be no code and therefore will not participate in compilation. These macros are generally used to configure the quantity value.

[section::BE_DEBUG]

This macro switch is used to turn on or off the debugging function of the interpreter itself. When the value of *BE_DEBUG* is 0, debugging is turned off, otherwise it will be turned on. The debugging function mentioned here refers to the debugging of the interpreter, not the debugging function of the Berry program. The default value of *BE_DEBUG* is 0. If you use *Makefile* that comes with the interpreter project to compile, this macro switch will be turned on automatically when you use the `make debug` command.

When this macro is opened, some assertions will be turned on, and an error message will be output when the interpreter encounters an error that the assertion can catch. You can open *BE_DEBUG* when debugging the interpreter, and close it when compiling the release.

This macro switch configures the floating point type used by the *breal* type. When the value of the macro is 0, the *double* type will be used to implement *breal*, otherwise the *float* type will be used to implement *breal*. This macro switch can be turned on in some environments with low performance or memory configuration. In the default implementation, this macro switch is turned off.

This macro configures the implementation of the *bint* type. When the value of the macro is 0, the *int* type will be used to implement *bint*, when the value is 1, the *long* type will be used to implement *bint*, and when the value is

2, 64 will be used] Bit signed integer type (`__int64` under Windows, `long long` on other platforms) implements `bint`. The default value of this macro is 2. If you want to reduce memory usage, you can set this macro to 0 or 1 to enable 32-bit integer type.

This macro is used to configure runtime debugging information of Berry code. It has 3 available values: set to 0 to turn off the output of the file name and line number of the runtime debugging information, and set to 1 to output the file name and line number in the runtime debugging information, set to 2 When using `uint16_t` (16-bit integer) type to store the row number information. Its default value is 1.

Setting this macro to 0 will not store the file name and line number information, so the memory consumption is minimal. When set to 2, it consumes less memory, but if the program is too long, `uint16_t` will overflow.

This macro switch configures the function of constructing objects at compile time. Turning on this macro means that constructing objects at compile time is enabled. This macro is turned on by default. When this macro is turned on, the native objects in the standard library will be generated at compile time, and when this macro is turned off, the objects in the standard library will be built at runtime.

`be_regfunc` and `be_regclass` functions will be affected by this macro. The built-in object table cannot be modified when using compile-time object construction. At this time, these two functions cannot register objects in the built-in scope, but register objects in the global scope.

The objects constructed during the compile time are stored together with the code and will not occupy RAM (or the readable and writable area in the memory) resources. The construction technology during the compile time can also reduce the startup time of the interpreter, so it is recommended to open this macro. Please refer to section [section::precompiled_build] for more details on compile-time construction techniques.

This macro defines the maximum Berry stack capacity, which refers to the number of Berry objects. When the Berry code uses more than this amount of stack, it will stop executing the program and return an error message. The default value of this macro is 2000, which can be modified according to the memory capacity of the system.

This value does not affect the memory usage of the Berry stack, because the capacity of the Berry stack is dynamically adjusted, so no matter how much it is set to, it cannot help reduce memory usage. Its main function is to terminate execution when the Berry program consumes too much stack. These programs are very likely to be incorrect, for example, recursive function calls without return conditions will continue to consume the stack.

This macro defines the minimum available space in the Berry stack, and its default value is 10. The native function may push a value into the Berry stack. At this time, the stack will not automatically grow, so make sure that there is enough space in the stack for the native function to use. It is not recommended to modify this value, but to use the `be_stack_require` function where more stack space is really needed.

In order to detect stack overflow errors when debugging the interpreter, you can open the `BE_DEBUG` macro (section [section::BE_DEBUG]).

When this macro is opened, the short string object will save the hash value of the string to improve the running speed, but the size of each string object will increase by 4 bytes. This macro is turned off by default, and the current tests have not found that opening this macro will bring significant improvement.

This macro switch is used to enable or disable the `string` module, which is turned on by default.

This macro switch is used to enable or disable the `json` module, which is turned on by default.

This macro switch is used to enable or disable the `math` module, which is turned on by default.

This macro switch is used to enable or disable the `time` module, which is turned on by default.

This macro switch is used to enable or disable the `os` module, which is turned on by default.

This macro determines the `abort` function used internally by the Berry interpreter. By default or when the macro is not defined, the `abort` function in the C standard library will be used. This macro is defined as `abort` by default. If the user needs to explicitly specify the `abort` function used by the interpreter, then replace the macro definition with the function required by the user. This function should be in the same form as the `abort` function declaration in the standard library.

This macro determines the `exit` function used internally by the Berry interpreter. By default or when the macro is not defined, the `exit` function in the C standard library will be used. This macro is defined as `exit` by default. If the user needs to explicitly specify the `exit` function used by the interpreter, then replace the macro definition with the function required by the user. This function should be in the same form as the `exit` function declaration in the standard library.

This macro determines the `malloc` function used internally by the Berry interpreter. By default or when the macro is not defined, the `malloc` function in the C standard library will be used. This macro is defined as `malloc` by default. If the user needs to explicitly specify the function `malloc` used by the interpreter, then replace the macro definition with the function required by the user. This function should be in the same form as the `malloc` function declaration in the standard library.

This macro determines the `free` function used internally by the Berry interpreter. By default or when the macro is not defined, the `free` function in the C standard library will be used. This macro is defined as `free` by default. If the user needs to explicitly specify the `free` function used by the interpreter, then replace the macro definition with the function required by the user. This function should be in the same form as the `free` function declaration in the standard library.

This macro determines the `realloc` function used internally by the Berry interpreter. By default or when the macro is not defined, the `realloc` function in the C standard library will be used. This macro is defined as `realloc` by default. If the user needs to explicitly specify the `realloc` function used by the interpreter, then replace the macro definition with the function required by the user. This function should be in the same form as the `realloc` function declaration in the standard library.

This macro is used to define the implementation of the assertion function. By default, the `assert` function in the C standard library is used to implement the assertion. If the target system is inconvenient to use the `assert()` function in the standard library to make an assertion, you can modify the definition of the `be_assert` macro. A correct assertion function should use the following declaration:

```
void assert(int condition);
```

Among them, `condition` is the assertion condition. If the condition is not met, an error message will be output and the program will be terminated. Of course, the “assert” function is usually implemented using a macro.

berry_port.c File

This file implements the low-level IO functions of the Berry interpreter, including standard input and output and file system support. The *berry_port.c* file in the *default* directory contains a set of portable IO support. File operations and standard input and output are implemented using APIs in the C standard library. Path and folder operations support both Windows and POSIX standard APIs. This file also implements a set of FatFs-based IO operation functions for users to use directly. If you need to use the Berry interpreter in other environments, then these functions must be implemented separately (may only need to be implemented partially).

This section will introduce the functions of the functions implemented in the *berry_port.c* file and guide users to implement their own version.

```
void be_writebuffer(const char *buffer, size_t length);
```

Output a piece of data to the standard output device, the parameter `buffer` is the first address of the output data block, and `length` is the length of the output data block. This function outputs to the `stdout` file by default. Inside the interpreter, this function is usually used as a character stream output, not a binary stream.

`be_writebuffer` Functions are very versatile and must be implemented.

```
char* be_readstring(char *buffer, size_t size);
```

Input a piece of data from the standard input device, and read at most one row of data each time this function is called. The parameter `buffer` is the data buffer passed in by the caller, and the capacity of the buffer is `size`. This function will stop reading and return when the buffer capacity is used up, otherwise it will return when a newline character or end of file character is read. If the function executes successfully, it will directly use the `buffer` parameter as the return value, otherwise it will return `NULL`.

This function will add the read line breaks to the read data, and each time the `be_readstring` function is called, it will continue to read from the current position. This function is only called in the implementation of the native function `input`, and the `be_readstring` function may not be implemented when it is not necessary.

```
void* be_fopen(const char *filename, const char *modes);
```

To open a file, `filename` is the name of the file to be opened, and `modes` is the opening method. The function will return a file handle or a pointer to the file operation structure. The usage of this function is similar to the `fopen` function in the C standard library. The file name is a C-style string (ending with a `\0` character), and the pattern should at least support the following conditions:

- `r`, `rt`: To open a text file in read-only mode, the file must exist.
- `r+`, `rt+`: Open a text file in read-write mode, and create a new file if the file does not exist.
- `rb`: Open a binary file in read-only mode, the file must exist.
- `rb+`: Open a binary file in read-write mode, and create a new file if the file does not exist.
- `w`, `wt`: Create and open a text file in write-only mode, and the existing file will be deleted.
- `w+`, `wt+`: Create and open a text file in read-write mode, and the existing file will be deleted.
- `wb`: Create and open a binary file in write-only mode, and the existing file will be deleted.
- `wb+`: Create and open a binary file in read-write mode, and the existing file will be deleted.

By default, the `fopen` function in the C standard library is used to implement `be_fopen`. If you use other methods to achieve, you should ensure that the above operating modes can be achieved. If no file operations are required, this function can be left blank. The file operations here include all scenarios such as using the `open` function in the script, loading the script from a file (using the `be_loadfile` function), etc.

```
int be_fclose(void *hfile);
```

Close a file, `hfile` is the closed file handle. The function of this function is similar to the function `fclose` in the C standard library.

```
size_t be_fwrite(void *hfile, const void *buffer, size_t length);
```

Write a piece of data to the specified file. Parameter `hfile` is the file handle to be written, `buffer` is the pointer of the data to be written, `length` is the number of data to be written (in bytes).

```
size_t be_fread(void *hfile, void *buffer, size_t length);
```

Read a piece of data from the specified file. The parameter `hfile` is the file handle to be read, `buffer` is the pointer to the read buffer, and `length` is the number of bytes to be read.

```
char* be_fgets(void *hfile, void *buffer, int size);
```

Read a line from the file, similar to the `fgets` function in the C standard library. Parameter `hfile` is the file handle to be read, `buffer` is the pointer of the read buffer, and `size` is the capacity of the read buffer. This function will return when `size - 1` bytes, newline characters and end of file characters are read, and the return value is `buffer`.

```
int be_fseek(void *hfile, long offset);
```

Set the position of the file read and write pointer. The parameter `hfile` is the file handle to be operated, and `offset` is the value to be set.

```
long int be_ftell(void *hfile);
```

Get the current read and write pointer of the file, the parameter `hfile` is the handle of the file to be operated, and the return value of this function is the read and write pointer of the file.

```
long int be_fflush(void *hfile);
```

Write the data in the file buffer to the file. Parameter `hfile` is the file to be operated.

```
size_t be_fsize(void *hfile);
```

Get the size of the file. Parameter `hfile` is the file to be operated.

Making a Native Function

Berry's C FFI (Foreign Function Interface) operates on a virtual stack to interact with the VM. If we need to make an add function to add two numbers and use it in Berry in this way:

```
result = add(1, 2)
```

We need to know how the C code gets the arguments from the Berry function call and how to return the value.

The function arguments are stored in a stack, and the first argument to the last argument of the function is stored from the bottom of the stack to the top of the stack. If you want to use C to fetch elements from the stack, use the following set of FFIs:

```
int be_toint(bvm *vm, int index);
breal be_toreal(bvm *vm, int index);
int be_tobool(bvm *vm, int index);
const char* be_tostring(bvm *vm, int index);
void* be_tocomptr(bvm *vm, int index);
```

If you want to test if a value in the stack is a specified type, use the following set of FFIs:

```
int be_isnil(bvm *vm, int index);
int be_isbool(bvm *vm, int index);
int be_isint(bvm *vm, int index);
int be_isreal(bvm *vm, int index);
int be_isnumber(bvm *vm, int index);
int be_isstring(bvm *vm, int index);
int be_isclosure(bvm *vm, int index);
int be_isntvclos(bvm *vm, int index);
int be_isfunction(bvm *vm, int index);
int be_isproto(bvm *vm, int index);
int be_isclass(bvm *vm, int index);
int be_isinstance(bvm *vm, int index);
int be_islist(bvm *vm, int index);
int be_ismap(bvm *vm, int index);
int be_iscomptr(bvm *vm, int index);
```

If you need to push values onto the stack, use these FFIs:

```
void be_pushnil(bvm *vm);
void be_pushbool(bvm *vm, int b);
void be_pushint(bvm *vm, bint i);
void be_pushreal(bvm *vm, breal r);
void be_pushstring(bvm *vm, const char *str);
void be_pushnstring(bvm *vm, const char *str, size_t n);
const char* be_pushfstring(bvm *vm, const char *format, ...);
void be_pushvalue(bvm *vm, int index);
void be_pushntvclosure(bvm *vm, bntvfunc f, int nupvals);
void be_pushntvfunction(bvm *vm, bntvfunc f);
void be_pushclass(bvm *vm, const char *name, const bntfuncinfo *lib);
void be_pushcomptr(bvm *vm, void *ptr);
```

index is the position of the element on the stack, the positive value is the offset from the bottom of the stack to the top of the stack, and the negative value is the offset from the top of the stack to the bottom of the stack.

The return value uses two FFIs:

```
be_return(vm)
be_return_nil(vm)
```

These FFIs are actually macros. `be_return` returns the object at the top of the stack, and `be_return_nil` returns `nil`.

These FFIs are defined in `berry.h`.

Now let's implement the add function:

```
int my_add_func(bvm *vm)
{
    /* check the arguments are all integers */
    if (be_isint(vm, 1) && be_isint(vm, 2)) {
        bint a = be_toint(vm, 1); /* get the first argument */
        bint b = be_toint(vm, 2); /* get the second argument */
        be_pushint(vm, a + b); /* push the result to the stack */
    } else if (be_isnumber(vm, 1) && be_isnumber(vm, 2)) { /* check the arguments are all nu
```

```
        breal a = be_toreal(vm, 1); /* get the first argument */
        breal b = be_toreal(vm, 1); /* get the second argument */
        be_pushreal(vm, a + b); /* push the result to the stack */
    } else { /* unacceptable parameters */
        be_pushnil(vm); /* push the nil to the stack */
    }
    be_return(vm); /* return calculation result */
}
```

Then register it in the appropriate place:

```
be_regcfunc(vm, "add", my_add_func);
```

Instantiate a list object in a native function

Generating instantiated native classes in C can be cumbersome compared to simple types. This section will guide the reader to instantiate the list class.

The list class is a wrapper around the list structure, which has a .data property for the list structure. Therefore, we first need to construct a list structure:

```
be_newlist(vm);
```

The be_newlist function constructs a value of type BE_LIST. Then we can operate on the data:

```
be_pushint(vm, 100);
be_data_append(vm, -2);
be_pop(vm, 1); /* popping the integer 100 */
```

The first two lines of code are used to append the integer 100 to the list, and the third line to the integer 100 is popped to facilitate subsequent operations.

Since the BE_LIST type cannot be used directly in Berry, but is used by the list class, we have to build the list class for it:

```
be_getglobal(vm, "list");
be_pushvalue(vm, -2); /* push the list data to top */
be_call(vm, 1); /* call constructor */
```

The constructor of the list class allows the use of the BE_LIST type parameter, which takes the argument as list data.

The complete code is as follows:

```
int m_listtest(bvm *vm)
{
    be_getglobal(vm, "list");
    be_newlist(vm);
    be_pushint(vm, 100);
    be_data_append(vm, -2);
    be_pop(vm, 1);
    be_call(vm, 1);
    be_pop(vm, 1); /* pop the arguments */
    be_return(vm);
}
```

Register the native function in the appropriate place:

```
be_regcfunc(vm, "listtest", m_listtest);
```

Berry Introduction (in 20 minutes or less)

This quick start will drive you in the basics of the Berry language. It should take no more than 20 minutes and is inspired by [Ruby in Twenty Minutes](#)

Berry is an ultra-lightweight dynamically typed scripting language. It is designed for lower-performance embedded devices. It also runs on a regular computer, and it can run directly in your browser for quick testing.

Berry is the next generation scripting for Tasmota, embedded by default in all ESP32 based firmwares. It is used for advanced scripting and superseded Rules. Its advanced features are used to extend Tasmota: adding commands, adding drivers (I2C, serial...), extending the web UI, adding full applications (TAPP files), driving advanced graphics with [LVGL](#).

To start with Berry, you have at least 3 choices:

- use the [Berry online console](#) and start in less than 10 seconds
- [flash an ESP32](#) based device with Tasmota and use the Berry console
- compile Berry on your computer from sources and run the Berry interpreter (less preferred)

Hello, Berry

In the console type:

```
> print("Hello, Berry")
Hello, Berry
```

What just happened? We just sent the simplest possible Berry program `print("Hello, Berry")`. Internally this program was compiled into Berry bytecode and ran using the Berry virtual machine.

In Berry you can append commands one after the other. Contrary to C you don't need any separator like `;`. Unlike Python indentation has no importance. Commands need only to be separated by at least one space-like character: space, tab, newline.

```
> print("Hello, Berry") print("Hello, Berry")
Hello, Berry
Hello, Berry
```

In this second example, the implicit program contains 2 commands.

Your free calculator is here

Not surprisingly, like most scripting languages you can do direct calculation.

```
> 3+2
5
> 3*2
6
```

The above computations are made against integers. Berry supports either 32 bits or 64 bits integers depending on the underlying platform (usually 32 bits on embedded systems).

Berry supports floating point calculation, as soon as at least one member is floating point. Floating point uses either 32 bits `float` or 64 bits `double` depending on compilation options (usually 32 bits on embedded systems).

```
> 3/2
1
> 3.0/2
1.5
> 1/3.0
0.333333
```

The command `3/2` works on integers and returns an integer result. `3.0/2`, `3/2.0` or `3.0/2.0` work on floating point numbers since at least one operand is floating point.

You can convert an integer to floating point using `real()` and truncate to integer with `int()`.

```
> 3/2
1
> real(3)/2
1.5
```

Welcome to berry's documentation!

```
> int(3.0/2)
1
```

Beyond the core Berry language, advanced math function are available via the additional module `math` [see documentation](#).

```
> import math
> math.sqrt(2)      # square root of 2
1.41421

> math.pow(2,3)     # 2^3
8
```

Defining a function

What if you want to say “Hello” a lot without getting your fingers all tired? You should define another function:

```
> def hi() print("Hello, Berry") end
>
```

Now let's call the function:

```
> hi()
Hello, Berry
```

`hi` is a function that takes no argument, returns nothing, and prints a message in the console. Calling a function always requires sending arguments between parenthesis (). Otherwise Berry thinks that you want to manipulate the function itself as an entity.

```
> hi      # return the function entity itself
<function: 0x3ffdac6c>

> hi()    # call the function
Hello, Berry
```

What if we want to say hello to one person, and not only to Berry? Just redefine `hi` function to take a name as an argument.

```
> def hi(name) print("Hello, " + name) end
```

This way, `hi` is a function that takes a single argument as string.

```
> hi("Skiars") hi("Theo")
Hello, Skiars
Hello, Theo
```

This function only works if the argument is a string, and fails if you use any other type of argument. Let's use `str()` built-in function to force-convert the argument to a string.

```
> def hi(name) print("Hello, " + str(name)) end

> hi("Skiars") hi("Theo")
Hello, Skiars
Hello, Theo

> hi(2)
Hello, 2
```

What happens if you don't send any argument to a function that expects one? Let's try:

```
> def hi(name) print("Hello, " + str(name)) end

> hi()
Hello, nil
```

The knights who say nil

What is this nil thing? Berry has a special value nil meaning “nothing”. nil is the implicit value passed to a function when no argument is sent, or the value returned by a function that does not return anything.

```
> nil
nil

> hi(nil)
Hello, nil

> hi()
Hello, nil
```

As you see, nil is the implicit value passed when arguments are missing, but also a value that you can pass explicitly.

Formatting strings

In the above example, we only concatenated two strings. Berry provides a more advanced scheme to format numerical values as well. It is widely inspired from C formatting used by printf. Don't forget to import the string module first.

```
> import string
> def say_hi(name) print(string.format("Hello, %s!", name)) end
> def say_bye(name) print(string.format("Bye, %s, come back soon", name)) end

> say_hi("Bob")
Hello, Bob!
> say_bye("Bob")
Bye, Bob, come back soon
```

You can combine with string functions like toupper() to convert to uppercase

```
> import string
> name = "Bob"
> string.format("Hello, %s!", string.toupper(name))
Hello, BOB!
```

In the example above, we have created a global variable called name containing the string "Bob" and used string.toupper() to convert it to all uppercase.

Evolving into a Greeter

What if we want a real greeter around, one that remembers your name and welcomes you and treats you always with respect. You might want to use an object for that. Let's create a “Greeter” class.

Note: since it's a multi-line example, you may need to copy the entire block and paste it at once in the console (not line-by-line).

```
class Greeter
  var name

  def init(name)
    self.name = name
  end

  def say_hi()
    import string
    print(string.format("Hi %s", self.name))
  end

  def say_bye()
    import string
```



```
    print(string.format("Bye %s, come back soon.", self.name))
end
end
```

The new keyword here is `class`. This defines a new class called `Greeter` and a bunch of methods for that class. Also notice `var name`. This is an instance variable, and is available to all the methods of the class. As you can see it's used by `say_hi` and `say_bye` as `self.name`.

The `init()` method is a special method called a "constructor". It is implicitly called when you create a new instance for the class, and the arguments are passed to `init()`. The constructor is responsible for complete initialization of the object, and it's always the first method called. The above example is typical of any object: it takes an argument name and copies it to an instance variable `self.name` to make it available to any method.

Note: Berry has no concept of private members (contrary to C++). All instance variables and methods are always public.

Creating a greeter object

Now let's create a greeter object and use it:

```
> greeter = Greeter("Pat")
> greeter.say_hi()
Hi Pat
> greeter.say_bye()
Bye Pat, come back soon.
```

Once the greeter object is created, it remembers that the name is Pat. If you want to get the name from a greeter, you can ask a greeter by accessing the name variable on it (without parenthesis):

```
> greeter.name
Pat
```

Subclasses

Methods and instance variables are defined at the class creation. You can't add method or instance variables to an already existing class. To extend a class you can create a sub-class:

```
class SurGreeter : Greeter      # subclass of Greeter
    var surname

    def init(name, surname)      # sub-class takes 2 arguments
        super(self).init(name)  # call constructor of super-class
        self.surname = surname
    end

    def say_hi()
        import string
        print(string.format("Hi %s %s", self.name, self.surname))
    end
end
```

The class `SurGreeter` extends `Greeter` with an additional surname field. It overrides `say_hi()` but leaves `say_bye()` unchanged.

There is a special syntax for calling a method of the subclass `super(self).init(name)`.

Note: classes have always an `init()` method, either because it was explicitly defined, or implicitly. It is always ok to call `super(self).init()` even if the subclass has no explicit `init()` method.

Now let's try this new class:

```
> greet = SurGreeter("John", "Smith")
> greet.say_hi()
Hi John Smith
> greet.say_bye()
```

Welcome to berry's documentation!

Bye John, come back soon.

```
> greet.name
John
> greet.surname
Smith
```

Greetings everyone!

This greeter isn't all that interesting though, it can only deal with one person at a time. What if we had some kind of MegaGreeter that could either greet the world, one person, or a whole list of people? Let's try to build that. We will start with a class definition:

```
class MegaGreeter
  var names

  def init(name)
    self.names = []          # empty list
    if name != nil
      self.names.push(name)
    end
  end
end
```

So MegaGreeter objects have a list of names. The names field is initialized to the empty list []. The body of the MegaGreeter constructor adds the given name argument to the end of the list of names if it's not nil. Mega greeters don't have a single name and no name field, so here the name is just an ordinary parameter that we can use in the body of the constructor.

Let's try it:

```
> greeter = MegaGreeter()
> greeter.names
[]

> greeter = MegaGreeter("World")
> greeter.names
['World']
```

We can now go ahead and add greeter methods that add more names and show all the names:

```
class MegaGreeter
  var names

  def init(name)
    self.names = []          # empty list
    if name != nil
      self.names.push(name)
    end
  end

  def add(name)
    self.names.push(name)
  end

  def say_hi()
    import string
    for n: self.names
      print(string.format("Hello %s!", n))
    end
  end
end
```

Welcome to berry's documentation!

```
def say_bye()  
  import string  
  for n: self.names  
    print(string.format("Bye %s, come back soon.", n))  
  end  
end  
end
```

We introduced here a new construct known as an iterator. `for n: self.names` creates a new local variable `n` and iterate the following code for each value in `self.names`.

Let's try the full example now:

```
> greeter = MegaGreeter()  
> greeter.add("Skiars")  
> greeter.add("Theo")  
> greeter.add("Stephan")  
  
> greeter.say_hi()  
Hello Skiars!  
Hello Theo!  
Hello Stephan!  
  
> greeter.say_bye()  
Bye Skiars, come back soon.  
Bye Theo, come back soon.  
Bye Stephan, come back soon.
```

Comments

Sometimes, it is nice just to add comments that explain interesting things related to your code. In the example in the last section, there were a few single line comments:

```
self.names = []           # empty list
```

Such comments start with `#` and tell the system to ignore the rest of the line.

You can also use multi-line comments starting with `#-` and ending with `-#`.

```
#-  
  This is a comment  
-#  
  
#-  
# This is also a comment block (`#` are ignored)  
-#  
  
#-----  
  Alternative way to make comment blocks  
-----#
```

Indentation has no impact on Berry compiler, it's only by convention to make source code more readable.

Maps

Maps are a very common and powerful feature to store key/value pairs. They are declared using `{}`.

```
> m1 = {}           # empty map  
> m  
{}  
  
> m2 = {"k1": "v1", "k2": "v2", "k3": "v3"}  
> m2  
{'k2': 'v2', 'k1': 'v1', 'k3': 'v3'}
```

Actually keys and values can be of arbitrary type.

```
> m3 = { 1.5: 3, 2:"two", true:1, false:nil }
> m3
{1.5: 3, true: 1, 2: 'two', false: nil}
```

The main restriction is that a key can't be nil. Setting adding a key of value nil is silently ignored.

```
> m4 = { nil:"foo" }
> m4
{}
```

Accessing a value in the map uses [<key>]:

```
> m1 = {}
> m1['k1'] = "value1"
> m1
{'k1': 'value1'}

# working with numerical values
> m1['k2'] = 0
> m1['k2'] += 5      # shortcut for `m1['k2'] = m1['k2'] + 5`
> m1
{'k': 'value', 'k2': 5}
```

Accessing a non-existent key raises an error. There is an alternative function find() to access a key and return a default value if the key is absent. contains() can also be used to check the presence of the key.

```
> m1 = {"foo":"bar"}
> m1.contains("foo")
true
> m1.contains("bar")      # only checks for keys, not values
false

> m1["foo"]
bar
> m1["bar"]
key_error: bar
stack traceback:
  <native>: in native function
  stdin:1: in function main

# alternative with find
> m1.find("foo", "not_found")
bar
> m1.find("bar", "not_found")
not_found
> m1.find("bar")          # returns nil by default if not found
```

Note: `m[k] = v` is syntactic sugar for `m.setitem(k, v)`. When reading a value, `m[k]` is equivalent to `m.item(k)`.

If statements and basic expressions

We can program a ridiculously inefficient Fibonacci sequence generator using if and recursion:

```
def fib(n)
  if n <= 1 return n end
  return fib(n-1) + fib(n-2)
end
```

This defines a top-level function called fib that is not a member of any class. The fib function is recursive, calling itself, and also makes use of a few new features. The if-statement is well known from other languages. In Berry it works by taking an expression and conditionally evaluating a block.

Berry also has the usual array of infix operators, +, -, *, /, % etc. and the relational operators <, <=, >, >=, == and !=.

```
> fib(10)
55
```

Cycling and Looping

As we've seen in MegaGreeter it is very simple to iterate over a list for `n: self.names [...] end`.

Iterators can also be used over ranges like for `i:0..4` which will iterate over all values between 0 and 4 inclusive (5 iterations in total).

```
> for i:0..4 print(i) end
0
1
2
3
4
```

Iterating over maps goes in two flavors: iterating over values, or over keys.

```
> m = {"k1":"v1", "k2":"v2", "k3":"v3"}
> print(m)      # keep in mind that there is no order in a map
{'k2': 'v2', 'k1': 'v1', 'k3': 'v3'}
```

iterate over values

```
> for v: m      print(v) end
v2
v1
v3
```

iterate over keys

```
> for k: m.keys() print(k) end
k2
k1
k3
```

iterate over both keys and values

```
> for k: m.keys() print(k, m[k]) end
k2 v2
k1 v1
k3 v3
```

For C programmers, the equivalent of `for (int i=0; i<a; i++) { [...] } is` `for i: 0..a-1 [...] end`

Functions and arguments

In Berry, functions are first class entities (Berry supports functional programming as well as object oriented). Berry is not a strongly types language, which means that you don't define any type as input or output when you define a function. This may seem as a problem, but it's a very powerful feature instead.

Berry relies on what is known as "Duck Typing", as in "If it walks like a duck and it quacks like a duck, then it must be a duck". As long as the type you provide supports the right methods and calls, then it's fine.

A function only defines the number of arguments it receives:

```
> def f(a, b) return str(a) + str(b) end      # takes only 2 arguments
```

`f` expects 2 arguments, if you provide less than 2, the non-defined are set to `nil`. If you provide more than 2, the extra-arguments are silently ignored.

```
> def f(a, b) return str(a) + str(b) end      # takes only 2 arguments
> f("foo", "bar")
```

Welcome to berry's documentation!

```
foobar
> f("foo")
foonil
> f("foo", "bar", "baz")
foobar
```

A function may or may not return a value with `return <expression>`. If you call just `return` or the function ends without any `return` statement, the function returns `nil`.

Closures

Let's finish this introduction with a very powerful feature known as closures. It is sometimes seen as intimidating or complex, but it's actually very simple. We will visit only the most common use of closures, if you want to get more details see the [Berry documentation](#).

Let's go back to our simple Byer example (class that says Bye).

```
class Byer
  var name
  def init(name)
    self.name = name
  end
  def say_bye()
    import string
    print(string.format("Bye %s, see you soon.", self.name))
  end
end
```

Let's define an instance of this class:

```
> bye_bob = Byer("Bob")
> bye_pat = Byer("Pat")
> bye_bob.say_bye()
Bye Bob, see you soon.
```

Nothing new until now. Closure are useful as soon as you need callbacks. Let's say that you are using a framework that accepts a callback (a function you provide that will be fired in the future). We want to pass a function that says Bye to Bob.

The naive approach would be to use `bye_bob.say_bye` method, which is a valid function. However this function has no context and can't know which instance you are referring to.

```
> bye_bob.say_bye
<function: 0x3ffb3200>
> bye_pat.say_bye
<function: 0x3ffb3200>    # same function as above
```

As shown above, since the context is missing, you can't distinguish from the method `bye_bob.say_bye` and `bye_pat.say_bye`. They are the same function.

Closure allows to create a new synthetic function that encapsulates transparently the context.

```
> cb = def () bye_bob.say_bye() end
> cb
<function: 0x3ffd9df4>

# let's check that a closure on bye_pat is different
> cb_pat = def () bye_pat.say_bye() end
> cb_pat
<function: 0x3ffd9daa0>
```

`cb` is a closure, it creates a function that captures the instance `bye_bob` and then calls `say_bye()` on it. Let's call the closures to check they are working.

Welcome to berry's documentation!

```
> cb()  
Bye Bob, see you soon.
```

Tasmota this is widely used in Tasmota for example for deferred functions. For example if you want to run `bye_bob.say_bye()` in 5 seconds in the future:

```
> tasmota.set_timer(5000, cb)      # cb() is called in 5000 milliseconds
```

Advanced users: there is a compact syntax for simple callbacks: `def cb(a,b) return <expr> end` becomes `a,b -> <expr>`

Consider Yourself Introduced

So that's a quick tour of Berry. Please have a look at the online [Berry documentation](#).

For Tasmota users, also have a look at the [Tasmota Berry documentation](#) and [Tasmota Berry Cookbook](#).

Extra

Here is a short comparison of Berry and Python syntax, courtesy of @Beormund

Berry vs Python	Berry	MicroPython
Current object	<code>self</code>	<code>self</code>
Single line comments	<code>#</code>	<code>#</code>
Multi line comments	<code>#- ... -#</code>	
Logical 'and', 'or' and 'not' operators	<code>&& !</code>	<code>and or not</code>
Shift left, right	<code><< >></code>	<code><< >></code>
Integer division	<code>/</code>	<code>/</code>
Statement blocks/grouping	<code>(scope)</code>	<code>(indent)</code>
Class definition & inheritance	<code>class a:b</code>	<code>class a(b):</code>
Class constructor	<code>def init(x) ... end</code>	<code>def __init__(self, x):</code>
Class and superclass constructors	<code>def init(x) super(self).init(x) end</code>	<code>def __init__(self, x): super(b, self).__init__(x)</code>
Class constructor that assigns to fields	<code>def init(x) self.x = x end</code>	<code>def __init__(self, x): self.x = x</code>
Check object's type	<code>isinstance(b, a)</code>	<code>isinstance(b, a)</code>
Call method foo with 2 arguments	<code>foo(x, y)</code>	<code>foo(x, y)</code>
Declare a member variable in a class	<code>self.x = nil</code>	<code>self.x = None</code>
Declare a local variable in a method	<code>var x = 2, y = nil</code>	<code>x = 2 y = None</code>
Define a constant in a class	<code>static x = 2</code>	
Define a top level function	<code>def foo(x,y) end</code>	<code>def foo(x,y):</code>
Define an instance method in a class	<code>def foo(x,y) end</code>	<code>def foo(self, x, y):</code>
Define a static method in a class	<code>static def foo(x,y) end</code>	
If statement	<code>if condition end</code>	<code>if condition:</code>
Fixed loop	<code>for i: range end</code>	<code>for i in range(end):</code>

Iterate over collection	<code>for k:coll.keys() end</code>	<code>for x in coll:</code>
While loop	<code>while condition end</code>	<code>while condition:</code>
Import from library	<code>import library</code>	<code>import library</code>
Print	<code>print('hello world')</code>	<code>print('hello world')</code>
Interpolation	<code>print(format("Hello %s", name))</code>	<code>print("Hello %s" %(name))`</code>
f-strings	<code>print(f"Hello {name}")</code>	<code>print(f"Hello {name}")</code>
Simple types	<code>int</code> <code>real</code> <code>bool (true\ false)</code> <code>string</code> <code>nil</code>	<code>int</code> <code>float</code> <code>bool (True\ False)</code> <code>string</code> <code>None</code>
Class types	<code>list</code> <code>map</code> <code>range</code>	<code>list</code> <code>dict</code> <code>tuple</code> <code>set</code>

Language fast reference PDF

Download the [Language Fast Reference PDF](../../_static/berry_short_manual.pdf) (8 pages).

Memory Requirements

RAM required

- extreme: 4 KB
- low: 8 KB
- recommended: ≥ 16 KB

ROM/Flash required

- extreme: 64 KB
- low: 128 KB
- recommended: ≥ 256 KB

Explanation

- **extreme**: The memory capacity required in the minimum configuration. Lower than this value means that it is almost impossible to run Berry.
- **low**: The minimum memory required for the complete Berry interpreter includes some user and third-party code.
- **recommended**: In addition to the complete Berry interpreter, there are many users and third-party code.

API documentation

Main berry entry API.

This file is part of the Berry default interpreter. **Author**skiars@qq.com, <https://github.com/Skiars/berry> **Copyright**(c) 2018-2022 Guan Wenliang MIT License (<https://github.com/Skiars/berry/blob/master/LICENSE>)

`native-module` member type specifier.

Welcome to berry's documentation!

BE_CNIL
native-module member type specifier.
BE_CNIL

BE_CINT
BE_CINT

BE_CREAL
BE_CREAL

BE_CB00L
BE_CBOOL

BE_CFUNCTION
BE_CFUNCTION

BE_CSTRING
BE_CSTRING

BE_CMODULE
BE_CMODULE

only linux.

BERRY_LOCAL
only linux
BERRY_LOCAL

BE_EXPORT_VARIABLE
BE_EXPORT_VARIABLE

debug hook typedefs.

BE_HOOK_LINE
debug hook typedefs.
BE_HOOK_LINE

BE_HOOK_CALL
BE_HOOK_CALL

BE_HOOK_RET
BE_HOOK_RET

BE_HOOK_EXCEPT
BE_HOOK_EXCEPT

Defines

BERRY_VERSION
do not modify the version number!

BE_INTEGER
BE_INTEGER

BE_INT_FMTLEN
BE_INT_FMTLEN

BE_INT_FORMAT
BE_INT_FORMAT

bbool
bbool

bfalse
bfalse

btrue
btrue

BERRY_API
API function mark.

be_native_module_nil(_name)

Welcome to berry's documentation!

Welcome to berry's documentation!

```
define bproto
be_nested_proto (_nstack, _argc, _varg, _has_upval, _upvals, _has_subproto, _protos,
    _has_const, _ktab, _fname, _source, _code)
    new version for more compact literals
be_define_local_closure (_name)
    be_define_local_closure
be_local_closure (_name, _proto)
    new version for more compact literals
be_assert (expr)
    the default assert definition
be_writestring (s)
    be_writestring
```

Note

FFI function

```
be_writenewline ()
    be_writenewline
```

Note

FFI function

```
be_return (vm)
    be_return
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance

```
be_return_nil (vm)
    be_return_nil
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance

```
be_loadfile (vm, name)
    be_loadfile
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **name** – (???)

be_loadmodule (vm, name)
be_loadmodule

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **name** –

be_loadstring (vm, str)
be_loadstring

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **str** – (???)

be_dostring (vm, s)
be_dostring

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **s** – (???)

Typedefs

typedef uint8_t bbyte
bbyte

typedef BE_INTEGER bint
bint

typedef double breal
brear

typedef struct bvm bvm
virtual machine structure

typedef int (*bntvfunc)(bvm*)
native function pointer

typedef struct bntvmodobj bntvmodobj_t
bntvmodobj_t

typedef struct bntvmodule bntvmodule_t
bntvmodule_t

typedef const struct bclass *bclass_ptr
we need only the pointer to bclass here

Welcome to berry's documentation!

```
typedef bclass_ptr bclass_array[]  
    array of bclass* pointers, NULL terminated  
  
typedef struct bhookinfo bhookinfo_  
    bhookinfo_  
  
typedef void (*bntvhook)(bvm *vm, bhookinfo_t *info)  
    void (*bntvhook)(bvm *vm, bhookinfo *info)
```

Param vm: virtual machine instance

Param info:

```
typedef void (*bobshook)(bvm *vm, int event, ...)  
    Observability hook.
```

Param vm: virtual machine instance

Param event:

```
typedef int (*bctypefunc)(bvm*, const void*)  
    bctypefunc
```

Enums

```
enum berrorcode  
    error code definition  
Values:
```

```
enumerator BE_OK  
    BE_OK
```

```
enumerator BE_EXIT  
    BE_EXIT
```

```
enumerator BE_MALLOC_FAIL  
    BE_MALLOC_FAIL
```

```
enumerator BE_EXCEPTION  
    BE_EXCEPTION
```

```
enumerator BE_SYNTAX_ERROR  
    BE_SYNTAX_ERROR
```

```
enumerator BE_EXEC_ERROR  
    BE_EXEC_ERROR
```

```
enumerator BE_IO_ERROR  
    BE_IO_ERROR
```

```
enum beobsshookevents  
    beobsshookevents  
Values:
```

```
enumerator BE_OBS_PCALL_ERROR  
    called when be_callp() returned an error, most likely an exception
```

```
enumerator BE_OBS_GC_START  
    start of GC, arg = allocated size
```

```
enumerator BE_OBS_GC_END  
    end of GC, arg = allocated size
```

```
enumerator BE_OBS_VM_HEARTBEAT  
    VM heartbeat called every million instructions
```

```
enumerator BE_OBS_STACK_RESIZE_START  
    Berry stack resized
```

```
enumerator BE_OBS_MALLOC_FAIL  
    Memory allocation failed
```

Functions

```
BERRY_API bint be_str2int(const char *str, const char **endstr)
(???)
```

Note

FFI function

Parameters:

- **str** – (???)
- **endstr** – (???)

Returns: (???)

```
BERRY_API breal be_str2real(const char *str, const char **endstr)
(???)
```

Note

FFI function

Parameters:

- **str** –
- **endstr** – (???)

Returns: (???)

```
BERRY_API const char *be_str2num(bvm *vm, const char *str)
(???)
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **str** – (???)

Returns: (???)

```
BERRY_API int be_top(bvm *vm)
```

returns the absolute index value of the top element in the virtual stack

This function returns the absolute index value of the top element in the virtual stack. This value is also the number of elements in the virtual stack (the capacity of the virtual stack). Call this function before adding or subtracting elements in the virtual stack to get the number of parameters of the native function.

Note

FFI function

Parameters: **vm** – virtual machine instance virtual machine instance

Returns: (???)

```
BERRY_API const char *be_typeof(bvm *vm, int index)
```

converts the type of the Berry object into a string and returns it

This function converts the type of the Berry object into a string and returns it. For example, it returns “int” for an integer object, and “function” for a function object.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – index of the object to be operated

Returns: string corresponding to the parameter type (see: baselib_type)

BERRY_API **const char** *be_classname (bvm *vm, **int** index)

converts the type of the Berry object into a string and returns it.

This function converts the type of the Berry object into a string and returns it. For example, it returns “int” for an integer object, and “function” for a function object

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – index of the object to be operated

Returns: string corresponding to the parameter type (see: baselib_type)

BERRY_API bbool be_classof (bvm *vm, **int** index)

(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – (???)

Returns: (???)

BERRY_API **int** be_strlen (bvm *vm, **int** index)

length of the specified Berry string

This function returns the number of bytes in the string at index (the '\0' characters at the end of the Berry string are not counted). If the value of the index position is not a string, the be_strlen function will return 0. Although the Berry string is compatible with the C string format, it is not recommended to use the strlen function of the C standard library to measure the length of the Berry string. For Berry strings, be_strlen is faster than strlen and has better compatibility.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – index of the object to be operated

Returns: length

BERRY_API **void** be_strconcat (bvm *vm, **int** index)

splice two Berry strings

This function will concatenate the string at the parameter position of index with the string at the top position of the stack, and then put the resulting string into the position indexed by index.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – (???)

BERRY_API **void** be_pop (bvm *vm, **int** n)

pops the value at the top of the stack

Note that the value of n cannot exceed the capacity of the stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **n** – number of values to be popped

BERRY_API **void** be_remove (bvm *vm, **int** index)

remove a value from the stack

After the value at index is moved out, the following values will be filled forward, and the stack capacity will be reduced by one. The value of index cannot exceed the capacity of the stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – the object to be removed

BERRY_API **int** be_absindex (bvm *vm, **int** index)

absolute index value of a given index value

If index is positive, the return value is the value of index. If index is negative, the return value of textttbe_absindex is the absolute index value corresponding to index. When index is a negative value (relative index), its index position cannot be lower than the bottom of the stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – index value

Returns: absolute index

BERRY_API **bbool** be_isnil (bvm *vm, **int** index)

value in virtual stack is nil

This function returns whether the value indexed by the parameter index in the virtual stack is nil, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isbool(bvm *vm, **int** index)

value in virtual stack is bool

This function returns whether the value indexed by the parameter index in the virtual stack is bool, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isint(bvm *vm, **int** index)

value in virtual stack is int

This function returns whether the value indexed by the parameter index in the virtual stack is int, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isreal(bvm *vm, **int** index)

value in virtual stack is real

This function returns whether the value indexed by the parameter index in the virtual stack is real, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isnumber(bvm *vm, **int** index)

value in virtual stack is number

This function returns whether the value indexed by the parameter index in the virtual stack is number, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isstring (bvm *vm, **int** index)

value in virtual stack is string

This function returns whether the value indexed by the parameter index in the virtual stack is string, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isclosure (bvm *vm, **int** index)

value in virtual stack is closure

This function returns whether the value indexed by the parameter index in the virtual stack is closure, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isntvclos (bvm *vm, **int** index)

value in virtual stack is primitive closure type

This function returns whether the value indexed by the parameter index in the virtual stack is primitive closure type, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isfunction (bvm *vm, **int** index)

value in virtual stack is function

This function returns whether the value indexed by the parameter index in the virtual stack is function, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isproto (bvm *vm, **int** index)

value in virtual stack is proto

This function returns whether the value indexed by the parameter index in the virtual stack is proto, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isclass (bvm *vm, **int** index)

value in virtual stack is class

This function returns whether the value indexed by the parameter index in the virtual stack is class, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isinstance (bvm *vm, **int** index)

value in virtual stack is instance

This function returns whether the value indexed by the parameter index in the virtual stack is instance, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_ismapinstance (bvm *vm, **int** index)

value in virtual stack is instance

This function returns whether the value indexed by the parameter index in the virtual stack is an instance of class map (or derived). If it is, it returns 1, otherwise it returns 0

This function returns whether the value indexed by the parameter index in the virtual stack is an instance of class list (or derived). If it is, it returns 1, otherwise it returns 0

Note

FFI function

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index
- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

Returns: true/false

BERRY_API bbool be_islistinstance (bvm *vm, **int** index)

BERRY_API bbool be_ismodule (bvm *vm, **int** index)

value in virtual stack is module

This function returns whether the value indexed by the parameter index in the virtual stack is module, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_islist (bvm *vm, **int** index)

value in virtual stack is list

This function returns whether the value indexed by the parameter index in the virtual stack is list, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_ismap (bvm *vm, **int** index)

value in virtual stack is map

This function returns whether the value indexed by the parameter index in the virtual stack is map, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_iscomptr (bvm *vm, **int** index)

value in virtual stack is universal pointer type

This function returns whether the value indexed by the parameter index in the virtual stack is universal pointer type, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_iscomobj (bvm *vm, **int** index)

(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isderived (bvm *vm, **int** index)

(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bbool be_isbytes (bvm *vm, **int** index)

value in virtual stack is instance or sub-instance of class bytes

This function returns whether the value indexed by the parameter index in the virtual stack is instance or sub-instance of class bytes, if it is, it returns 1, otherwise it returns 0

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: true/false

BERRY_API bint be_toint (bvm *vm, **int** index)

virtual stack to integer type

Get the value of the index position of index from the virtual stack and return it as an integer type. This function does not check the correctness of the type. If the value is an instance, the method toint() is called if it exists.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** – value index

Returns: (???)

BERRY_API breal be_toreal (bvm *vm, **int** index)

virtual stack to floating-point number type

Get the value of the index position of index from the virtual stack and return it as an floating-point number type. This function does not check the correctness of the type.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** –

Returns: (???)

BERRY_API **int** be_toindex (bvm *vm, **int** index)

virtual stack to integer type

Get the value of the index position of index from the virtual stack and return it as an integer type. This function does not check the correctness of the type. Unlike be_toint, the return value type of be_toindex is int, while the return value of the former is bint.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** –

Returns: (???)

BERRY_API bbool be_tobool (bvm *vm, **int** index)

virtual stack to Boolean type

Get the value of the index position of index from the virtual stack and return it as a Boolean type. If the indexed value is not of Boolean type, it will be converted according to the rules in section type_bool, and the conversion

process will not cause the indexed value to change. If the value is an instance, the method tobool() is called if it exists.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** –

Returns: (???)

BERRY_API **const char** *be_tostring (bvm *vm, **int** index)

virtual stack to string

Get the value of the index position of index from the virtual stack and return it as a string type. If the indexed value is not a string type, the indexed value will be converted to a string, and the conversion process will replace the value at the indexed position in the virtual stack with the converted string. The string returned by this function always ends with '\0' characters. If the value is an instance, the method tostring() is called if it exists.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** –

Returns: (???)

BERRY_API **const char** *be_toescape (bvm *vm, **int** index, **int** mode)
(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** –
- **mode** –

Returns: (???)

BERRY_API **void** *be_tocomptr (bvm *vm, **int** index)

virtual stack to general pointer

Get the value of the index position of index from the virtual stack and return it as a general pointer type. This function does not check the correctness of the type.

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **index** –

BERRY_API **void** be_moveto (bvm *vm, **int** from, **int** to)

(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance virtual machine instance
- **from** –
- **to** –

BERRY_API **void** be_pushnil (bvm *vm)

Push a nil value onto the virtual stack.

Note

FFI function

Parameters:

vm – virtual machine instance

BERRY_API **void** be_pushbool (bvm *vm, **int** b)

Push a Boolean value onto the virtual stack.

Push a Boolean value onto the virtual stack. The parameter b is the boolean value to be pushed onto the stack. When the value is 0, it means false, otherwise it is true.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **b** –

BERRY_API **void** be_pushint (bvm *vm, bint i)

Push an integer value i onto the virtual stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **i** –

BERRY_API **void** be_pushreal (bvm *vm, breal r)

Push a floating point value r onto the virtual stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **r** –

BERRY_API **void** be_pushstring (bvm *vm, **const char** *str)

Push the string str onto the virtual stack.

Push the string str onto the virtual stack. The parameter str must point to a C string that ends with a null character '\0', and a null pointer cannot be passed in.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **str** –

BERRY_API **void** be_pushnstring (bvm *vm, **const char** *str, size_t n)

Push the string str of length n onto the virtual stack.

Push the string str of length n onto the virtual stack. The length of the string is subject to the parameter n, and the null character is not used as the end mark of the string.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **str** –
- **n** –

BERRY_API **const char** *be_pushfstring (bvm *vm, **const char** *format, ...)

Push the formatted string into the virtual stack.

Push the formatted string into the virtual stack. The parameter format is a formatted string, which contains the text to be pushed onto the stack, and the format parameter contains a label, which can be replaced by the value specified by the subsequent additional parameter and formatted as required. According to the label of the format string, a series of additional parameters may be required, and each additional parameter will replace the corresponding % label in the format parameter.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **format** –

Returns: (???)

BERRY_API **void** *be_pushbuffer (bvm *vm, size_t size)
(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **size** –

BERRY_API **void** be_pushvalue (bvm *vm, **int** index)

Push the value with index index onto the top of the virtual stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** –

```
BERRY_API void be_pushclosure (bvm *vm, void *cl)
(???)
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **cl** –

```
BERRY_API void be_pushntvclosure (bvm *vm, bntvfunc f, int nupvals)
```

Push a native closure onto the top of the virtual stack.

Push a native closure onto the top of the virtual stack. The parameter f is the C function pointer of the native closure, and nupvals is the upvalue number of the closure.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **f** –
- **nupvals** –

```
BERRY_API void be_pushntvfunction (bvm *vm, bntvfunc f)
```

Push a native function onto the top of the virtual stack, and the parameter f is the native function pointer.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **f** –

```
BERRY_API void be_pushclass (bvm *vm, const char *name, const bnfinfo *lib)
```

Push a native class onto the top of the virtual stack.

Push a native class onto the top of the virtual stack. The parameter name is the name of the native class, and the parameter lib is the attribute description of the native class.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **name** –
- **lib** –

BERRY_API **void** be_pushntvclass (bvm *vm, **const struct** bclass *c)
(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **c** –

BERRY_API **void** be_pushcomptr (bvm *vm, **void** *ptr)

Push a general pointer onto the top of the virtual stack.

Push a general pointer onto the top of the virtual stack. The general pointer ptr points to a certain C data area. Since the content pointed to by this pointer is not maintained by Berry's garbage collector, users have to maintain the life cycle of the data themselves.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **ptr** –

BERRY_API bbool be_pushiter (bvm *vm, **int** index)

Push an iterator onto the top of the virtual stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** –

Returns: (???)

BERRY_API **void** be_newlist (bvm *vm)

creates a new list value

After this function is successfully called, the new list value will be pushed onto the top of the stack. list value is an internal representation of a list, not to be confused with an instance of the list class.

Note

FFI function

Parameters: **vm** – virtual machine instance

BERRY_API **void** be_newmap (bvm *vm)

creates a new map value

After this function is successfully called, the new map value will be pushed onto the top of the stack. map value is an internal representation of a list, not to be confused with an instance of the map class.

Note

FFI function

Parameters: **vm** – virtual machine instance

```
BERRY_API void be_newmodule (bvm *vm)
(???)
```

Note

FFI function

Parameters: **vm** – virtual machine instance

```
BERRY_API void be_newcomobj (bvm *vm, void *data, bntvfunc destroy)
(???)
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **data** –
- **destroy** –

```
BERRY_API void be_newobject (bvm *vm, const char *name)
(???)
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **name** –

```
BERRY_API bbool be_copy (bvm *vm, int index)
(???)
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** –

Returns:

```
BERRY_API bbool be_setname (bvm *vm, int index, const char *name)
(???)
```

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** –
- **name** –

Returns: (???)

BERRY_API bbool be_getglobal(bvm *vm, const char *name)

pushes the global variable with the specified name onto the stack

After this function is called, the global variable named name will be pushed onto the top of the virtual stack

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **name** –

Returns: (???)

BERRY_API void be_setglobal(bvm *vm, const char *name)
(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **name** –

BERRY_API bbool be_getbuiltin(bvm *vm, const char *name)
(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **name** –

Returns: (???)

BERRY_API bbool be_setmember(bvm *vm, int index, const char *k)

set the value of the member variable of the instance object class

This function will copy the value at the top of the stack to the member k of the index position instance. Note that the top element of the stack will not pop up automatically.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – index of the instance object
- **k** – name of the member

Returns: (???)

BERRY_API bbool be_getmember (bvm *vm, **int** index, **const char** *k)

get the value of the member variable of the instance object class

This function pushes the value of the member of the index position instance k onto the top of the virtual stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – index of the instance object
- **k** – name of the member

Returns: (???)

BERRY_API bbool be_getmethod (bvm *vm, **int** index, **const char** *k)

(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** –
- **k** –

Returns: (???)

BERRY_API bbool be_getindex (bvm *vm, **int** index)

get the value of list or map

This function is used to get an element from the map or list container (internal values, not instances of map or list classes), and the index of the element is stored at the top of the stack (relative index is -1). After calling this function, the value obtained from the container will be pushed onto the top of the stack. If there is no subscript pointed to by the container, the value of nil will be pushed onto the top of the stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – index of the object to be operated

Returns: (???)

BERRY_API bbool be_setindex (bvm *vm, **int** index)

set a value in list or map

This function is used to write an element of the map or list container. The index of the value to be written in the virtual stack is -1, and the index of the subscript of the write position in the virtual stack is -2. If the element with the specified subscript does not exist in the container, the write operation will fail.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – index of the object to be operated

Returns: (???)

BERRY_API **void** be_getupval (bvm *vm, **int** index, **int** pos)
read an Up Value of the native closure
The read Up Value will be pushed onto the top of the virtual stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – the native closure index value of the Up Value to be read
- **pos** – position of the Up Value in the native closure Up Value table (numbering starts from 0)

BERRY_API **bbool** be_setupval (bvm *vm, **int** index, **int** pos)
set an Up Value of the native closure.
This function obtains a value from the top of the virtual stack and writes it to the target Up Value. After the operation is completed, the top value of the stack will not be popped from the stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – the native closure index value of the Up Value to be read
- **pos** – position of the Up Value in the native closure Up Value table (numbering starts from 0)

Returns: (???)

BERRY_API **bbool** be_setsuper (bvm *vm, **int** index)
(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** –

Returns: (???)

BERRY_API **void** be_getsuper (bvm *vm, **int** index)
get the parent object of the base class or instance of the class.

If the value at index is a class with a base class, the function will push its base class onto the top of the stack; if the value at index is an object with a parent object, the function will take its parent. The object is pushed onto the top of the stack; otherwise, a value of nil is pushed onto the top of the stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – the class or object to be operated

BERRY_API **int** be_data_size (bvm *vm, **int** index)

get the number of elements contained in the container

If the value at index is a Map value or List value, the function returns the number of elements contained in the container, otherwise it returns -1.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – index of the container object to be operated

Returns: (???)

BERRY_API **void** be_data_push (bvm *vm, **int** index)

append a new element to the end of the container.

The object at index must be a List value. This function gets a value from the top of the stack and appends it to the end of the container. After the operation is completed, the value at the top of the stack will not be popped from the stack.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – index of the container object to be operate

BERRY_API **bbool** be_data_insert (bvm *vm, **int** index)

insert a pair of elements into the container

The object at index must be a List value or a Map value. The inserted element forms a pair of key-value pairs. The value is stored at the top of the stack, and the key is stored at the previous index on the top of the stack. It should be noted that the key inserted into the Map container cannot be a nil value, and the key inserted into the List container must be an integer value. If the operation is successful, the function will return btrue, otherwise it will return bfalse.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – container object to be operated

Returns: (???)

BERRY_API bbool be_data_remove (bvm *vm, **int** index)

remove an element in the container

The object at index must be a List value or Map value. For the Map container, the key to delete the element is stored on the top of the virtual stack (need to be pressed before the function call); for the List container, the index of the element to be deleted is stored on the top of the virtual stack (need to be before the function call) push into). If the operation is successful, the function will return btrue, otherwise it will return bfalse.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – container object to be operated.

Returns: (???)

BERRY_API bbool be_data_merge (bvm *vm, **int** index)

(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** –

Returns: (???)

BERRY_API **void** be_data_resize (bvm *vm, **int** index)

reset the capacity of the container

This function is only available for List containers, and the new capacity is stored on the top of the virtual stack (must be an integer).

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – container object to be operated

BERRY_API **void** be_data_reverse (bvm *vm, **int** index)

(???)

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** –

BERRY_API **int** be_iter_next (bvm *vm, **int** index)

get the next element of the iterator

The iterator object may be an iterator of a List container or a Map container. For the List iterator, this function pushes the iteration result value onto the top of the stack, while for the Map iterator, it pushes the key value into

the previous position and the top of the stack respectively. Calling this function will update the iterator. If the function returns 0, the call fails, returning 1 to indicate that the current iterator is a List iterator, and returning 2 to indicate that the current iterator is a Map iterator.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – iterator to be operated

Returns: (???)

BERRY_API bbool be_iter_hasnext (bvm *vm, **int** index)

test whether there is another element in the iterator

The iterator object may be an iterator of a List container or a Map container. If there are more iterable elements in the iterator, return 1, otherwise return 0.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – iterator to be operated

Returns: (???)

BERRY_API bbool be_refcontains (bvm *vm, **int** index)

test whether there is a reference to the specified object in the reference stack

This function is used to test whether there is a reference to the specified object in the reference stack. It must be used in conjunction with be_refpush and be_refpop. This API can avoid recursion when traversing objects that have their own references. This function is used for the value of an instance type. If there is a reference to the object in the reference stack, it returns 1, otherwise it returns 0.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – object to be operated

Returns: (???)

BERRY_API **void** be_refpush (bvm *vm, **int** index)

Push the reference of the specified object onto the reference stack.

This function is used for the value of an instance type.

Note

FFI function

Parameters:

- **vm** – virtual machine instance
- **index** – object to be operated

BERRY_API **void** be_refpop (bvm *vm)

Pop the object at the top of the reference stack.
This function is used in pairs with `be_repush`

Note

FFI function

Parameters: `vm` – virtual machine instance

BERRY_API `void` `be_stack_require` (`bvm *vm`, `int` `count`)
tests the amount of free space on the stack and expands the stack space if it is insufficient
If the free capacity of the virtual stack allocated by the VM to the native function is lower than this value, an expansion operation will be performed.

Note

FFI function

Parameters:

- `vm` – virtual machine instance
- `count` – required free stack capacity.

BERRY_API `bbool` `be_getmodule` (`bvm *vm`, `const char` `*k`)
(???)

Note

FFI function

Parameters:

- `vm` – virtual machine instance
- `k` –

Returns: (???)

BERRY_API `bbool` `be_iseq` (`bvm *vm`)
(???)

Note

relop operation API

Parameters: `vm` – virtual machine instance

Returns: (???)

BERRY_API `bbool` `be_isneq` (`bvm *vm`)
(???)

Note

relop operation API

Parameters: `vm` – virtual machine instance

Returns: (???)

BERRY_API `bbool` `be_islt` (`bvm *vm`)

(???)

Note

relop operation API

Parameters: **vm** – virtual machine instance

Returns: (???)

BERRY_API bbool be_isle (bvm *vm)
(???)

Note

relop operation API

Parameters: **vm** – virtual machine instance

Returns: (???)

BERRY_API bbool be_isgt (bvm *vm)
(???)

Note

relop operation API

Parameters: **vm** – virtual machine instance

Returns: (???)

BERRY_API bbool be_isge (bvm *vm)
(???)

Note

relop operation API

Parameters: **vm** – virtual machine instance

Returns: (???)

BERRY_API **int** be_returnvalue (bvm *vm)
(???)

Note

Function call/return API

Parameters: **vm** – virtual machine instance

Returns: (???)

BERRY_API **int** be_returnnilvalue (bvm *vm)
(???)

Note

Function call/return API

Parameters: **vm** – virtual machine instance

Returns: (???)

```
BERRY_API void be_call (bvm *vm, int argc)
(???)
```

Note

Function call/return API

Parameters:

- **vm** – virtual machine instance
- **argc** –

```
BERRY_API int be_pcall (bvm *vm, int argc)
(???)
```

Note

Function call/return API

Parameters:

- **vm** – virtual machine instance
- **argc** –

Returns: (???)

```
BERRY_API void be_exit (bvm *vm, int status)
(???)
```

Note

Function call/return API

Parameters:

- **vm** – virtual machine instance
- **status** –

```
BERRY_API void be_raise (bvm *vm, const char *except, const char *msg)
(???)
```

Note

exception API

Parameters:

- **vm** – virtual machine instance
- **except** –
- **msg** –

```
BERRY_API int be_getexcept (bvm *vm, int code)
(???)
```

Note

exception API

Parameters:

- **vm** – virtual machine instance
- **code** –

Returns: (???)

BERRY_API **void** be_dumpvalue (bvm *vm, **int** index)
(???)

Note

exception API

Parameters:

- **vm** – virtual machine instance
- **index** –

BERRY_API **void** be_dumpexcept (bvm *vm)
(???)

Note

exception API

Parameters: **vm** – virtual machine instance

BERRY_API **void** be_stop_iteration (bvm *vm)
(???)

Note

exception API

Parameters: **vm** – virtual machine instance

BERRY_API **void** be_regfunc (bvm *vm, **const char** *name, bntvfunc f)
register a native function

The specific behavior of this function is related to the value of the BE_USE_PRECOMPILED_OBJECT macro (although the FFI is still available when using the compile-time construction technique, it cannot dynamically register the built-in variables. In this case, please refer to the method of registering the built-in objects.

Note

exception API

Parameters:

- **vm** – virtual machine instance
- **name** – name of the native function
- **f** – pointer of the native function

BERRY_API **void** be_regclass (bvm *vm, **const char** *name, **const** bnfuncinfo *lib)
(???)

Note

exception API

Parameters:

- **vm** – virtual machine instance
- **name** –
- **lib** –

BERRY_API bvm *be_vm_new(**void**)
Construct a VM.

Note

VM management API

Returns: (???)

BERRY_API **void** be_vm_delete(bvm *vm)
Destroy a VM.

Note

VM management API

Parameters: **vm** – virtual machine instance

BERRY_API **void** be_set_obs_hook(bvm *vm, bobshook hook)
(???)

Note

Observability hook

Parameters:

- **vm** – virtual machine instance
- **hook** –

BERRY_API **void** be_set_ctype_func_handler(bvm *vm, bctypefunc handler)
(???)

Note

Observability hook

Parameters:

- **vm** – virtual machine instance
- **handler** –

BERRY_API bctypefunc be_get_ctype_func_handler(bvm *vm)
(???)

Note

Observability hook

Parameters: **vm** – virtual machine instance

Returns: (???)

BERRY_API **int** be_loadbuffer (bvm *vm, **const char** *name, **const char** *buffer, size_t length)
load a piece of source code from the buffer and compile it into bytecode
if the compilation is successful, be_loadbuffer will compile the source code into a Berry function and place it on the top of the virtual stack. If the compilation encounters an error, be_loadbuffer will return an error value of type berrorcode (Section errorcode), and if possible, will store the specific error message string at the top of the virtual stack.

Note

code load API

Parameters:

- **vm** – virtual machine instance
- **name** – string, which is usually used to mark the source of the source code
- **buffer** – buffer for storing the source code
- **length** – length of the buffer

Returns: (???)

BERRY_API **int** be_loadmode (bvm *vm, **const char** *name, bbool islocal)
(???)

Note

code load API

Parameters:

- **vm** – virtual machine instance
- **name** –
- **islocal** –

Returns: (???)

BERRY_API **int** be_loadlib (bvm *vm, **const char** *path)
(???)

Note

code load API

Parameters:

- **vm** – virtual machine instance
- **path** –

Returns: (???)

BERRY_API **int** be_savecode (bvm *vm, **const char** *name)
(???)

Note

code load API

Parameters:

- **vm** – virtual machine instance
- **name** –

Returns: (???)

```
BERRY_API void be_module_path (bvm *vm)
(???)
```

Note

module path list API

Parameters: **vm** – virtual machine instance

```
BERRY_API void be_module_path_set (bvm *vm, const char *path)
(???)
```

Note

module path list API

Parameters:

- **vm** – virtual machine instance
- **path** –

```
BERRY_API void *be_pushbytes (bvm *vm, const void *buf, size_t len)
Push a bytes() buffer.
```

Note

bytes operation

Parameters:

- **vm** – virtual machine instance
- **buf** – starting at position
- **len** – size

```
BERRY_API const void *be_tobytes (bvm *vm, int index, size_t *len)
return virtual stack as a bytes buffer
```

Get the value of the index position of index from the virtual stack and return it as a bytes buffer. The pointer of the buffer is returned, and the size is stored in *len (unless len is NULL). This function works only for instances of the bytes class, or returns NULL.

Note

bytes operation

Parameters:

- **vm** – virtual machine instance
- **index** – index from the virtual stac
- **len** – size

BERRY_API **void** be_sethook (bvm *vm, **const char** *mask)
(???)

Note

debug API

Parameters:

- **vm** – virtual machine instance
- **mask** –

BERRY_API **void** be_setntvhook (bvm *vm, bntvhook hook, **void** *data, **int** mask)
(???)

Note

debug API

Parameters:

- **vm** – virtual machine instance
- **hook** –
- **data** –
- **mask** –

BERRY_API **void** be_writebuffer (**const char** *buffer, size_t length)
implement on berry_port.c

Output a piece of data to the standard output device, the parameter buffer is the first address of the output data block, and length is the length of the output data block. This function outputs to the stdout file by default. Inside the interpreter, this function is usually used as a character stream output, not a binary stream.

Note

basic character IO API

Parameters:

- **buffer** –
- **length** –

BERRY_API **char** *be_readstring (**char** *buffer, size_t size)
implement on berry_port.c

Input a piece of data from the standard input device, and read at most one row of data each time this function is called. The parameter buffer is the data buffer passed in by the caller, and the capacity of the buffer is size. This function will stop reading and return when the buffer capacity is used up, otherwise it will return when a newline character or end of file character is read. If the function executes successfully, it will directly use the buffer parameter as the return value, otherwise it will return NULL.

This function will add the read line breaks to the read data, and each time the be_readstring function is called, it will continue to read from the current position. This function is only called in the implementation of the native function input, and the be_readstring function may not be implemented when it is not necessary.

Note

basic character IO API

Parameters:

- **buffer** –
- **size** –

Returns: (???)

struct bnfuncinfo
native function information

Public Members

const char *name
name

bntvfunc function
function

struct bntvmodobj
bntvmodobj

Public Members

const char *name
name

int type
type

union bntvmodobj::value u
u

union value
< value

Public Members

bint i
i

brear r
r

bbool b
b

bntvfunc f
f

const char *s
s

const void *o
o

struct bntvmodule
bntvmodule

Public Members

const char *name
native module name

const bntvmodobj_t *attrs
native module attributes

size_t size
native module attribute count

Welcome to berry's documentation!

```
const struct bmodule *module
    const module object

struct bhookinfo
    bhookinfo
```

Public Members

```
int type
    current hook type

int line
    current line number

const char *source
    source path information

const char *func_name
    current function name

void *data
    user extended data
```

Index

B

[bbool](#) (C macro)
[bbyte](#) (C++ type)
[bclass_array](#) (C++ type)
[bclass_ptr](#) (C++ type)
[bctypfunc](#) (C++ type)
[be_absindex](#) (C++ function)
[be_assert](#) (C macro)
[be_call](#) (C++ function)
[BE_CBOOL](#) (C macro)
[BE_CFUNCTION](#) (C macro)
[BE_CINT](#) (C macro)
[be_classname](#) (C++ function)
[be_classof](#) (C++ function)
[BE_CMODULE](#) (C macro)
[BE_CNIL](#) (C macro)
[be_copy](#) (C++ function)
[BE_CREAL](#) (C macro)
[BE_CSTRING](#) (C macro)
[be_data_insert](#) (C++ function)
[be_data_merge](#) (C++ function)
[be_data_push](#) (C++ function)
[be_data_remove](#) (C++ function)
[be_data_resize](#) (C++ function)
[be_data_reverse](#) (C++ function)
[be_data_size](#) (C++ function)
[be_define_local_closure](#) (C macro)
[be_define_local_const_str](#) (C macro)
[be_define_local_proto](#) (C macro)
[be_define_native_module](#) (C macro)
[be_dostring](#) (C macro)
[be_dumpexcept](#) (C++ function)
[be_dumpvalue](#) (C++ function)
[be_exit](#) (C++ function)
[BE_EXPORT_VARIABLE](#) (C macro)
[be_extern_native_class](#) (C macro)
[be_extern_native_module](#) (C macro)
[be_get_ctype_func_handler](#) (C++ function)
[be_getbuiltin](#) (C++ function)

[be_getexcept](#) (C++ function)
[be_getglobal](#) (C++ function)
[be_getindex](#) (C++ function)
[be_getmember](#) (C++ function)
[be_getmethod](#) (C++ function)
[be_getmodule](#) (C++ function)
[be_getsuper](#) (C++ function)
[be_getupval](#) (C++ function)
[BE_HOOK_CALL](#) (C macro)
[BE_HOOK_EXCEPT](#) (C macro)
[BE_HOOK_LINE](#) (C macro)
[BE_HOOK_RET](#) (C macro)
[BE_IIF](#) (C macro)
[BE_IIF_0](#) (C macro)
[BE_IIF_1](#) (C macro)
[BE_INT_FMTLEN](#) (C macro)
[BE_INT_FORMAT](#) (C macro)
[BE_INTEGER](#) (C macro)
[be_isbool](#) (C++ function)
[be_isbytes](#) (C++ function)
[be_isclass](#) (C++ function)
[be_isclosure](#) (C++ function)
[be_iscomobj](#) (C++ function)
[be_iscompstr](#) (C++ function)
[be_isderived](#) (C++ function)
[be_iseq](#) (C++ function)
[be_isfunction](#) (C++ function)
[be_isge](#) (C++ function)
[be_isgt](#) (C++ function)
[be_isinstance](#) (C++ function)
[be_isint](#) (C++ function)
[be_isle](#) (C++ function)
[be_islist](#) (C++ function)
[be_islistinstance](#) (C++ function)
[be_islt](#) (C++ function)
[be_ismap](#) (C++ function)
[be_ismapinstance](#) (C++ function)
[be_ismodule](#) (C++ function)
[be_isneq](#) (C++ function)
[be_isnil](#) (C++ function)
[be_isntvclos](#) (C++ function)
[be_isnumber](#) (C++ function)

be_isproto (C++ function)	be_pushfstring (C++ function)
be_isreal (C++ function)	be_pushint (C++ function)
be_isstring (C++ function)	be_pushiter (C++ function)
be_iter_hasnext (C++ function)	be_pushnil (C++ function)
be_iter_next (C++ function)	be_pushnstring (C++ function)
be_loadbuffer (C++ function)	be_pushntvclass (C++ function)
be_loadfile (C macro)	be_pushntvclosure (C++ function)
be_loadlib (C++ function)	be_pushntvfunction (C++ function)
be_loadmode (C++ function)	be_pushreal (C++ function)
be_loadmodule (C macro)	be_pushstring (C++ function)
be_loadstring (C macro)	be_pushvalue (C++ function)
be_local_closure (C macro)	be_raise (C++ function)
be_local_const_str (C macro)	be_readstring (C++ function)
be_local_const_upval (C macro)	be_refcontains (C++ function)
be_module_path (C++ function)	be_refpop (C++ function)
be_module_path_set (C++ function)	be_refpush (C++ function)
be_moveto (C++ function)	be_regclass (C++ function)
be_native_class (C macro)	be_regfunc (C++ function)
be_native_module (C macro)	be_remove (C++ function)
be_native_module_attr_table (C macro)	be_return (C macro)
be_native_module_bool (C macro)	be_return_nil (C macro)
be_native_module_function (C macro)	be_returnnilvalue (C++ function)
be_native_module_int (C macro)	be_returnvalue (C++ function)
be_native_module_module (C macro)	be_savecode (C++ function)
be_native_module_nil (C macro)	be_set_ctype_func_handler (C++ function)
be_native_module_real (C macro)	be_set_obs_hook (C++ function)
be_native_module_str (C macro)	be_setglobal (C++ function)
be_nested_const_str (C macro)	be_sethook (C++ function)
be_nested_proto (C macro)	be_setindex (C++ function)
be_newcomobj (C++ function)	be_setmember (C++ function)
be_newlist (C++ function)	be_setname (C++ function)
be_newmap (C++ function)	be_setntvhook (C++ function)
be_newmodule (C++ function)	be_setsuper (C++ function)
be_newobject (C++ function)	be_setupval (C++ function)
be_pcall (C++ function)	be_stack_require (C++ function)
be_pop (C++ function)	be_stop_iteration (C++ function)
be_pushbool (C++ function)	be_str2int (C++ function)
be_pushbuffer (C++ function)	be_str2num (C++ function)
be_pushbytes (C++ function)	be_str2real (C++ function)
be_pushclass (C++ function)	be_strconcat (C++ function)
be_pushclosure (C++ function)	be_strlen (C++ function)
be_pushcomprr (C++ function)	be_tobool (C++ function)

be_tobytes (C++ function)
 be_tocomptr (C++ function)
 be_toescape (C++ function)
 be_toindex (C++ function)
 be_toint (C++ function)
 be_top (C++ function)
 be_toreal (C++ function)
 be_tostring (C++ function)
 be_typename (C++ function)
 be_vm_delete (C++ function)
 be_vm_new (C++ function)
 be_writebuffer (C++ function)
 be_writenewline (C macro)
 be_writestring (C macro)
 beobshookevents (C++ enum)
 beobshookevents::BE_OBS_GC_END (C++ enumerator)
 beobshookevents::BE_OBS_GC_START (C++ enumerator)
 beobshookevents::BE_OBS_MALLOC_FAIL (C++ enumerator)
 beobshookevents::BE_OBS_PCALL_ERROR (C++ enumerator)
 beobshookevents::BE_OBS_STACK_RESIZE_START (C++ enumerator)
 beobshookevents::BE_OBS_VM_HEARTBEAT (C++ enumerator)
 berrorcode (C++ enum)
 berrorcode::BE_EXCEPTION (C++ enumerator)
 berrorcode::BE_EXEC_ERROR (C++ enumerator)
 berrorcode::BE_EXIT (C++ enumerator)
 berrorcode::BE_IO_ERROR (C++ enumerator)
 berrorcode::BE_MALLOC_FAIL (C++ enumerator)
 berrorcode::BE_OK (C++ enumerator)
 berrorcode::BE_SYNTAX_ERROR (C++ enumerator)
 BERRY_API (C macro)
 BERRY_LOCAL (C macro)
 BERRY_VERSION (C macro)
 bfalse (C macro)
 bhookinfo (C++ struct)
 bhookinfo::data (C++ member)
 bhookinfo::func_name (C++ member)
 bhookinfo::line (C++ member)
 bhookinfo::source (C++ member)

bhookinfo::type (C++ member)
 bhookinfo_t (C++ type)
 bint (C++ type)
 bnfuncinfo (C++ struct)
 bnfuncinfo::function (C++ member)
 bnfuncinfo::name (C++ member)
 bntvfunc (C++ type)
 bntvhook (C++ type)
 bntvmodobj (C++ struct)
 bntvmodobj::name (C++ member)
 bntvmodobj::type (C++ member)
 bntvmodobj::u (C++ member)
 bntvmodobj::value (C++ union)
 bntvmodobj::value::b (C++ member)
 bntvmodobj::value::f (C++ member)
 bntvmodobj::value::i (C++ member)
 bntvmodobj::value::o (C++ member)
 bntvmodobj::value::r (C++ member)
 bntvmodobj::value::s (C++ member)
 bntvmodobj_t (C++ type)
 bntvmodule (C++ struct)
 bntvmodule::attrs (C++ member)
 bntvmodule::module (C++ member)
 bntvmodule::name (C++ member)
 bntvmodule::size (C++ member)
 bntvmodule_t (C++ type)
 bobshook (C++ type)
 breal (C++ type)
 btrue (C macro)
 bvm (C++ type)

P

PROTO_RUNTIME_BLOCK (C macro)
 PROTO_SOURCE_FILE (C macro)
 PROTO_SOURCE_FILE_STR (C macro)
 PROTO_VAR_INFO_BLOCK (C macro)